

UNIVERSITÀ DEGLI STUDI DI PADOVA

Dipartimento di Fisica e Astronomia "Galileo Galilei"

Master Degree in Physics of Data

Final Dissertation

GPU-Accelerated Triggerless Data Acquisition Pipeline for Machine Learning-Based Anomaly Detection

Thesis supervisor

Prof. Jacopo Pazzini

Thesis co-supervisor

Dr. Alberto Zucchetta

Candidate

Pietro Bernardi

ID: 2097494

Academic Year 2025/2026

Abstract

This thesis presents a GPU-accelerated triggerless data acquisition (DAQ) pipeline to perform a specialized task of Data Quality Monitoring (DQM), that is Anomaly Detection, in the context of a prototype muon particle detector operated with cosmic-ray muons.

DQM refers to the set of procedures, software tools, and monitoring workflows used to assess the quality of experimental data both during acquisition and during subsequent offline processing. It is an essential component of complex DAQ systems, such as those that one could find, for instance, in the field of experimental high-energy particle physics. In these contexts DQM is essential to identify abnormal detector behavior and to provide feedback, either during data taking or offline when performing further processing.

The pipeline developed in this work is designed to remain GPU-resident across its three processing stages that perform each a specialized task: pre-processing, reconstruction and anomaly detection for DQM. Because of the triggerless property, no conventional decision signal produced by a hardware-level trigger is used to select events. Instead that specific task is assigned to the software. The detector streams zero-suppressed data continuously towards the input of the pipeline, where it is accumulated into batches of configurable size.

The system is operated in online mode, meaning that processing begins once a fully populated batch becomes available. The pre-processing stage transforms the low-level information contained in each input batch into a set of candidate events also acting as a software trigger. The reconstruction stage then employs custom-made GPU kernels to reconstruct particle tracks for each of the candidates furnishing a dataset with higher-level features.

This dataset is consumed by the DQM stage, which performs anomaly detection through a reference-based multivariate two-sample test, numerically implemented using a scalable kernel-based machine-learning algorithm. After an initial calibration under the null hypothesis, the test compares a sample drawn from the reconstructed-feature dataset of the input batch with that drawn from a nominal reference sample and returns a scalar test statistic to which a p-value is assigned. The p-value quantifies the degree of compatibility between the observed batch and the reference condition under the null hypothesis. To gain further insights on the possible origin of the anomalies, a lightweight classification layer is attached at the end to associate detected anomalies with known injected classes.

Different GPU libraries are used across the pipeline stages, and a unified GPU memory-management strategy is adopted to support interoperability, efficient memory reuse, and stable long-term execution. The performance of the system is evaluated in terms of throughput, memory stability, and anomaly-detection sensitivity.

The results show that the proposed pipeline can detect anomalous detector conditions with high sensitivity while sustaining input rates well above those expected from the cosmic-ray muon data used in this study. This demonstrates the feasibility of a GPU-resident, reference-based anomaly-detection pipeline for online DQM and motivates its possible extension to higher-throughput detector applications.

Contents

1 Introduction **1**
- 1.1 Data Quality Monitoring in experimental physics . . . 1
  - 1.1.1 Anomaly detection for DQM . . . 1
- 1.2 Offline DQM . . . 2
- 1.3 Online DQM and timing constraints . . . 3
  - 1.3.1 Latency and throughput . . . 3
  - 1.3.2 Processing speed and statistical reliability . . . 3
  - 1.3.3 Usage of computing accelerators . . . 5
  - 1.3.4 Relevance to this work . . . 5

2 Anomaly Detection **7**
- 2.1 Introduction . . . 7
- 2.2 Classical approaches . . . 8
  - 2.2.1 Threshold-based and histogram-based anomaly detection . . . 9
- 2.3 Statistical methods . . . 10
  - 2.3.1 General introduction to the problem and scope of the current work . . . 10
  - 2.3.2 Goodness-of-Fit formulation and Likelihood-ratio testing . . . 11
  - 2.3.3 NPLM test statistic . . . 12
  - 2.3.4 Falkon: fast large-scale kernel solver based on Nyström approximation . . . 15

3 Experimental setup **19**
- 3.1 Muon detector prototype . . . 19
  - 3.1.1 Introduction . . . 19
  - 3.1.2 Drift chamber and muon tracks . . . 20
- 3.2 Data Acquisition system . . . 21
  - 3.2.1 Front-End (FE) and Back-End (BE) electronics . . . 21
  - 3.2.2 Raw hit rate . . . 21
- 3.3 Anomaly detection for DQM system . . . 22

4 Pipeline implementation and validation **23**
- 4.1 Pre-Processing block . . . 23
  - 4.1.1 Architecture and Data model . . . 23
  - 4.1.2 Time calibration . . . 25
  - 4.1.3 Implementation and Performance analysis . . . 26
- 4.2 Track reconstruction . . . 29
  - 4.2.1 The left-right ambiguity problem . . . 29
  - 4.2.2 Initial assumptions . . . 30
  - 4.2.3 Geometrical track model . . . 31
  - 4.2.4 Track reconstruction algorithm . . . 32
  - 4.2.5 Track reconstruction algorithm implementation with CUDA kernels . . . 38
  - 4.2.6 Performance Analysis . . . 49
- 4.3 Anomaly detection and DQM block . . . 56

4.3.1 FalkonML implementation . . . 56
4.3.2 Falkon's theory recall: hyperparameters and calibration procedure . . . 60
4.3.3 Calibration results (reference-vs-reference) . . . 61
4.3.4 Altered-data results (reference-vs-data) . . . 65
4.3.5 Performance analysis . . . 70
4.4 Anomaly classification head . . . 73
4.4.1 Bayesian density-based classifier . . . 73
4.4.2 XGBoost classifier . . . 76

5 Pipeline memory management and latency 79
5.1 Memory management . . . 79
5.1.1 Introduction . . . 79
5.1.2 Rapids Memory Manager (RMM) . . . 80
5.1.3 Pipeline memory stability analysis . . . 80
5.1.4 Monitoring dashboard . . . 88
5.2 Latency and online processing . . . 88
5.2.1 Latency and input rate . . . 88
5.2.2 Minimum latency . . . 89
5.2.3 Final remarks on the online capabilities of the pipeline . . . 91

6 Conclusions and Outlook 93
6.1 Main results . . . 93
6.2 Limitations of the present study . . . 93
6.3 Outlook . . . 94

Bibliography 95

Appendices 96

A Additional material 97
A.1 Slowdown for pre-processing and track-reconstruction . . . 97
A.2 Extra fit results . . . 98
A.3 GPU vs NumPy track reconstruction . . . 100

B Technical notes 101
B.1 Kernel profilings . . . 101
B.1.1 Combinatorial kernel . . . 101
B.1.2 Track selection kernel . . . 103
B.1.3 Event builder kernel . . . 104
B.1.4 Track merger kernel . . . 105
B.2 FalkonML modifications . . . 106
B.2.1 RMM allocator related changes . . . 106
B.2.2 GPU-residency related . . . 107

C Pipeline monitoring dashboard 111
C.1 Main features . . . 111
C.2 Remarks . . . 111

List of Figures

1.1 Simple DAQ system . . . 2
1.2 Concept of granularity . . . 3

2.1 Simplified diagram of anomaly-detection for DQM . . . 7
2.2 CMS experiment DQM systems . . . 8
2.3 D'Agnolo and Wulzer [8] . . . 10
2.4 Test statistic calibration procedure . . . 13
2.5 Nyström approximation . . . 16

3.1 Experimental apparatus . . . 19
3.2 Schematic diagram of the drift chamber. . . . 20
3.3 Time measurement system . . . 21
3.4 Scintillator arrangement . . . 22

4.1 Main steps of the preprocessing stage. . . . 24
4.2 Timebox calibration procedure showing both the initial and final, corrected, timeboxes and the construction used to determine the value for t_c. . . . 25
4.3 Preprocessor performances . . . 27
4.4 Details on GPU execution time measurements . . . 28
4.5 Fluctuations in GPU time measurements . . . 29
4.6 Left and right ambiguity . . . 30
4.7 Detector operating conditions . . . 30
4.8 Detector logical track volumes . . . 31
4.9 Maximum observable track angle . . . 32
4.10 Schematic representation of track reconstruction algorithm . . . 33
4.11 Penalty function . . . 36
4.12 Schematic diagram of the GPU kernel chain . . . 38
4.13 Input shaper kernel . . . 39
4.14 Thread to Detector cell mapping . . . 40
4.15 Combinatorial kernel thread organization . . . 41
4.16 Shuffle operations . . . 42
4.17 Detector view and shuffle operations . . . 42
4.18 Detector zoomed-in region (1) . . . 42
4.19 Detector zoomed-in region (2) . . . 43
4.20 Track geometry doublets . . . 44
4.21 Track object . . . 44
4.22 Combinatorial kernel output memory organization (1) . . . 46
4.23 Combinatorial kernel output memory organization (2) . . . 46
4.24 XOR shuffle operation . . . 47
4.25 Example of collision between tracks . . . 47
4.26 Track selection kernel output memory organization . . . 48
4.27 Track angle definition . . . 49

4.28 Schematic of the partial synchronous pipeline with the pre-processor and event reconstruction kernel chain. . . . 49
4.29 Performance of pre-processor and track-reconstruction blocks . . . 50
4.30 Distributions related to reconstructed tracks . . . 51
4.31 Event with a single track and 6 total recorded hit-words. . . . 51
4.32 Event with a single track and 8 total recorded hit-words. . . . 51
4.33 Event with two parallel tracks and 11 total recorded hit-words. . . . 52
4.34 Example of a reconstructed event with three almost parallel tracks and 16 total recorded hit-words. . . . 52
4.35 Example of a reconstructed event with four tracks and 25 total recorded hit-words. . . . 53
4.36 CPU vs GPU reconstruction outcomes . . . 53
4.37 CPU vs GPU event-per-event comparison . . . 55
4.38 Synchronous pipeline diagram . . . 56
4.39 Memory usage differences between CPU and GPU FalkonML . . . 57
4.40 Drift times distributions for the four layers. . . . 58
4.41 Drift time as a function of track angle for each layer. . . . 59
4.42 Adjacent layer correlations. . . . 60
4.43 Adjacent layer sums. . . . 60
4.44 Pairwise euclidean distance distribution on the standardized input dataset. . . . 63
4.45 Test statistic distributions in fp64 . . . 64
4.46 Test statistic distributions in fp32 . . . 65
4.47 Pairwise separations in fp64 mode (1) . . . 66
4.48 Pairwise separations in fp64 mode (2) . . . 67
4.49 Test statistic distributions in fp64 mode . . . 67
4.50 Pairwise separations in fp32 mode (1) . . . 68
4.51 Pairwise separations in fp32 mode (2) . . . 69
4.52 Test statistic distributions in fp32 mode . . . 69
4.53 Sampling and training times for Falkon. . . . 71
4.54 Falkon input latency . . . 71
4.55 Test statistic throughputs in fp64 and fp32 modes . . . 72
4.56 Test statistic distributions in fp64 and large model . . . 74
4.57 Confusion matrix for the Bayesian density-based classifier. . . . 75

5.1 GPU-memory allocation . . . 79
5.2 Unified GPU-memory allocation . . . 80
5.3 Pipeline memory pressure with and without the usage of RMM. . . . 81
5.4 Pipeline time-trace of memory events RMM enabled . . . 82
5.5 Memory stability analysis . . . 83
5.6 Bootstrap distributions of the peak-memory slopes. . . . 85
5.7 Forest plot of the bootstrap peak-memory drift. . . . 86
5.9 Bootstrap distribution of the peak-memory slopes. . . . 87
5.10 Forest plot of the bootstrap peak-memory drift. . . . 88
5.11 Pipeline maximum acceptable input rate . . . 91

A.1 Slowdowns for preprocessing and track reconstruction. . . . 97

C.1 Dashboard in operation, showing the detection of an anomalous batch. . . . 112

List of Tables

3.1 The various datasets that have been used in this work. The acronyms defined in this table are used across the remainder of this work. . . . 22

4.1 Hit-Word data structure . . . 23
4.2 Schema of the GPU-resident output CuPy array. . . . 25
4.3 Layer-wise calibration fit parameters t_c. . . . 26
4.4 Memory indexing for the input shaper kernel . . . 39
4.5 Combinatorial kernel global memory access pattern. . . . 40
4.6 Track geometry mappings . . . 45
4.7 Final output features. . . . 49
4.8 Agreement between CPU and GPU reconstruction . . . 54
4.9 CPU vs GPU reconstruction event-per-event comparison . . . 55
4.10 Adjacent layer sums numerical results. . . . 60
4.11 Selected models in fp64 mode . . . 64
4.12 Selected models in `fp32` mode. . . . 65
4.13 Median p-values for each anomaly kind for the `fp64` models. . . . 68
4.14 Median p-values for each anomaly kind for the `fp32` models. . . . 70
4.15 Falkon input latency in fp64 mode . . . 72
4.16 Falkon input latency in fp32 mode . . . 72
4.17 Bayesian density-based classifier output . . . 75
4.18 XGBoost classifier output . . . 77

5.1 Linear fits for each run to detect systematic drifts. . . . 84
5.2 Summary of the moving-block bootstrap distributions of the peak-memory drift. . . . 85
5.3 Summary of the moving-block bootstrap distributions of the peak-memory drift in the absence of RMM. . . . 87
5.4 Minimal median times to reconstruct a single track as a function of the aperture when the last sub-batch is fully saturated. . . . 90
5.5 Median times to reconstruct a single track as a function of the aperture for partially saturated last sub-batches. . . . 90
5.6 Pipeline total latency . . . 91

6.1 Summary of results . . . 94

A.1 CPU vs GPU reconstruction statistical test . . . 100

B.1 Kernel profilings summary . . . 101

Chapter 1

Introduction

1.1 Data Quality Monitoring in experimental physics

Data Quality Monitoring (DQM) refers to the set of procedures, software tools, and monitoring workflows used to assess the quality of experimental data both during acquisition and during subsequent offline processing. In particle-physics experiments, DQM is essential because the recorded data are produced by a complex chain involving detector sensors, front-end electronics, trigger and readout systems, data acquisition infrastructure, reconstruction algorithms, and storage systems. Abnormal behavior can therefore originate at many different levels, for example from noisy or inefficient detector channels, synchronization problems, calibration drifts, changes in trigger rates, or failures in the reconstruction chain. By identifying such conditions, DQM plays a central role in protecting the quality of the recorded dataset and in supporting the real-time operation of the detector.

Because of this, DQM is closely connected to the Data Acquisition system (DAQ). The DAQ is in fact responsible for collecting the signals produced by the detector, transporting them through the readout chain, selecting or organizing the events, and making them available for storage and reconstruction. Since each of these stages involves many complex operations, it is sensible to monitor them and so one typically has many DQM layers operating at the various stages of the DAQ and at various levels within a given stage. In this way the system can provide feedback on both the status of the detector and about the quality of the data being collected.

A very simplified schematic is shown in Figure 1.1 where two DQM systems are shown operating at two different levels of the DAQ system of a detector. In real operating systems, the configuration is often much more complex than the one depicted here. In large experiments such as those commissioned at the LHC at CERN one has several layers of DQM systems operating at any given time as described before. In the simplified diagram, only the simplest division between online and offline monitoring was made.

The complexity of the system notwithstanding, it is quite intuitive to see how an online operating system poses several challenges that an offline system typically does not have: that of respecting time constraints. In fact, when a system needs to perform online DQM it needs to generate its feedback and/or decisions within a specific time window so as to not introduce delays in the DAQ chain and remain synchronized with it at all times. More on this will be given in the following.

1.1.1 Anomaly detection for DQM

One kind of DQM that is central for this work is that of Anomaly Detection, namely the identification of data patterns that are not compatible with nominal detector behavior. Traditionally, this is often achieved through predefined monitoring histograms, threshold checks, and comparisons with reference distributions. These approaches are robust and interpretable, but they may be limited when detector anomalies appear as correlated changes in several variables rather than as a large deviation in a single monitored quantity. To enrich this approach, the usage of statistical methods that allow for a

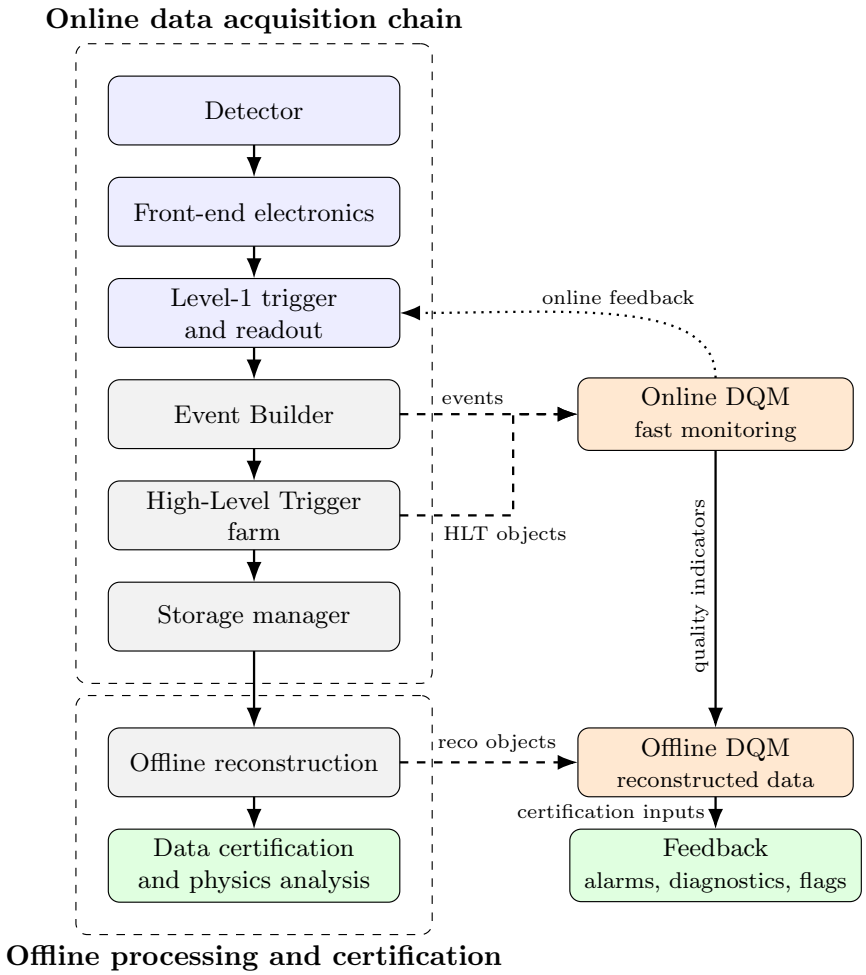


Figure 1.1: Simplified example schematic of a DAQ system with the online and offline DQM systems at a particle detector.

multivariate approach through, for instance, reference-based anomaly-detection is explored. In these cases the current data stream is compared with a validated reference sample representing nominal operation. This is the approach that will be embodied in the object of this work.

1.2 Offline DQM

Offline DQM, is performed after a data-taking period has been completed and the recorded data have been processed by the reconstruction chain. Since it is not subject to the same latency constraints as online monitoring, offline DQM can use more complete reconstruction information. For this reason, it is often used as a validation tool. In particular, it allows one to verify that the data recorded during a given run are compatible with previously already validated data taken under the same nominal conditions, or to reference data that represents the expected detector behavior. In this way one can assess whether the newly collected data satisfy the quality requirements needed for subsequent analysis.

Deviations between the current and reference distributions may indicate changes in detector response, readout behavior, calibration, reconstruction performance, or data-taking conditions. Offline DQM therefore provides a way to validate the stability of the experiment over time and to certify whether a given dataset is suitable for physics analysis.

1.3 Online DQM and timing constraints

As it was previously introduced, among the different DQM scenarios, online monitoring is particularly demanding and this is precisely the use-case to which the object being developed in this work belongs to. In this case, the monitoring system operates while the experiment is taking data and must therefore satisfy the *latency* and *throughput* constraints imposed by the DAQ.

1.3.1 Latency and throughput

The **latency** refers to the amount of time that is needed to accumulate sufficient information to execute the monitoring task. In the case of this work, input data is always accumulated in **batches** and task decisions are always referred to the batch and never to its individual elements.

The **throughput** can instead be referred to either the input or the output sides. The input throughput refers to the rate at which the DQM system is able to process incoming information. The output throughput refers instead to the rate at which the DQM system is able to produce a decision.

The DQM tasks should not introduce significant delays in the acquisition chain, and they have to provide sufficiently fast and informative feedback to detect abnormal detector conditions. This requirement becomes increasingly challenging as modern experiments produce large data rates and high-dimensional monitoring information. For this reason, online DQM is naturally connected to high-throughput computing architectures and to methods capable of processing data streams efficiently, furnishing fast feedback and statistical reliability.

1.3.2 Processing speed and statistical reliability

The tension between fast feedback and statistical reliability can be described in terms of the granularity of the monitoring. In a DQM system, different notions of granularity can be relevant. The *time granularity* specifies the time interval of detector operation summarized by one monitoring output. The *event or object granularity* specifies how many events, hits, tracks, or reconstructed objects enter one monitoring decision. The *spatial granularity* specifies the detector region to which the monitoring result refers, for example a full subsystem, a chamber, a layer, or a single channel. Finally, for algorithms based on statistical comparisons, one can also define a *statistical granularity*, namely the number of data points effectively used in the statistical test. These granularities are not independent in an online system, since increasing the amount of data used for one decision usually improves statistical stability but also reduces the frequency with which new monitoring information is produced.

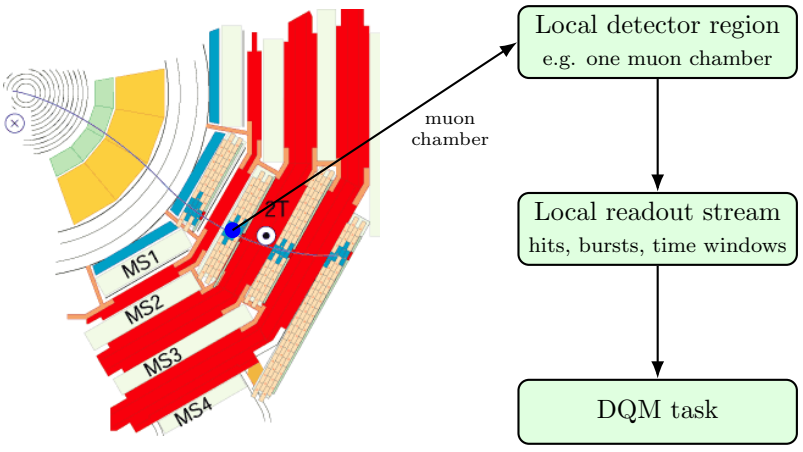


Figure 1.2: Exemplifying the granularity concepts as it would apply to a small portion of the CMS [1] detector at LHC at CERN. In this example, a single muon chamber is monitored (spatial granularity) and the DAQ provides hits data from that chamber (object granularity) with a certain rate/bursts that accumulate over a time interval (time granularity) to form an input batch. The DQM task then uses a sub-set of the data to perform the test (statistical granularity).

To illustrate the interplay between processing efficiency and timing constraints, consider a detector whose DAQ system delivers data to an online DQM task with input rate R_{in}. If the DQM algorithm is applied to batches of size N, the latency (time needed to accumulate one batch) is

$$t_{\text{lat}} = \frac{N}{R_{\text{in}}}.$$

This quantity defines the DQM time granularity as defined earlier. Once the batch is available (called the *monitoring batch*), the DQM algorithm requires a finite processing time $t_{\text{proc}}(N)$ to produce the corresponding monitoring quantity or decision. In order to keep up with the input stream, the processing time must be smaller than, or at least comparable to, the batch accumulation time,

$$t_{\text{proc}}(N) \lesssim t_{\text{lat}}$$

Equivalently, defining the output throughput of the DQM system as $R_{\text{proc}} = N/t_{\text{proc}}$, this must be greater than, or comparable to, the input rate:

$$t_{\text{proc}} \lesssim t_{\text{lat}} \Rightarrow \frac{N}{R_{\text{proc}}} \lesssim \frac{N}{R_{\text{in}}} \Rightarrow R_{\text{proc}} \gtrsim R_{\text{in}}$$

Since the processing time t_{proc} is never zero (even for small input sizes, an algorithm has a finite execution overhead) one has that a minimum processing time per DQM decision exists. If the monitoring batch is chosen too small, the algorithm must be executed many times per unit time, and this fixed per-run cost can become a limiting factor. Increasing the batch size reduces the number of DQM executions per unit time, but also increases the time interval over which data are accumulated before one monitoring decision is produced.

For example, if the input stream delivers R_{in} data elements per second and the DQM algorithm processes batches of N elements, the number of algorithm executions required per second is approximately

$$n_r \simeq \frac{R_{\text{in}}}{N}.$$

If each execution has a minimum processing time $t_{\text{proc,min}}$, the minimum computing time required per second is approximately

$$n_r t_{\text{proc,min}} \simeq \frac{R_{\text{in}}}{N} t_{\text{proc,min}}.$$

This expression shows why very small monitoring batches may be inefficient: even if each batch contains little data, the fixed cost of repeatedly launching the algorithm can become significant. A larger batch size reduces this overhead per unit time, but at the price of a coarser DQM time granularity.

This distinction is particularly relevant for DQM algorithms based on statistical comparisons with a reference sample, such as the one that is embodied in this work. The monitored batch may contain N input objects, but the algorithm does not necessarily use all of them explicitly in the final statistical comparison. It may instead operate on aggregated monitoring quantities, histograms, or on a representative subsample of size $N_{\mathcal{D}} \leq N$. In this case, N determines the time window of detector data represented by one DQM output, while $N_{\mathcal{D}}$ controls the number of data points used by the statistical test.

Thus, two different levels must be distinguished in a batch-based, reference-comparison DQM algorithm. The first is the DQM time granularity, fixed by the monitored batch size N, which determines how often the system can produce a new monitoring output. The second is the statistical granularity, fixed by the number $N_{\mathcal{D}}$ of data points used in the comparison with the reference sample. Smaller values of N improve the time localization of DQM decisions, but increase the number of algorithm executions per unit time and may reduce the statistical power of the test. Larger values of N improve statistical stability and reduce per-run overheads, but provide slower feedback and coarser time localization.

1.3.3 Usage of computing accelerators

In order to aid in meeting the timing requirements, it is beneficial to leverage on the processing power of dedicated computing accelerators. These come in many different forms and functionalities: Field Programmable Gate Arrays (FPGA), Graphics Processing Units (GPU), Tensor Processing Units (TPU) and more exotic ones designed to accelerate a specific family of tasks (or sometimes even more specialized). Depending on which stage the DQM system is operating at along the DAQ system, one would use the appropriate accelerator. For instance an FPGA could encode a DQM algorithm to check upon the raw information coming from the detector, a GPU could instead be used to process higher level information where it is required to perform heavy vectorized operations and so forth.

1.3.4 Relevance to this work

The work presented in this thesis consists in the development of a GPU-resident pipeline to perform online DQM, particularly anomaly detection, for a muon detector prototype. The detector itself is a scaled-down version of one of the drift chambers of the CMS [1] experiment at the LHC at CERN.

The pipeline ingests the *raw hits* produced by the detector Front-End electronics (FE) accumulated in batches of configurable size, builds candidate events, reconstructs particle tracks and finally computes a scalar value that represents an *anomaly score*, quantifying the compatibility between each monitored input batch with a reference condition in which the detector was operating in nominal conditions. These tasks are performed by processing data that is resident in the GPU memory buffers in a sequence of functional blocks: pre-processing, track reconstruction and DQM.

Further, the pipeline operates in a *triggerless* fashion, so no explicit trigger signal is used as a hardware control signal to decide whether or not to stream data forward. In fact, the FE builds the time information starting from the sampling of the detector raw signals and also a scintillator pad. If the time information is not null (zero-suppression) the data is sent over to the back-end. The "software trigger" then consists in the pre-processing algorithm requesting that a scintillator signal be present and collecting the time information from the detector in an appropriate time window thus identifying a possible candidate event to be then reconstructed and, together with all the others, build the monitored batch for the DQM stage. The DQM stage consists in a special kind of two-sample statistical test [2] that leverages on the power of a machine-learning framework for fast-kernel methods [3] in order to perform it.

The focus of this work is to demonstrate the feasibility of the online operation of the pipeline with respect to the experimental conditions of the reduced size detector and to speculate on the possibility of scaling the approach up to higher throughput experiments. Further, the specific DQM task the pipeline focuses on is that of **anomaly detection** (DQM is in fact a broader framework of different tasks, methods, etc. out of which anomaly detection constitutes a part).

Chapter 2

Anomaly Detection

In the following, the concept of anomaly detection is introduced as a kind of DQM task.

2.1 Introduction

Anomaly detection refers to the identification of observations or patterns in the data coming from a source (for instance a particle detector) that deviate from the behavior expected under normal operating conditions. A particular source of data (measurements) could be a sensor sampling a signal or a more complex detector streaming measurements on several channels at once. These measurements can become corrupted by faults, environmental variations, tampering or miscalibrations. The goal in monitoring applications is then to be able to detect corrupted measurements and also to recognize changes in the statistical distribution of the incoming data stream itself. For instance if the data that arrives from the source at nominal conditions has a certain distribution, repeated arrivals of corrupted or anomalous measurements can induce distortions in the distribution.

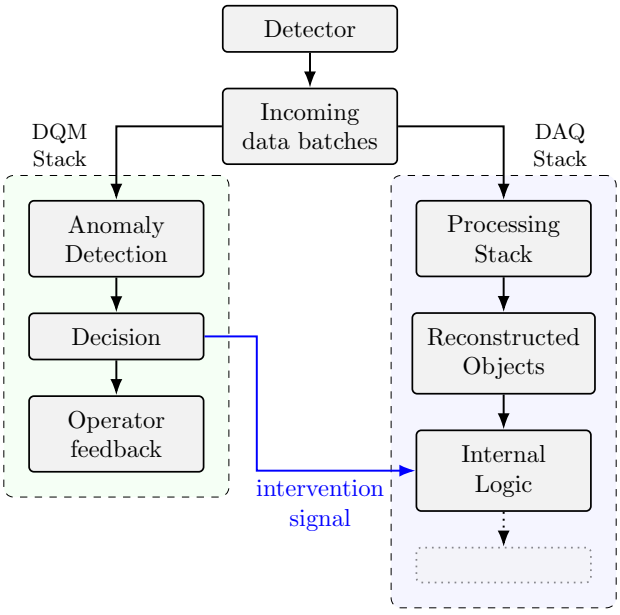


Figure 2.1: Simplified schematic diagram of an example anomaly detection for DQM stack running parallel to a DAQ stack at a particle detector. The same incoming data batches are sent to the two hardware/software stacks. The DQM stack will compute a decision and thus provide an intervention signal (warning, alarm, etc.) to the detector stack. While this happens, the DAQ does its own processing and then the internal logic decides on how to use the intervention signal.

In streaming systems such as the example system depicted in Figure 2.1 anomaly detection can be

performed either at the level of individual observations or at the level of data batches. This latter case is especially relevant when single events are not anomalous by themselves, but the ensemble of events reveals a change in the underlying distribution. For instance if a particle detector is undergoing a fault or deviation with respect to normal operating conditions (i.e. fluctuations in power supply, electromagnetic noise, etc) and the time interval in which the deviation occurs is longer than the typical data timings, then many measurements may occur in such degraded conditions and so an entire data batch could be affected and not only a single measurement.

The idea is then to define a reference description of nominal behavior and to compare newly acquired data against it. The reference may be obtained from simulation, calibration runs, or previously validated measurements. A new observation, or a batch of observations, is considered anomalous when it is statistically incompatible with the reference according to a chosen metric or test statistic. This setting naturally suggests a two-sample problem formulation where a reference and a monitored sample are compared and compatibility with the same probability distribution is checked.

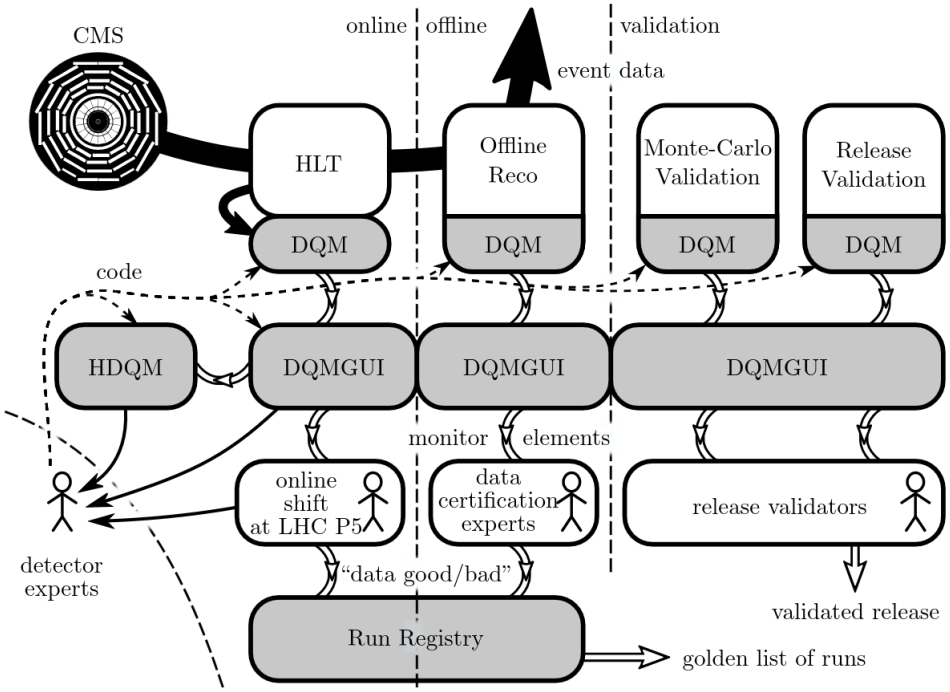


Figure 2.2: CMS detector DQM systems (Azzolini et al. [4]).

In the following, several examples of approaches to anomaly detection are provided. Given the scope of this work, they are tailored to detector applications, especially the CMS [1] experiment at LHC at CERN. The DQM systems that are in place at the CMS experiment are shown schematically in Figure 2.2, originally published in [4] and anomaly detection tasks constitute a subset of all the tasks that these systems perform.

2.2 Classical approaches

Classical approaches to anomaly detection for DQM rely on simple but powerful operations, such as threshold checks, trend monitoring, histogram comparisons, and statistical tests between reference and observed quantities. Despite their simplicity, these methods represent the backbone of many DQM systems. Their strength lies in their robustness, interpretability, and direct connection with detector expertise. They allow operators to monitor well-understood quantities, identify known failure modes, and provide fast feedback during data taking.

2.2.1 Threshold-based and histogram-based anomaly detection

The simplest DQM strategy consists in defining a set of detector-level monitoring quantities, specifying the range of values that are considered compatible with normal operation. This is the **threshold-based** approach where an anomaly is identified whenever an observed quantity falls outside its predefined acceptable interval. In the case of CMS these strategies are detailed for instance in [4, 5] and most recently with respect to this work in [6]. Typical monitored quantities include channel occupancies, hit rates, timing distributions, trigger rates, reconstruction efficiencies, or fit-quality variables. This approach is computationally cheap and highly interpretable, since each alarm can often be traced back to a specific detector quantity.

A more detailed class of classical methods is based on histogram-level anomaly detection. In this case, the detector response is not reduced to a single scalar quantity. Instead, distributions of relevant observables are accumulated over a given parameter interval (time, luminosity, etc.) and compared with reference histograms representing nominal detector behavior. An anomaly is then associated with a significant deviation between the observed and reference distributions. This comparison can be performed through bin-by-bin residuals, ratios, χ^2-like tests, Kolmogorov-Smirnov tests, or dedicated quality criteria. This approach is widely used because it preserves a direct connection with detector observables and allows experts to identify the origin of a problem. Its limitation is that the monitored variables and the comparison criteria must be chosen in advance.

The main limitation of these approaches is that the monitored quantities, the reference distributions, and the comparison criteria must be specified in advance. They are therefore most effective when the relevant detector signatures are already known or can be captured by a small number of observables. Anomalies that appear only as weak, correlated changes across several variables may be more difficult to identify with independent threshold checks or one-dimensional histogram comparisons and in this case a multivariate statistical approach might be more beneficial.

Rule-based systems

In many operational systems, simple thresholds are combined into rule-based diagnostic logic. Expert knowledge is used to associate patterns of deviations with possible detector problems, such as noisy channels, timing shifts, inefficient regions, or front-end electronics failures. These systems are useful for known and well-characterized faults, but they are less flexible when the detector response changes in an unexpected way or when several weak effects are distributed across many observables. Also in the case of CMS this is detailed in [7].

Time-series/trend monitoring

Another class of methods focuses on the temporal evolution of monitoring quantities. Rather than testing each batch independently, the DQM system tracks rates, efficiencies, occupancies, or calibration-sensitive observables as time series. Anomalies can then be identified through trend analysis, moving averages, control charts, or other time-series methods. This approach is particularly useful for detecting slow drifts, intermittent instabilities, or correlations with external conditions such as temperature, luminosity, or detector configuration.

Reconstruction-level anomaly detection

Anomaly detection can also be performed at the level of reconstructed quantities as it was earlier shown in Figure 2.2. In this case, the monitored observables are not raw detector signals, but higher-level features such as reconstructed tracks, fit residuals, hit multiplicities, reconstruction efficiencies, or quality estimators. An anomaly is therefore identified as a deviation of these reconstructed observables from their nominal behavior. This strategy is sensitive to effects that may not be obvious at the raw-data level but become visible after reconstruction. At the same time, the interpretation of anomalies can be more complex, since a change in a reconstructed observable may originate from detector conditions, calibration constants, or reconstruction algorithms. Then on the one hand one

has that reconstruction-level anomalies provide more integrated information. On the other hand, the interpretation of the anomalies can be more complex. A deviation might in fact originate from several sources, perhaps even as biases from the algorithms themselves and so tracing back an anomaly to its underlying cause may require more ancillary information.

2.3 Statistical methods

In the following the data-driven / statistical method that is at the core of the anomaly detection task that is embodied in this work is presented. Further, in this work the anomaly detection task is always performed on data **batches** and never on individual observations.

2.3.1 General introduction to the problem and scope of the current work

The classical methods described earlier are largely based on monitoring quantities, reference comparisons, and decision criteria defined a priori. As a consequence, they may be less sensitive to anomalies that are distributed over several observables, appear only through correlations among variables, or do not match any predefined failure signature. This motivates the development of complementary data-driven methods in which the monitoring problem is formulated directly in statistical terms.

In this respect, let $x \in \mathbb{R}^d$ denote the set of features used to describe one event, track, or reconstructed detector object. Under nominal detector conditions, these features are assumed to be distributed according to an **unknown** reference probability density $p_{\text{ref}}(x)$. A detector fault, calibration error, or change in the operating conditions can modify the detector response and therefore induce a different distribution, denoted by $p_{\text{data}}(x)$, for the newly acquired data.

From this point of view, *anomaly detection in DQM can be interpreted as the problem of testing whether the monitored data batches are still statistically compatible with those taken from the reference distribution.* If an anomaly is found it is then associated with the monitored batch as a whole. This batch-monitoring fashion is particularly relevant in detector monitoring, where variations in electronic thresholds, drift velocity, timing calibration, noise conditions, or detector efficiency can produce small but coherent changes in the multivariate feature distribution over the many elements constituting the batch.

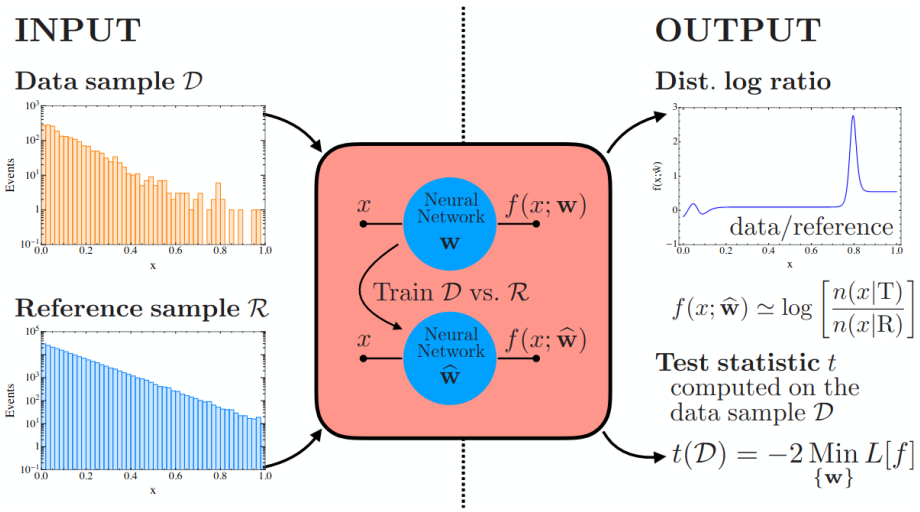


Figure 2.3: D'Agnolo and Wulzer [8]

The goal is therefore to construct a test statistic, that is sensitive to deviations of $p_{\text{data}}(x)$ from $p_{\text{ref}}(x)$. Such a statistic should remain compatible with its nominal behavior when the detector is operating correctly, while assuming larger values when the monitored batches are drawn from a distribution different from the reference one. The interpretation of this statistic requires a calibration under nominal conditions, so that an observed deviation can be translated into a measure of statistical

compatibility with the reference hypothesis. Since the probability densities p_{ref} and p_{data} are not known analytically in realistic detector applications, the comparison must be performed using finite samples drawn from the reference and monitored data streams.

These ideas were at the core of the proposal by d'Agnolo and Wulzer [8] that consisted in the usage of neural networks to detect departures from a reference model through a likelihood-ratio hypothesis-test construction as shown in Figure 2.3, returning a p-value for reference compatibility. This approach has then been improved upon [9] replacing neural networks with kernel methods, yielding much faster response times, possibly compatible with online applications. This last possibility was initially explored in [2] and [10] upon which this current work is based, establishing an online-runnable anomaly detection for DQM system in the form of a GPU-resident pipeline.

2.3.2 Goodness-of-Fit formulation and Likelihood-ratio testing

Since the probability density $p_{\text{ref}}(x)$ describing nominal detector behavior is not known analytically in realistic applications, the reference hypothesis must be represented through a finite reference dataset. Let

$$X_{\text{ref}} = \{x_i^{\text{ref}}\}_{i=1}^{N_{\text{ref}}}$$

denote the reference batch drawn from a larger reference dataset acquired under nominal conditions, and let

$$X_{\text{data}} = \{x_i^{\text{data}}\}_{i=1}^{N_{\text{data}}}$$

denote a data batch drawn from the monitored data stream. The statistical question is whether the monitored data batch is compatible with the reference hypothesis, or whether it is better described by a different detector response.

This can be formulated as a goodness-of-fit problem. The null hypothesis corresponds to the case in which the monitored data are distributed according to the same probability law as the reference data,

$$H_0 : p_{\text{data}}(x) = p_{\text{ref}}(x),$$

while the alternative hypothesis corresponds to the presence of a deformation of the reference distribution,

$$H_1 : p_{\text{data}}(x) \neq p_{\text{ref}}(x).$$

In the Neyman–Pearson framework, the optimal test between two simple hypotheses is based on the likelihood ratio. In the present case, however, the alternative distribution is not known in advance, since the possible detector anomalies may be diverse and not explicitly parametrized. The strategy is therefore to introduce a flexible model capable of describing deviations from the reference distribution and to use the corresponding likelihood-ratio-inspired quantity as a test statistic.

The role of the anomaly-detection algorithm is then to construct a scalar statistic t

$$t = t(X_{\text{ref}}, X_{\text{data}})$$

which is written in this way to emphasize that it quantifies the degree of incompatibility between the monitored data and the reference hypothesis. If the detector is operating under nominal conditions, the statistic fluctuates according to its null distribution, determined by finite-sample effects and by the details of the testing procedure. If the monitored data are affected by an anomaly, the statistic is expected to move toward larger values, indicating that the reference hypothesis provides a poorer description of the observed batch.

The numerical value of the statistic is not directly meaningful on its own and must be calibrated under the null hypothesis. This calibration can be performed by repeatedly comparing independent samples drawn from the reference dataset,

$$t_i^{\text{null}} = t\Big(X_{\text{ref},i}^{(1)}, X_{\text{ref},i}^{(2)}\Big),$$

thereby constructing an empirical null distribution T_{null}. Given a new monitored batch with observed statistic

$$t_{\text{obs}} = t(X_{\text{ref}}, X_{\text{data}}),$$

its compatibility with the reference condition can be quantified by the upper-tail probability

$$p_{\text{obs}} = P(T_{\text{null}} \geq t_{\text{obs}} \mid H_0).$$

Small values of p_{obs} indicate that the observed discrepancy would be unlikely under nominal detector conditions and therefore provide evidence for an anomalous batch. Because the reference distribution is represented by a finite dataset, the procedure involves the comparison of a reference sample and a monitored sample. In this operational sense, the method resembles a two-sample comparison. However, the statistical interpretation followed here is that of a likelihood-ratio-inspired goodness-of-fit test, where the goal is to assess the compatibility of the monitored data with the reference hypothesis.

2.3.3 NPLM test statistic

The likelihood-ratio-inspired goodness-of-fit construction described above is referred to as the New Physics Learning Machine [11] (NPLM) which is a machine-learning-based strategy for detecting deviations of observed data from a reference hypothesis, without assuming a specific anomaly model in advance. The method is motivated by the likelihood-ratio test and introduces a flexible function $f_w(x)$ to parametrize possible deformations of the reference distribution. In particular, the alternative hypothesis is modeled as a local rescaling of the reference density,

$$p_w(x) \propto p_{\text{ref}}(x) e^{f_w(x)},$$

so that $f_w(x)$ can be interpreted as a learned log-density-ratio-like deformation between the monitored data and the reference model [2, 12].

The parameters $\vec{\mathbf{w}}$ of this function are determined through a maximum-likelihood fit, implemented as a machine-learning optimization problem with a dedicated loss function [2]. Once fitted, $f_w(x)$ is used to construct a scalar test statistic that measures the improvement obtained by the deformed model with respect to the undeformed reference hypothesis. By calibrating this statistic under reference conditions, the observed value can be assigned a p-value quantifying the compatibility of the data with the reference model.

In the original formulation [13], the function $f_w(x)$ was parametrized via a neural networks model. This was improved upon [9] switching from neural networks to fast kernel methods. These provide an alternative nonparametric realization of the same principle: they compare events through similarity functions in feature space and can therefore capture nonlinear, multivariate differences between the reference and monitored samples.

Generally speaking, kernel methods represent a learned function as a weighted combination of similarity functions centered on the training data:

$$f_w(x) = \sum_{i=1}^{\mathcal{N}} w_i k_\sigma(x, x_i), \tag{2.1}$$

where w_i are the learned coefficients, $\mathcal{N}$ is the number of points, σ is a hyperparameter and k_σ is the kernel which in the case of these works and also this present work is a Gaussian kernel:

$$k_\sigma(x, x') = e^{-\|x-x'\|^2/2\sigma^2}.$$

This approach is called nonparametric because the number of parameters correspond to the number of data points. The main issue with standard kernel methods is their poor scaling with the the number of points, since the full kernel matrix grows quadratically with the sample size. For this reason an approximation is used, resulting in the so called fast-kernel methods: the kernel problem is approximated with a reduced representation.

One such approximation is called the **Nyström** approximation. This consists in writing Equation (2.1) in the following way:

$$f_w(x) = \sum_{i=1}^{M} w_i k_\sigma(x, \tilde{x}_i), \tag{2.2}$$

where this time the number of data points $\mathcal{N}$ is exchanged with the number M of *Nyström* centers with the kernels centered on them: $\tilde{x}_i$. The Nyström centers are sampled uniformly at random from the input data and their total number M is a hyperparameter.

In particular, the Nyström approximation replaces the full kernel matrix with a representation based on a smaller set of centers, making large-scale kernel learning computationally feasible. In this context, the computational implementation of the kernel learning step is provided by Falkon [3].

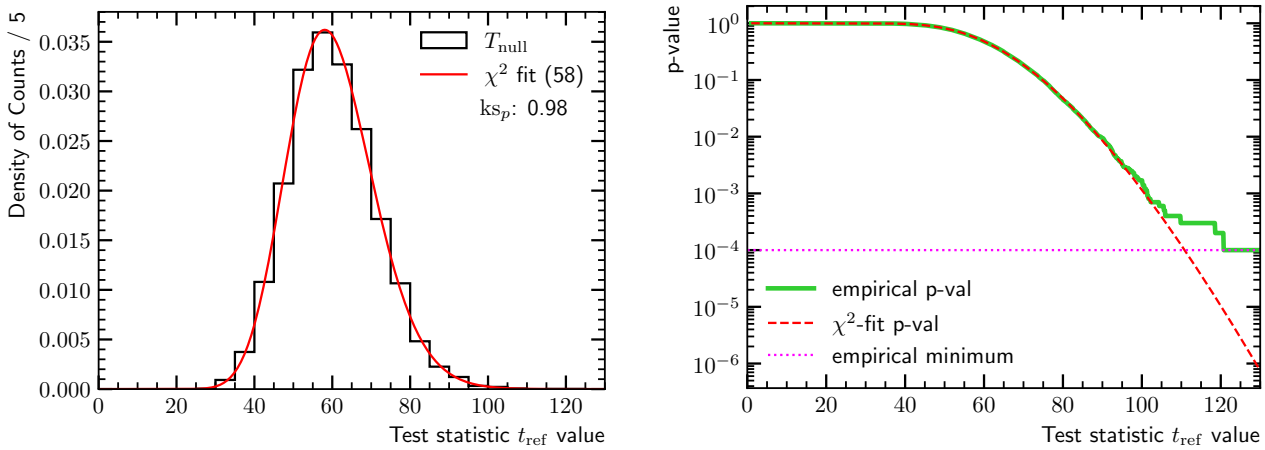
Finally, the NPLM test statistic is given [2] by:

$$t_{\hat{w}}(\mathcal{D}) = 2 \sum_{x \in \mathcal{D}} \log \frac{p(x|H_{\hat{w}})}{p(x|R)} = 2 \sum_{x \in \mathcal{D}} f_{\hat{w}}(x) \tag{2.3}$$

where $p(x|R)$ is the data under the null distribution (the reference), $p(x|H_{\hat{w}})$ is instead the data under the alternative hypothesis depending on the trainable parameters. The best set of such parameters is denoted with $\hat{w}$ and since f_w directly parametrizes the logarithm of the ratio, Equation (2.3) holds.

On the calibration procedure for the NPLM test statistic

Given the paramount importance of having a well-calibrated test statistic, the results of the calibration procedure for a possible case are shown in the following. The test statistic distribution shown in Figure 2.4a has been obtained in a series of repeated reference-vs-reference tests using Falkon that will be described in the dedicated section. The continuous line shows the fitted χ^2 model. The agreement between the empirical distribution and χ^2 is used to check that the reference test statistic distribution is well behaved. This distribution, in the limit of infinite sample size, tends asymptotically to a χ^2 distribution.



(a) Test statistic distribution and χ^2 model fit.

(b) Empirical and fit p-values versus t_{ref}.

Figure 2.4: Example of the outcome of a calibration procedure for the test statistic t_{ref}. Figure 2.4a shows the test statistic distribution and the fit to a χ^2 model with inferred 58 degrees of freedom, reporting a Kolmogorov-Smirnov $p-value$ of 0.98. Figure 2.4b shows instead the good agreement between the empirical p-value and the one obtained from the fit for a given test statistic value t_{ref}, demonstrating the validity of the calibration. Of course near the upper tail, the agreement breaks down.

Figure 2.4 shows the results of such procedure and the following quantities are introduced: the *empirical and the fit p-values*. In general, given an observed test statistic t_{obs}, one can compute its p-value

under the reference distribution (the null hypothesis) as:

$$p[t_{\text{obs}}] = \int_t^\infty dt' p(t'|R) \tag{2.4}$$

where $p(t'|R)$ is, operationally, the distribution of the test statistic obtained via the reference-vs-reference repeated tests. So the **empirical** way to estimate the p-value consists in simply doing:

$$p_{\text{emp}}[t_{\text{obs}}] = \frac{\#\{t_i^{\text{null}} \geq t_{\text{obs}}\} + 1}{N_{\text{null}} + 1}, \tag{2.5}$$

where N_{null} is the number of test statistics in the reference distribution. So this consists in considering the normalized fraction of the null distribution starting from t_{obs} and is the discretized version of Equation (2.4) and is shown in Figure 2.4b as the thick green trace. Instead the χ^2-**fit** way to estimate the p-value, consists in obtaining it from the fitted model to the reference distribution which is shown again in Figure 2.4b with the red dashed line. The agreement between the two is good until one starts to hit the upper tail where it eventually breaks down. In fact Equation (2.5) has a lower value: when the upper sum is zero, the minimum returned p-value is:

$$p_{\text{emp,min}} = \frac{1}{N_{\text{null}} + 1} \approx N_{\text{null}}^{-1},$$

for large N_{null}. For instance if $N_{\text{null}} = 10^6$, then $p_{\text{emp,null}} \approx 10^{-6}$ and so the empirical p-value cannot explore regions where the t_{obs} is indeed more extreme than this value.

On the contrary, the χ^2-fit p-value is smooth and can go beyond that but its meaningfulness depends on how well the upper tail in the data is described by the fitted model. For instance supposing that, with this example, one observes a $t_{\text{obs}} = 150$, the empirical p-value will return at minimum 10^{-6} so one will say that the "true" $p_{\text{obs}} \leq 10^{-6}$ whereas the χ^2-fit p-value will return 2.5×10^{-9}.

Summary on the NPLM procedure for DQM

To summarize the procedure: a model for f_w is picked up, selecting its hyperparameters via an appropriate search as it will be clarified in the dedicated section. Then the test statistic is calibrated so that its value $t(\mathcal{D})$ ($\mathcal{D}$ represents the input data batch) can be associated to a probability $p[t(\mathcal{D})]$ which is the p-value which will represent the input data batch from a DQM perspective, and a subsequent analyzer could decide to classify the batch as good or rejected as bad.

Additional remark even if it is not part of the official NPLM procedure, one can also study the value of $t(\mathcal{D})$ itself. While the p-value remains the statistical meaningful object to essentially define a binary classification context where one accepts or rejects an input batch depending on whether p is larger or smaller than a threshold, the value of t can also be used for a second-level classification once a batch has been labeled as anomalous.

In the case of the particle detector for instance, if one chooses in advance the failure modes of the device and collects data samples from the system when the detector is intentionally put in such degraded modes, it is possible to get the various distributions of the test statistics in the various operating modes. If the system is calibrated correctly and is sufficiently sensitive, these distributions will be shifted with respect to the reference one and so, ideally, a multi-class classifier could help to label the identified anomalous batch with a known cause, including also the possibility of an unknown origin/cause when the classifier cannot reliably decide upon known modes.

This possibility has been addressed in this work and for this reason a **classifier head** has been attached to the end of the pipeline as documented in Section 4.4.

2.3.4 Falkon: fast large-scale kernel solver based on Nyström approximation

The NPLM prescription does not require a specific choice for the function approximator used to model $f_w(x)$. In principle, this function could be represented by a neural network, a boosted decision tree, or another flexible parametric model. In the implementation considered in this thesis and also in the reference works, $f_w(x)$ is learned using Falkon [3], a scalable kernel method.

Falkon is a machine-learning approach since it learns a function from data in a supervised manner: the reference data and the real data are labeled with different values and Falkon learns to distinguish them. Specifically by defining the empirical error as:

$$\hat{L}(f_w) = \frac{1}{\mathcal{N}} \sum_{i=1}^{\mathcal{N}} l(y, f_w(x)), \tag{2.6}$$

where $l(y, f(w))$ is the weighted cross-entropy loss [9]:

$$l(y, f_w(x)) = (1-y)(1 + N_1/N_0) \log(1 + e^{f_w(x)}) + y(1 + N_0/N_1) \log(1 + e^{-f_w(x)}), \tag{2.7}$$

y is the class label (0 for reference $\mathcal{R}$, 1 for data $\mathcal{D}$), x is the input data point, N_0 and N_1 are the number of reference and data points respectively. One then has the following objective [9]:

$$\hat{L}(f_w) + \lambda R(f_w), \qquad \text{where} \qquad R(f_w) = \sum_{i,j} w_i w_j k_\sigma(x_i, x_j). \tag{2.8}$$

where λ controls the strength of the regularization term R.

Model hyper-parameters

To select a model, one needs to define the hyper-parameters. These are: the number of Nyström centers (M), the bandwidth of the gaussian kernel (σ) and the regularization parameter λ. In the following, $N_{\mathcal{R}}$ is the size of the reference sample and $N_{\mathcal{D}}$ is the size of the data sample and a similar approach to the one outlined in [2, 9] was used to select these parameters.

The number of centers M controls the expressive power of the model. On the one hand, the larger M is the more sensible it is to complex-shaped distributions. On the other hand, as M increases training time also increases. Experiments conducted in [14] showed that M should be at least as large as $\sqrt{N}$, with $N = N_{\mathcal{R}} + N_{\mathcal{D}}$ and that any value greater than the data batch size $N_{\mathcal{D}}$ does not impact sensitivity.

The Gaussian width σ sets the length scale over which two points are considered similar (this is exemplified in Figure 2.5). A small σ will enable the model to learn localized structures and will thus be also more sensitive to statistical fluctuations and noise. Conversely, a larger σ will "enlarge" the cloud within which many points are considered similar, the model will then learn smoother and more global differences, obfuscating the eventual fine-structure.

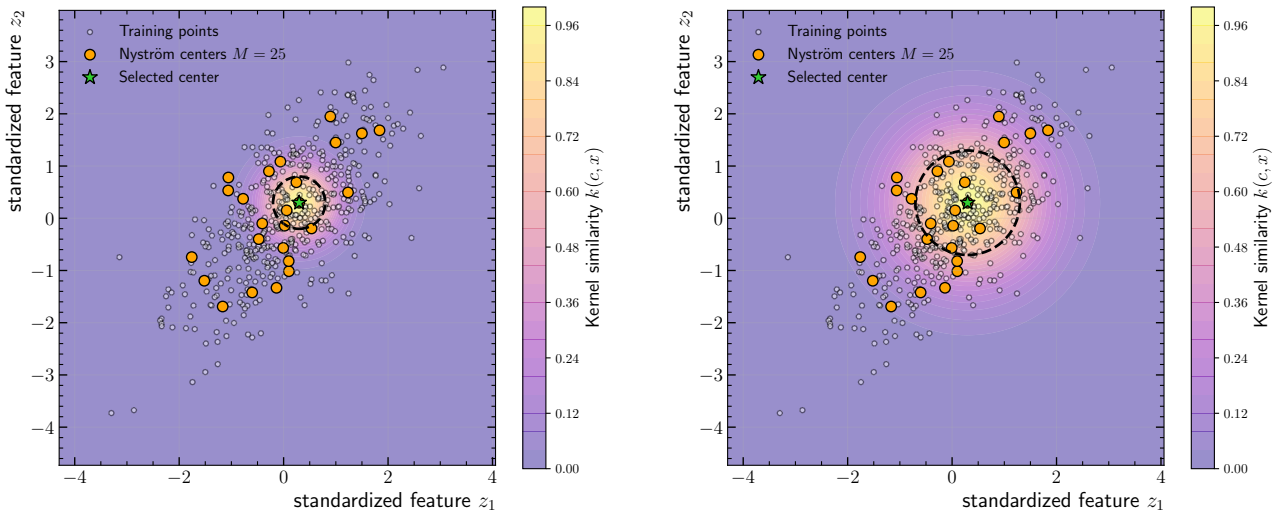
In this work the same prescription that was put forth in [9] was followed in that σ was taken as the 90th percentile of the pairwise distance between reference-distributed data points, after standardization, which is a fundamental step.

In fact, before applying the Gaussian kernel, the following transformation is applied to the input features using the mean (μ_{ref}) and standard deviation s_{ref} of the reference so that each input vector x_j becomes,

$$z_j = \frac{x_j - \mu_{\text{ref}}}{s_{\text{ref}}}. \tag{2.9}$$

This step is important because the Gaussian kernel

$$k(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right),$$



(a) $\sigma = 0.5$ (b) $\sigma = 1.0$

Figure 2.5: Schematic illustration of the Nyström approximation and Gaussian kernel similarity in a two-dimensional projection of the standardized feature space. The small points represent training events, while the highlighted points are the M Nyström centers used to build the approximate kernel expansion. The shaded contours show the Gaussian kernel centered on one selected center; the kernel width σ (shown with the dashed circle) controls the length scale over which points are considered similar.

depends on Euclidean distances in feature space. Without standardization, features with larger numerical ranges would dominate the distance $\|x - x'\|$, while features with smaller numerical ranges would have little effect on the kernel value. Standardization makes the kernel width σ meaningful in units of typical reference fluctuations and allows the different reconstructed quantities to contribute on comparable scales. Then, after standardization, σ controls similarity in the standardized feature space rather than in the original physical units.

Note on standardization Standardization is performed once from the full nominal reference dataset, obtaining the μ_{ref} and σ_{ref} parameters. In this way the same transformation Equation (2.9) can be applied to both reference-vs-reference calibration samples and reference-vs-data test samples. This ensures that the null calibration and the anomaly-detection tests are performed in the **same** standardized feature space.

The regularization parameter λ acts to constrain the model complexity as shown in Equation (2.8), regulating the balance between fitting the observed data-reference difference and maintaining a stable, smooth solution. Smaller values of λ allow a more flexible function and can increase sensitivity to subtle discrepancies, but they may also make the solution more sensitive to statistical fluctuations and can worsen the conditioning of the Falkon linear system. Larger values of λ produce more regularized solutions, improving numerical stability at the possible cost of reduced anomaly sensitivity.

The empirical rule here [2, 9] is to have λ as small as possible, so as to not blur the fine-structure in the data that the model could pick-up, but not too small to have unstable training or model. In fact, if λ is too small, model convergence problems could arise, preventing training from completing or requiring a time that is incompatible with the experimental conditions. Further, when small models are used, such as the case of this work, it has been observed [2] that too small values of λ induce compatibility violations of the distribution of the test statistic (Equation (2.3)) with respect to a χ^2. In this case λ is raised until agreement is restored.

The issue of **compatibility with** χ^2 is because of the following reasoning. During calibration, reference-vs-reference tests are conducted. Hence, the learned function f_w should only fit statistical fluctuations. The size of these fluctuations is measured by the NPLM test statistic which is constructed

as a likelihood-ratio type test statistic. Further, the size of the fluctuations behaves like a sum of squared Gaussian modes which, in turn, gives a χ^2 distribution.

The role of the size N of the model

After one has the correctly calibrated test statistic, reference-vs-data tests can be performed. So to each input data batch $\mathcal{D}$, there corresponds a single test statistic $t(\mathcal{D})$ to which the p-value is assigned as previously described. In order for this to be possible, one needs the input data batch to Falkon to have **at least** $N_{\mathcal{D}}$ elements, while the size N of the model is again given by $N = N_{\mathcal{D}} + N_{\mathcal{R}}$.

Then when one firstly picks out a model and runs the reference-vs-reference tests to collect the null test statistic distribution, a decision must be made regarding the size $N_{\mathcal{R}}$ of the reference batch and the size $N_{\mathcal{D}}$ of the data batch so that the following requirements are fulfilled:

- $N_{\mathcal{D}}$ controls the monitoring granularity: small values can be sometimes beneficial but the batches tend to be noisier since the data can be less generally represented. A larger value can instead yield smoother statistics but it has to be compatible with the experimental requirements (i.e. the ability to get enough data in a timely fashion compatible with the experimental conditions).
- $N_{\mathcal{R}}$ controls the reference precision and the same observations that have been advanced for $N_{\mathcal{D}}$ also apply here with the added requirement that $N_{\mathcal{R}} > N_{\mathcal{D}}$.
- When choosing N one must also ensure that the number of Nyström centers is chosen consistently with its own requirements (as previously stated $M = O(\sqrt{N})$).

The calibrated model expects the input data to contain $n \geq N_{\mathcal{D}}$ elements. In fact, each test is actually a single training being performed as explained in [9]. If the input batch does not contain sufficient data, the reference-vs-data test cannot be performed and so different approaches can be chosen depending on the desired application. In this respect one can follow either a *synchronous* or *asynchronous* approach with respect to the input data batch and the resulting output test statistic and p-value:

- Synchronous mode: to each input batch there must correspond one and only one test statistic and p-value. So there is a $1 \to 1$ correspondence between the input batch and the corresponding reference-vs-data test output. If the batch yields insufficient data, it must be dropped.
- Asynchronous mode: the input batches are accumulated until sufficient data is available to perform the test and so the test statistic is assigned to the input accumulation buffer, losing the $1 \to 1$ correspondence between input batch and output value.

In the case of this work when doing anomaly detection, one is generally required to decorate each batch with the result of the anomaly detection procedure, in this case the test statistic and the p-value associated to it, so all that will be described in the remainder of this work follows this paradigm.

Because of this hard requirement on the dimension of the batches, particularly $N_{\mathcal{D}}$, one must carefully select a value that is compatible with the kind of data that the experimental conditions provide and there is no universal recipe in this respect.

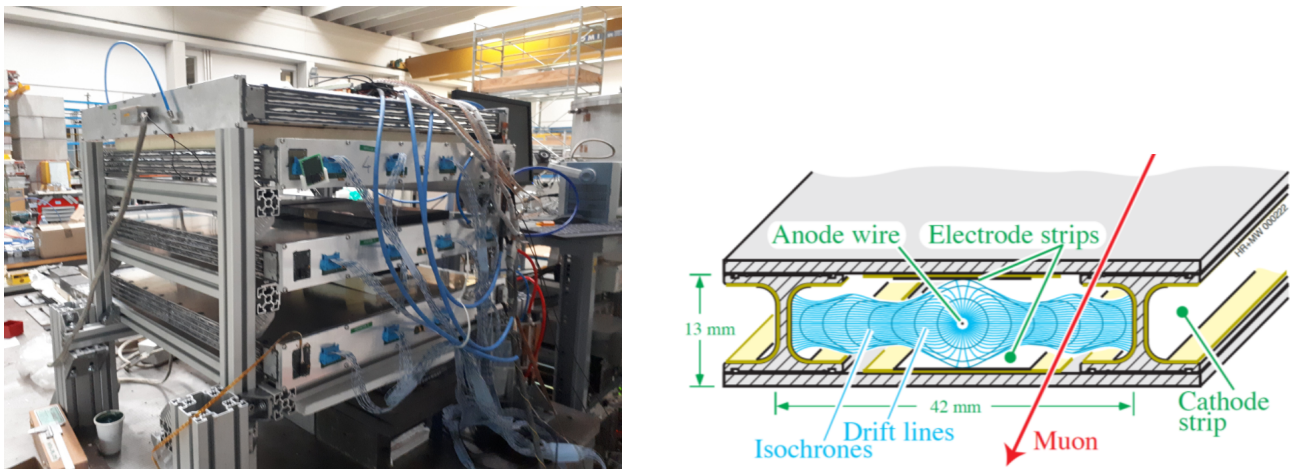
Chapter 3

Experimental setup

3.1 Muon detector prototype

3.1.1 Introduction

As previously mentioned, the detector prototype consists in a scaled-down version of one of the chambers currently deployed at the CMS [1] experiment at the LHC at CERN. The physical setup is the same that was used in [2, 10].



(a) Grosso et al. [2].

(b) The CMS Collaboration [15].

Figure 3.1: Left: picture of the experimental apparatus at the Legnaro INFN National Laboratory. The setup consists in four stacked drift chamber while this work only used signals coming from the third chamber counting from the top. Right: schematic diagram of a single drift tube cell. Each chamber consists in 64 of such cells stacked in four staggered layers of 16 cells each.

The experimental setup is shown in Figure 3.1a at the Legnaro INFN National Laboratory. The system consists in 4 drift chambers with 64 drift tube cells each, stacked in 4 staggered layers of 16 cells. The first chamber is rotated with respect to the other three. This work has only analyzed and processed the signals coming from the third (counting from the top) chamber that is sitting at the center of the photograph. Each tube in the chamber is 70 cm long with a cross section of $42 \times 13\text{mm}^2$. Looking at Figure 3.1b, the central anode wire is set at a voltage of 3.6kV while the two cathode strips at the sides are set at -1.2kV. In order to better shape the electric field ensuring greater homogeneity, two electrode strips are placed above and below the wire at a potential of 1.8kV. Finally, the tubes are filled with a mixture of argon and carbon dioxide gas (85%-15%). When a charged particle traverses the cell, it ionizes the gas and the resulting electrons drift towards the anode wire at a constant velocity, moving along the field lines. The Front-End electronics then records the time of arrival of each signal and amplifies it, filtering out the noise below a certain threshold that is nominally sitting

at 100mV (zero-suppression). The muon particles that are considered here originate from cosmic rays. They occur at a rate per unit area:

$$R_{\mu,\Sigma} \approx \frac{1}{60} \frac{\text{muons}}{\text{cm}^2 \times \text{s}}.$$

Two plastic scintillator pads are positioned on the top and at the bottom of the considered chamber as described in Figure 3.2 and shown schematically in Figure 3.4.

3.1.2 Drift chamber and muon tracks

The 64 cells that constitute a drift chamber are arranged over four staggered layers of 16 cells each.

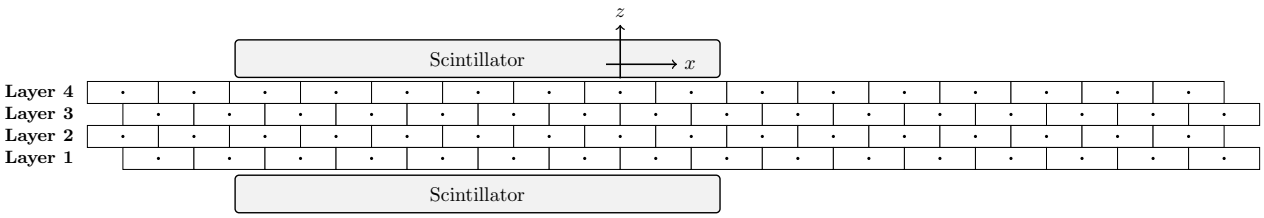


Figure 3.2: Schematic diagram of the drift chamber.

The physical arrangement is shown in Figure 3.2. The layers 4 and 2 are aligned as are layers 3 and 1 but these are shifted by half a drift cell length to the right, so that their left edge aligns with the anode wires of the first cells on the other two layers. This kind of *staggered* arrangement allows to reduce the so called *left–right* ambiguity intrinsic to drift-time measurements. Since the drift time provides only the distance of the particle from the sense wire, each hit is compatible with two possible positions on opposite sides of the wire. By shifting alternate layers by half a cell, the correct hit assignments are those that remain geometrically consistent across both aligned and staggered layers, while wrong combinations are more easily rejected. The staggering therefore improves local track-segment reconstruction and reduces the number of fake compatible patterns.

The left–right ambiguity is quite evident from the following equation that is used to reconstruct the spatial information starting from the temporal one:

$$x_{L/R} = x_{\text{wire}} \pm \Delta x, \qquad \Delta x = v_{\text{drift}} \times t_{\text{drift}}$$

where t_{drift} is the quantity that is measured by the electronics and x_{wire} is the x-coordinate of the anode wire at the center of the drift cell. Actually t_{drift} is obtained as a difference between what the electronics measure and the so called *time pedestal* [16] that is furnished by the coincidence signal of the two scintillators signals and it is found via a calibration procedure. The full details on that procedure and also on the reconstruction model that has been developed for this work are given in Section 4.2.

The position information is used to compute the linear fit to the track segment. In this respect following [2, 10] the following constraints have been imposed: no uncertainty is assumed on the z positions of each anode wire since they are fixed by the apparatus while the uncertainty on the x positions is fixed at $\sigma_x = 400\mu$m; Having four layers and a single valid point for each drift cell in a given layer, a track can have at most four points and it must have at least three points for the fit to be meaningful. While this is again explored in full detail in Section 4.2, only 4-point tracks are considered in this work because of the particular statistical machinery that powers the anomaly detection for DQM system that is performed as the final stage of the pipeline.

In fact, once a track has been computed and provided it satisfies the necessary quality requirements, the track angle with respect to the z-axis is computed as the tangent of the slope of the track and the four drift times associated to the data points that have been used in the fit are extracted. The final dataset is then built with the following five features: $(t_4, t_3, t_2, t_1, \theta)$.

3.2 Data Acquisition system

3.2.1 Front-End (FE) and Back-End (BE) electronics

The DAQ system consists in the front-end (FE) and the back-end (BE) electronics. The FE is itself made of two main functional blocks: the analog electronics, which is mounted right on the drift chamber, that processes the raw signals, filtering and shaping them so that they can be accepted by the digital electronics to compute the drift times. As it was previously mentioned, the system operates in triggerless mode, so there is a free running clock signal, set at 40MHz, the same frequency as the LHC machine clock. This clock drives the digital logic which is implemented on two FPGA boards, each one sampling half of the drift chamber channels (one of them also samples the scintillator pads coincidence signal as previously said).

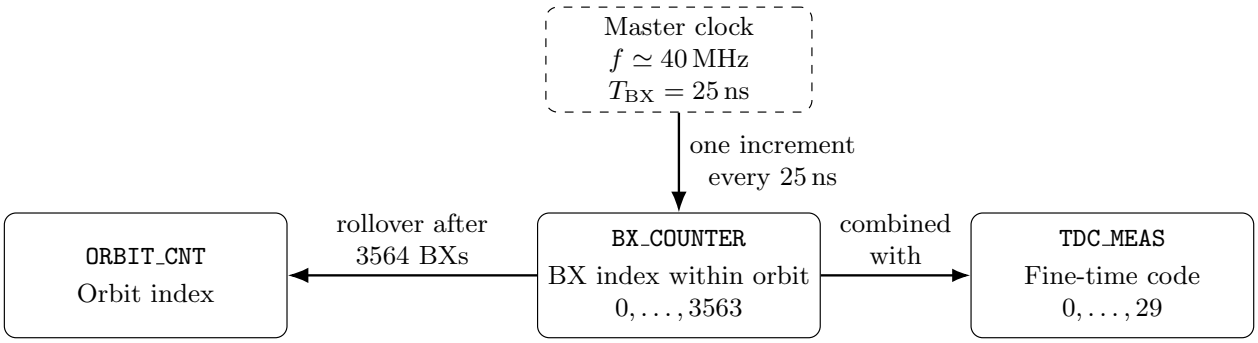


Figure 3.3: Time measurement system: fine and coarse time information is encoded via the three quantities ORBIT_CNT, BX_COUNTER and TDC_MEAS.

This internal clock allows the digital logic to encode the time information provided by the drift cells in the chamber using three quantities as depicted in Figure 3.3: the *ORBIT-counter*, the *BX-counter* and the *TDC* measurement. The TDC counts from $0 \rightarrow 29$ (30 total steps) within a clock cycle of 25 ns, incrementing by one each 0.83 ns. Each time the clock ticks, the BX counter steps up by one, counting from $0 \rightarrow 3563$ and each time it rolls back to zero, the ORBIT counter is incremented by 1. This is documented in depth in Section 4.1

The time information for each hit that was recorded in each drift cell is sent over to the BE via optical cables. The BE is a computer system that hosts two matching FPGA boards that receive the signals from the front-end and then transfer it to the computing server where the data is loaded to the GPU memory buffers in order to be processed by the pipeline. The computing server runs the pipeline software whose memory space resides on the GPU device, which is an NVIDIA A100 40GB (Ampere architecture). As a future extension, the detector FE will be able to write directly to the GPU buffers using GPUDirect–RDMA technology [17].

3.2.2 Raw hit rate

With respect to the hit rate due to cosmic muons interacting with the detector, it is possible to make the following hypothesis. As mentioned before, the two scintillator pads are placed on the top and at the bottom of the drift chamber, and they are positioned in such a way so that they cover the whole length of the chamber (70 cm) but only a limited width which spans roughly 7 drift cell widths.

This is shown in Figure 3.4, which is the output of the event display application that has been developed for this work. It shows a $x-z$ plane view of the drift chamber and the color-filled drift cells represent the area that is covered by the plastic scintillators. The red-filled cells are the ones where the most events are reconstructed. If one assumes ideal efficiencies and geometrical acceptance, the effective area that is exposed to the cosmic muon flux and the muon rate are in the order of:

$$\Sigma = 7 \times (70\mathrm{cm} \times 4.2\mathrm{cm}) = 2058\ \mathrm{cm}^2 \longrightarrow R_\mu \approx 34.3 \frac{\mathrm{muons}}{s}$$

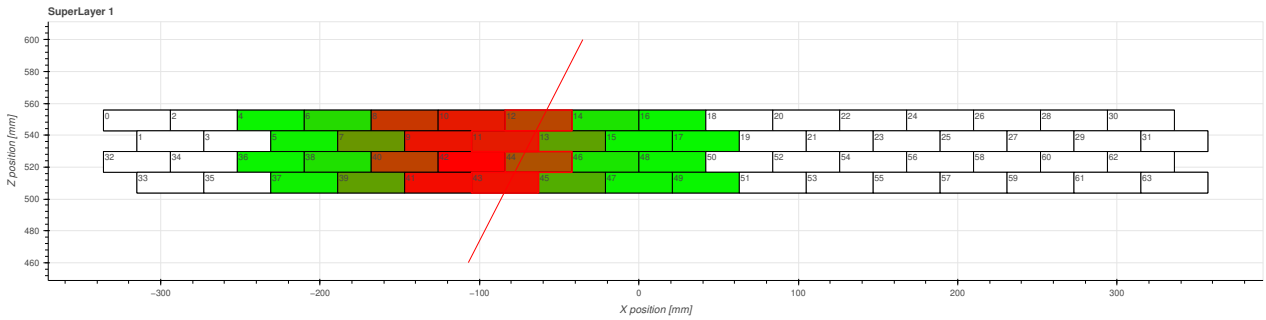


Figure 3.4: Event display highlighting the cell region being covered by the scintillator pad.

If one considers single muon events with 4 hits, one for each layer, then the rate at which hit information is produced would be of roughly $4 \times R_\mu$, yielding a hit rate $H_\mu \approx 137$ hits/s.

3.3 Anomaly detection for DQM system

The anomaly detection for DQM system consists in the GPU-resident, multi-library heterogeneous pipeline which runs in online mode and processes the input data in batches of fixed size as they arrive. The pipeline starts from the input raw data batch produced by the detector (raw hits / time measurements) and forms a list of *candidate events*. The amount of candidates available when the input batch is set at a certain size is strictly dataset dependent and so it depends on the experimental conditions (both apparatus and particle source characteristics). The candidates are then reconstructed into events (tracks) filtering out those who do not meet the required quality criteria (χ^2 cuts and optionally programmable explicit physics-backed requirements). The list of reconstructed events is then shaped in the previously defined form $(t_4, t_3, t_2, t_1, \theta)$ and is passed over as input to the DQM layer which performs anomaly-detection as explained thoroughly in Section 4.3.

Since anomaly detection involves the ability to recognize departures from normal operating conditions, several datasets were collected under different detector operating conditions. A *reference* dataset was collected when the detector operated at nominal conditions: power supply, gas pressure and analog signals thresholds all within prescribed tolerances. Then multiple datasets with intentionally altered conditions forcing the detector to operate in degraded yet controlled modes were taken. Each time a single alteration was introduced: three datasets were obtained by varying the cathode strips supply voltage (setting it at 75%, 50% and 25% of the reference value) and three other datasets were obtained by varying the front-end thresholds (again setting them at 75%, 50% and 25% of the reference value).

The details of the datasets are reported in Table 3.1.

Dataset	**Acronym**	**Description**	**N. of hits**	**N. of triggers**
Reference	REF	nominal conditions	461510115	597002
thresholds-75	D0	75% of nominal (75mV)	16142367	17573
thresholds-50	D1	50% of nominal (50mV)	17191400	17142
thresholds-25	D2	25% of nominal (25mV)	167360033	17883
cathodes-75	D3	75% of nominal (-900V)	12261837	17185
cathodes-50	D4	50% of nominal (-600V)	13024771	17568
cathodes-25	D5	25% of nominal (-300V)	12496957	17883

Table 3.1: The various datasets that have been used in this work. The acronyms defined in this table are used across the remainder of this work.

Chapter 4

Pipeline implementation and validation

This chapter describes the pipeline functional blocks thoroughly, furnishing architectural and performance details for each of them. The pipeline consists in several of such blocks: the pre-processing, the track reconstructor and event builder and the anomaly detection / data quality monitoring block.

4.1 Pre-Processing block

As it was said before, the system operates in triggerless mode and it streams the data to the pipeline input in the form of batches as cuDF dataframes. These batches are comprised of hit-words. A hit-word represents a unit of information generated by the detector's readout electronics and they do not constitute candidate events at this time. They merely contain information regarding the origin of the hit (whether it was produced in one of the drift tubes or the scintillator pad) and the time information of the signal. The main objective of this block is to generate candidate events starting from the raw hit-words.

4.1.1 Architecture and Data model

The pre-processing stage receives the batches with binary packed data from the detector's readout electronics. After unpacking, each hit-word is structured as shown in Table 4.1.

HEAD	FPGA	TDC_CHANNEL	ORBIT_CNT	BX_COUNTER	TDC_MEAS
uint8	uint8	uint8	uint64	uint16	uint8

Table 4.1: Hit-Word data structure. The input dataframe contains a fixed number of these, stacked as rows.

The HEAD, FPGA and TDC_CHANNEL fields specify each readout channel's mapping to the actual detector's tubes or scintillator pad. Instead the ORBIT_CNT, BX_COUNTER and TDC_MEAS fields directly measure time information according to the diagram depicted in Figure 3.3.

When a hit occurs both the channel-related (position) and counters-related (time) quantities are simultaneously recorded, generating the hit-word. The counters are arranged so that: BX increments by 1 at every master clock period and when it reaches 3563, the subsequent increment will make it roll back to 0. When this happens, ORBIT is incremented by 1. Within each clock period, the TDC circularly counts from 0 to 29.

In order to explain how the pre-processing block works one can refer to Figure 4.1. Each non-zero hit is recorded and saved in the input batch. This hit may come either from the scintillators pads coincidence signal or one of the detector's drift tubes. A scintillator coincidence (sHIT) is marked by a particular value of both the FPGA index and the readout channel (all scintillator hits only come from FPGA 1 at channel 128). Conversely, a drift tube hit (dHIT) can only be generated by the other

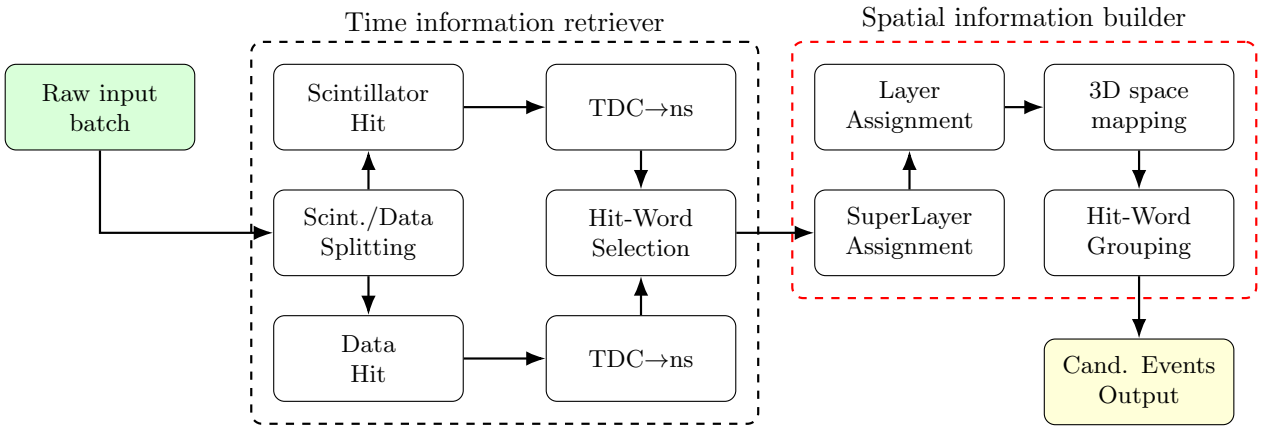


Figure 4.1: Main steps of the preprocessing stage.

channels (for either FPGA 1 or 2 and channel numbers ≤127). The pipeline is divided in two logical blocks: the first recovers time information while the second builds spatial information.

Time information retriever

This block translates the raw timing information that the readout electronics measured in clock ticks into a physically meaningful nanoseconds (ns) measure. This is done via the following equation:

$$t_{ns} = (BX_COUNTER \times 25ns) + (TDC_MEAS \times 25.0/30.0ns) \tag{4.1}$$

where for a sHIT this is called t_0 and for a dHIT instead is called tdc. The idea is thus to group the hits by their ORBIT_CNT value and then to obtain the drift time as:

$$t_{drift} = tdc - (t_0 + t_c), \tag{4.2}$$

where t_c is a calibration factor (more on this later). The physical approximation underlying this equation is that the drifting electrons in the tubes are orders of magnitude slower[1] than the muon traversing the detector. Finally, it is also required that the t_{drift} is within a valid time window, otherwise the hit-word is dropped.

Note It is also important to note that a study of the datasets showed that, within the same orbit (same value of ORBIT_CNT), there can be more than one scintillator coincidence event (t_0). When this happens, more than one hit is recorded in a given drift tube cell. Some examples of this phenomena are given later in Section 4.2.

Spatial information builder

This block builds the hit's spatial information, starting from the drift time estimate. The algorithm maps each FPGA board and channel that generated the signal to a specific drift tube of which the position is fully known. Hence, it is possible to calculate the position shift with respect to the tube center which is proportional to the drift time. The constant of proportionality is the drift velocity which is assumed to be constant:

$$\Delta x = v_{drift} \times t_{drift} \longrightarrow x_{L/R} = x_{wire} \pm \Delta x \tag{4.3}$$

where x_{wire} is the x-position of the center of the wire in the drift tube. Since there is no way of determining, at this time, whether the hit was shifted left or right with respect to the center of the tube, both the left-shifted (x_L) and right-shifted (x_R) positions needs to be computed.

[1] In fact, the drift velocity of the electrons is in the range of 5.38×10^{-2}mm/ns.

Output shaping

All of the operations described so far are conducted by means of cuDF. The final dataframe contains groups of filtered hit-words and each group thus represents a candidate event. As a final step, the dataframe is converted to a CuPy array with a specific shape:

$$(\mathrm{N}, 64, 8)$$

where N is the number of found candidate events and up to 64 hit words for each candidate event and 8 features are allowed. These features are listed in Table 4.2.

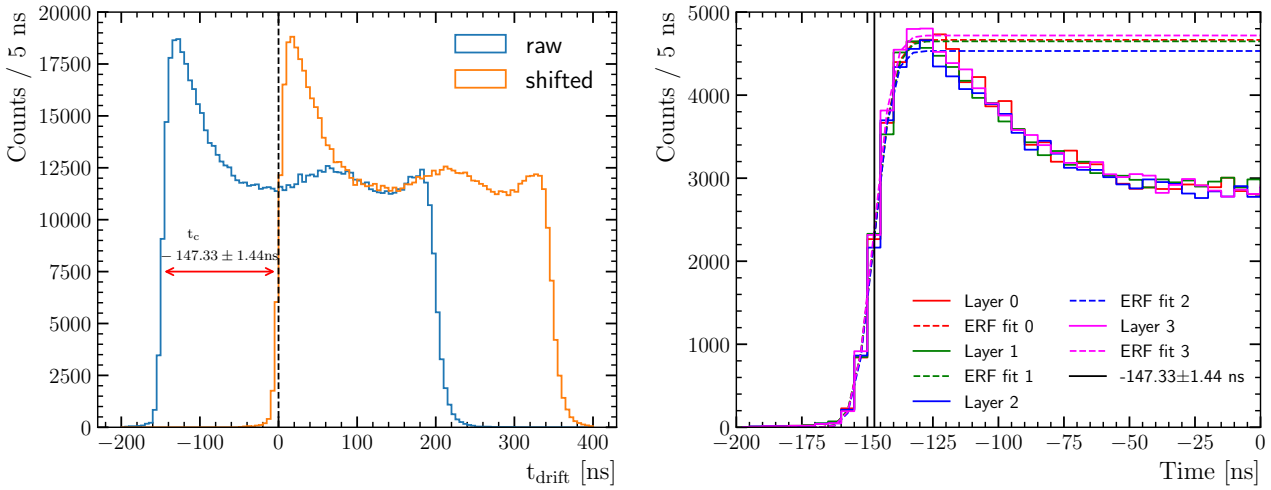
LAYER	X_LEFT	X_RIGHT	WIRE_Z_GLOB	WIRE_X_GLOB	t_drift	TDC_CHANNEL	AUX
int16	float32	float32	float32	float32	float32	int16	int16

Table 4.2: Schema of the GPU-resident output CuPy array.

This is passed over to the track reconstruction and event building stage.

4.1.2 Time calibration

It was mentioned before in Equation (4.2) that a calibration factor t_c is needed in order to correctly compute the drift time. This quantity takes into account the various delays that are induced by the electronics and that manifest themselves in the so called timebox plot.



(a) The raw and t_c corrected timeboxes for all of the four layers of the detector.

(b) The layer-wise ERF fits to the leading edge in order to recover the mean t_c.

Figure 4.2: Timebox calibration procedure showing both the initial and final, corrected, timeboxes and the construction used to determine the value for t_c.

The timeboxes are shown in Figure 4.2a. These are the histograms of the drift times calculated via Equation (4.2) with $t_c = 0$ for the raw and $t_c \neq 0$ for the shifted timeboxes. The timebox is an essential diagnostic tool and it shows how the electron drift times in the detector tubes are distributed in time. It has to have its rising edge near zero since this will account for the hits with minimal drift times (nearest to the pick-up wire). The distribution will then plateau and gradually fall back to zero near the region of maximum drift times (farthest from the pick-up wire and near the tube boundaries).

As one can see from the raw distribution, there is a time shift between the rising edge and the zero reference. This is precisely the t_c calibration parameter that was mentioned earlier. In order to determine it, the procedure outlined in Figure 4.2b was followed. The contributions to the full drift times distribution were separated for each of the four layers of the detector and an *erf* model was fitted to the rising edge:

$$\mathrm{erf}_{\mathrm{model}} = \frac{A}{2} \times \left(1 + \mathrm{erf}\left(\frac{t - t_0}{\sigma\sqrt{2}}\right)\right), \qquad \mathrm{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x \exp(-t^2)dt \tag{4.4}$$

where A, t_0, σ are to be determined by the fit procedure.

Layer	t_c [ns]	σ [ns]
4	-147.22	5.97
3	-147.25	6.02
2	-147.38	5.69
1	-147.46	5.71

Table 4.3: Layer-wise calibration fit parameters t_c.

The results reported in Figure 4.2b show good agreement between the fit estimates and, specifically, they are reported in Table 4.3. The final estimate for the common pedestal is obtained by taking the average of the per-layer t_c values shown in the table. Instead the estimation of the uncertainty requires a different approach. In fact, the estimates depend on the binning since the fits are performed against histograms. To check the stability of the fitting procedure, a bootstrap study was performed by fluctuating the histogram bin contents of each time distribution according to Poisson statistics and repeating the error-function fits. This study shows that the fitted midpoint is stable, giving a bootstrap spread of about 0.06 ns on the common pedestal. However, since the fits are performed on histograms with a bin width of $\Delta t = 5$ ns, the final uncertainty was conservatively assigned from the binning scale,

$$\sigma_{\mathrm{bin}} = \frac{\Delta t}{\sqrt{12}} = 1.44 \text{ ns}.$$

The common time pedestal was then found to be

$$t_0 = -147.33 \pm 1.44 \text{ ns},$$

Note on the bootstrap procedure The uncertainty was estimated with a bootstrap procedure applied to the binned time distributions. If n_i is the content of the i-th histogram bin, a pseudo-histogram was generated by replacing each bin content with a Poisson fluctuated value,

$$n_i^* \sim \mathrm{Poisson}(n_i).$$

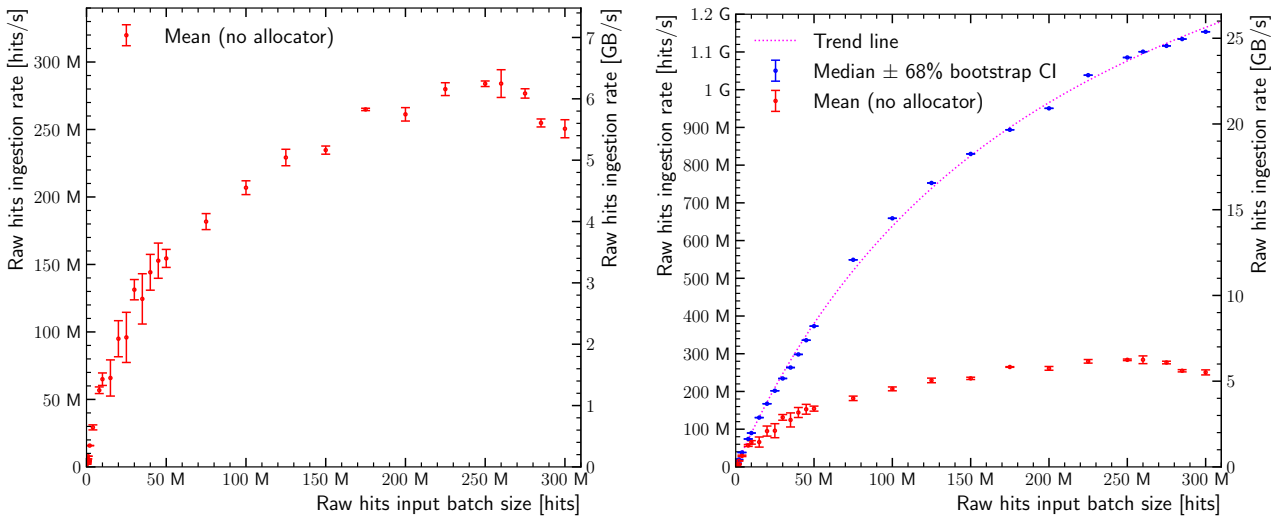
The same error-function fit was then repeated on the pseudo-histogram. Repeating this procedure many times gives a distribution of fitted pedestal values. The final pedestal was taken as the median of the distribution of the average pedestal across the four layers, while the uncertainty was estimated from its central 68% interval.

When this correction is taken into account in Equation (4.2), one obtains the shifted and zero-aligned timebox shown in Figure 4.2a. Since the position information of the hit directly depends on the measured drift time, the estimation of t_c is fundamental in order to yield consistent results.

4.1.3 Implementation and Performance analysis

With respects to the issue of implementing the pre-processing block on GPU and evaluating its performance, as stated earlier, a cuDF [18] dataframe is used to represent the raw hits input batch since cuDF offers very efficient ways to execute massively parallel transformations, indexing and grouping by index. The output of the pre-processor is instead a CuPy [19] tensor since this is particularly convenient for the next block that will perform track reconstruction and event building. This interplay between cuDF and CuPy required careful management of device memory in order to avoid undesirable situations as explained in detail in Section 5.1.

Specifically: in order to recast a cuDF dataframe into a CuPy tensor without a new and costly memory allocation, zero-copy operations were used (via DLpack [20]) and a unified memory allocator between cuDF and CuPy provided by Rapids Memory Manager [21] (RMM).



(a) Input throughput as function of batch size with no common allocator.

(b) Input throughput as function of batch size when RMM common allocator is in use.

Figure 4.3: Impact of the usage of the common cuDF - CuPy allocator on pre-processor performances. For both cases 100 repeated measurements were performed for each batch size.

This is crucial as Figure 4.3 shows. The figure in fact shows the main performance metric for the pre-processing block which is the input throughput. This is the amount of raw hits that the pre-processor can ingest and process in the unit of time and this is a function of the input batch size. One can see from Figure 4.3a that without a common allocator between cuDF and CuPy, the performance is dominated, at each batch size, by significant fluctuations. The distribution seems also to saturate and dip at around an input batch size of $250M$ hits. Instead Figure 4.3b shows how the behavior changes dramatically when the common RMM allocator is used compared to the previous no-allocator distribution with the same scale.

The results shown in Figure 4.3a and Figure 4.3b both follow the same experimental procedure that is described in Algorithm 1.

On the usage of the median. The repeated measurements occasionally exhibit slower executions, producing an asymmetric upper tail in the timing distribution and a corresponding lower tail in throughput. Since the arithmetic mean is sensitive to such observations, the median

$$\widetilde{R} = P_{50}\left(\{R_i\}_{i=1}^{N_{\text{repeat}}}\right)$$

was adopted as a robust estimator of the typical per-run throughput. This choice does not involve outlier rejection: all valid measurements are retained and the slower executions are analyzed separately through run-to-run dispersion and upper-tail latency metrics. The statistical uncertainty on the estimated median is obtained through non-parametric bootstrap resampling, with the 16th and 84th percentiles of the bootstrap distribution defining the reported 68% confidence interval.

Slow executions and upper-tail behavior. Although the central part of the timing distributions is generally narrow, a limited number of slower executions is observed for some input batch sizes. These measurements are not removed from the analysis, since no independent evidence indicates that they correspond to invalid timing samples. Instead, their effect is characterized separately from the central throughput estimate.

These slowdowns are shown in Figure 4.4 for each input batch size and in Figure 4.5 for the four selected batch sizes highlighted above (150 million acts as a control).

For each batch size, the upper timing tail is quantified through the relative slowdown of the 95th and

Algorithm 1 GPU preprocessing runtime measurement protocol

Require: Input batch D, N_{warmup}, N_{repeat}, N_{inner}, N_{boot}

for $j = 1, \ldots, N_{\text{warmup}}$ **do**
    Execute preprocessing on D
end for
for $i = 1, \ldots, N_{\text{repeat}}$ **do**
    Record start CUDA event
    for $j = 1, \ldots, N_{\text{inner}}$ **do**
        Execute preprocessing on D
    end for
    Record stop CUDA event
    Synchronize stop event
    Read aggregate elapsed time
    $t_i \leftarrow t_i^{\text{aggregate}} / N_{\text{inner}}$
    $R_i^{\text{hit}} \leftarrow N_{\text{hit}} / t_i$
end for
$\widetilde{R}^{\text{hit}} \leftarrow P_{50}\left(\{R_i^{\text{hit}}\}_{i=1}^{N_{\text{repeat}}}\right)$
for $b = 1, \ldots, N_{\text{boot}}$ **do**
    Draw with replacement N_{repeat} values from $\{R_i^{\text{hit}}\}$
    Compute the bootstrap median $\widetilde{R}_b^*$
end for
$CI_{68} \leftarrow \left[P_{16}\left(\{\widetilde{R}_b^*\}\right), P_{84}\left(\{\widetilde{R}_b^*\}\right)\right]$
$S_{95} \leftarrow 100\,[P_{95}(\{t_i\})/P_{50}(\{t_i\}) - 1]$
$S_{99} \leftarrow 100\,[P_{99}(\{t_i\})/P_{50}(\{t_i\}) - 1]$
return $\widetilde{R}^{\text{hit}}, CI_{68}, S_{95}, S_{99}$

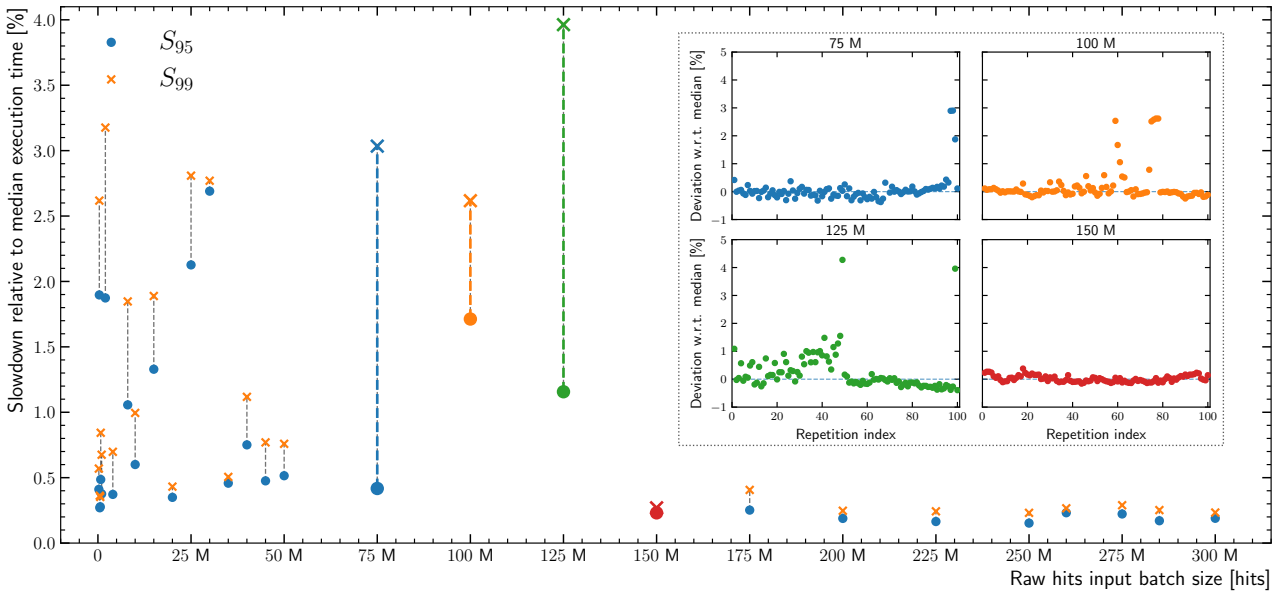


Figure 4.4: Upper-tail timing behavior of the GPU preprocessing stage. For each input batch size, the markers indicate the relative slowdowns S_{95} and S_{99} of the 95th and 99th execution-time percentiles with respect to the median execution time. Dashed vertical segments connect the two quantities obtained for the same batch, so that their length represents the extension of the extreme timing tail, $S_{99} - S_{95}$. The overlaid panels show the repetition-order evolution of the execution times for selected batch sizes, normalized to the corresponding batch median. Matching colors associate each highlighted batch in the main panel with its run-order sequence.

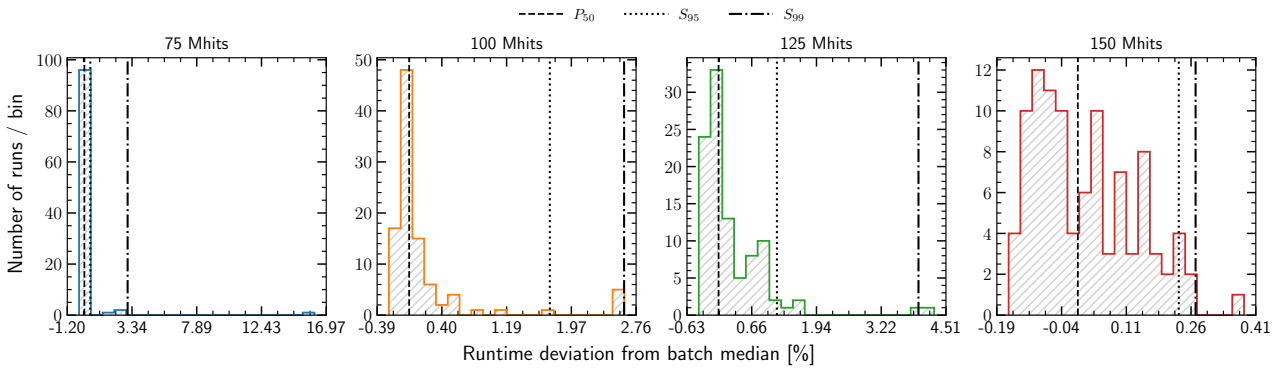


Figure 4.5: Distributions of the preprocessing runtime fluctuations for four representative input batch sizes: 75, 100, 125, and 150 million raw hits. For each batch size, the runtime measurements are expressed as the percentage deviation from the corresponding median, $\delta t_i = 100\,(t_i/P_{50}(t) - 1)$. The vertical dashed line marks the median, while the dotted and dash-dotted lines indicate the tail slowdowns S_{95} and S_{99}, respectively. Each distribution contains 100 timing measurements.

99th percentiles with respect to the median execution time:

$$S_{95} = 100\left(\frac{P_{95}(t)}{P_{50}(t)} - 1\right), \qquad S_{99} = 100\left(\frac{P_{99}(t)}{P_{50}(t)} - 1\right).$$

These quantities express, in percentage terms, how much slower the upper tail of the timing distribution is with respect to a typical execution. In particular, S_{95} provides a comparatively stable characterization of the slow tail, whereas S_{99} is more sensitive to the most extreme observations.

The repetition-order analysis shows that the slower measurements do not always appear as isolated outliers. For some batch sizes, they occur in short groups or transient intervals, indicating a temporary broadening of the timing distribution rather than a uniform shift of all executions. The effect is most visible for input sizes around 10^8 raw hits, while smaller and larger batches may exhibit a more tightly concentrated relative timing distribution. The monitored GPU temperature and clock frequencies remained stable during the measurements, providing no indication of thermal throttling as the origin of the observed slow executions. Possible explanations include transient runtime activity, dynamic memory-pool management, or changes in temporary workspace requirements.

4.2 Track reconstruction

The output of the pre-processing block is a $(N, 64, 8)$ CuPy tensor of float32 numbers, where N is the number of candidate events that have been found within the input batch. The Track Reconstruction and Event Building block consists in a group of custom CUDA kernels that, for each launch, process a number A of candidate events out of the N input candidates. These are described thoroughly in the following. The output of this block is a PyTorch tensor to be passed over to the final Anomaly Detection / Data Quality Monitoring block.

4.2.1 The left-right ambiguity problem

The major issue with drift cells making up the drift chamber is that one has a left-right ambiguity in the hit position as described in Figure 4.6.

When a signal is detected from a drift cell, one only gets to know the distance r from the central wire. One does not know, at this time, whether the hit occurred to the right or to the left of the wire. This is why one speaks of *left and right ambiguity* as previously introduced. Given the experimental setup described in Chapter 3, if one imagines a hit for each of four cells along the particle track, there will be 2^4 different possible left-right hit combinations to reconstruct the correct track. This is of course a great simplification since it can happen that multiple hits are recorded from the same

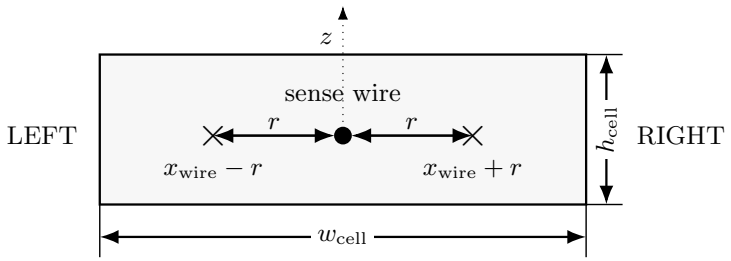


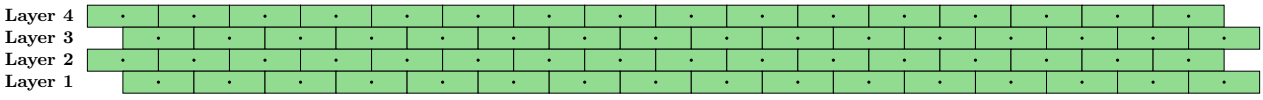
Figure 4.6: Schematic representation of the left–right ambiguity in a drift cell. The measured drift time determines the absolute distance $r = v_{\text{drift}} t_{\text{drift}}$ from the sense wire, but does not determine on which side of the wire the particle crossed the cell. The two compatible positions are therefore $x_{\text{wire}} - r$ and $x_{\text{wire}} + r$.

cell. So, for instance, if one was to consider two possible hits for each of the four cells, the number of possibilities ramps up to $4^4 = 256$. The problem of track reconstruction then naturally configures as a large combinatorial problem.

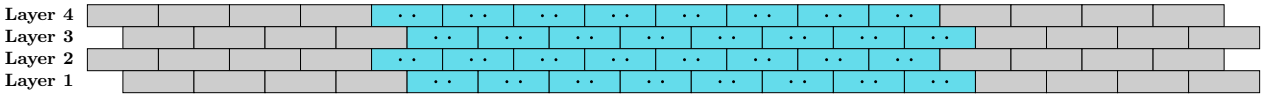
4.2.2 Initial assumptions

In order to handle the aforementioned combinatorial problem, two fundamental simplifications are introduced here and constitute the basis for the proposed solution.

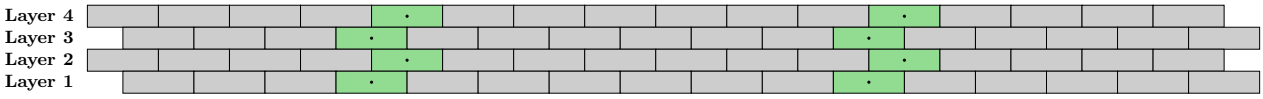
First assumption Reconstruction of candidate events is restricted to the ones with at most 64 hit-words, allowing up to 2 simultaneous hit-words for each cell. This is shown in Figure 4.7. So, for instance one could have a situation with the detector cells (64 total) with a hit each, or one could have 32 active cells with 2 hit-words each. These are two extreme situations which have never been observed in the experimental context[2] and thus this restriction is deemed reasonable. Each hit-word is specified by the 8 fields that were mentioned earlier in Table 4.2 and this is the reason for the GPU input tensor having dimensions $(N, 64, 8)$.



(a) Fully active detector with one hit for each of the 64 drift cells.



(b) Half active detector with 2 hits over 32 drift cells.



(c) Candidate event with two distinct candidate tracks.

Figure 4.7: The top and middle figures show the two extreme conditions within which the algorithm operates. Specifically the top figure shows a fully active detector, with a single hit being recorded for each drift cell. The middle figure shows instead the situation where half the cells are active with two hits being recorded in each of them. The bottom figure instead shows a very common experimental situation in which two distinct candidate tracks are present for a given candidate event.

Second assumption Given a candidate event, up to 16 distinct tracks can be built where each track can only have at most 1 active cell per layer and at least 3 or at most 4 total active cells.

[2]The reason for this is due to the very low rate of muons due to cosmic rays at ground level, even in the case of showers.

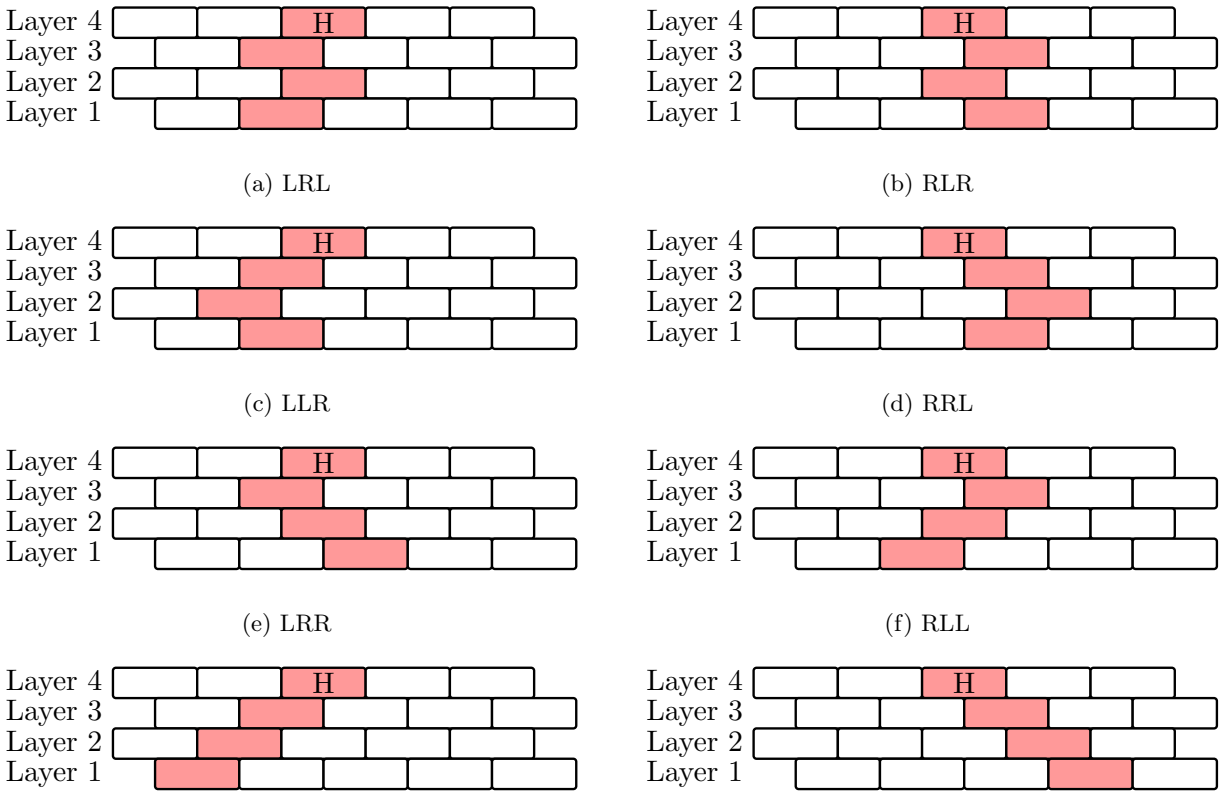
The GPU reconstruction algorithm processes sub-batches of A candidate events, where typically $A \leq N$ (in the opposite case the missing $A - N$ slots are simply filled with zeros and automatically ignored by the algorithm). For this reason, the reconstruction algorithm will be run a number:

$$\left\lceil \frac{N}{A} \right\rceil$$

of times for each pre-processed input batch that is presented at the input. The optimal number A is both device-dependent and application-dependent as it will be shown later. For each candidate event, as it was stated earlier, one can have up to 16 distinct tracks.

4.2.3 Geometrical track model

A geometrical hypothesis on the track itself is now made, defining the ways in which a track can occupy a volume of cells in the detector. Because of the previous assumptions in Section 4.2.2, any such volume will contain at most 4 cells and it will be contiguous. These volumes however are not arbitrary as displayed in Figure 4.8.



(a) LRL (b) RLR

(c) LLR (d) RRL

(e) LRR (f) RLL

(g) LLL (h) RRR

Figure 4.8: Logical volumes developing from the same top cell symbolized by the "H" character.

Starting from the top cell in layer 4, the so called **head-cell**, 8 different volumes can be developed. For instance in Figure 4.8a there is the Left-Right-Left volume which is called like this because, starting from the head-cell, the next active cell is to the immediate left of it. Going down further by one layer, the next active cell is to the right of previous and the final is to the left of the preceding one. These logical volumes are named *geometries* in the following.

Selecting one of the possible geometries, it is possible to repeat the construction for each compatible head-cell[3] and thus partition the detector volume. Within each construction, one has a combinatorial

[3]some geometries have different boundaries, for instance for LLR the first compatible head-cell will be the second from the left instead of the first

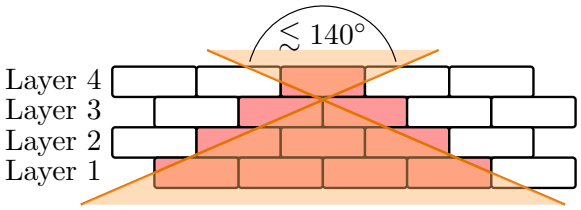


Figure 4.9: All geometries put together with the common head-cell, yielding a pyramid-like structure. Given the physical dimensions of the experimental apparatus, the most inclined tracks compatibile with the geometry resulted in a slope angle of $\approx 70°$ in both directions.

with up to 2 possible hit-words per cell, yielding 256 combinations for each construction. Essentially, within a given 4-cells volume one has 256 combinatorial track hypotheses compatible with that particular geometrical hypothesis. In fact, each volume acts as a constraint on the slope of the explainable tracks.

For instance if one defines the track angle as the angle between the track slope and the vertical line perpendicular to the detector layers (the z-axis in the present setup), then the LRL and RLR volumes will contain the tracks with the smallest angles (i.e. the most vertical ones) while, conversely, LLL and RRR will contain the ones with the largest angles. In Figure 4.9 all the possible volumes from a single head-cell are shown. Considering all the contributions from the volume partitioning operation, the physical dimensions of the detector and its positioning that does not introduce additional geometrical inefficiencies, the maximum track slope angle one can represent is of $\approx \pm 70°$.

4.2.4 Track reconstruction algorithm

The reconstruction algorithm acts in parallel over the A input candidate events, reconstructing up to 16 tracks for each one. For clarity one can consider a single candidate event as shown in Figure 4.10. The algorithm considers the 8 geometrical hypotheses, building each time a version of the detector with its volume partitioned according to that geometry. For a given geometry and for each head-cell, it then performs all the possible linear-fits separating the 4-points tracks and the 3-points tracks. For the 4-points tracks, for each head-cell, the best track is selected. For the 3-points tracks, the evaluation proceeds in the following manner: each detector layer is cyclically turned off and the fits are performed on the data of the remaining three layers. This time one will have again up to 2 hit-words per surviving cell, yielding $4^3 = 64$ combinations for each track volume with a chosen missing-layer. For each missing-layer configuration, the best 3-points track is chosen and then these best candidates are compared selecting the final 3-points track candidate for each head-cell. Finally, the results for each geometry are compared again keeping the separation between 3-points and 4-points tracks. At the end of this stage, each head-cell has zero or at most one best track, either 4-points or 3-points. If the final merging is requested, the algorithm will explicitly compare 4-points and 3-points tracks in order to produce a final output.

Linear fitting procedure

The linear fits are obtained by χ^2 minimization. In this case z position measurements are considered without errors while the x position measures are characterized by a fixed $\sigma_x = 400\mu$m. One thus has:

$$x(z) = a + b \cdot z \tag{4.5}$$

and by minimizing:

$$\chi^2 = \sum_{i=1}^{N} \left(\frac{x_i - (a + bz_i)}{\sigma_i} \right)^2 = \frac{1}{\sigma^2} \sum_{i=1}^{N} (x_i - (a + bz_i))^2 \tag{4.6}$$

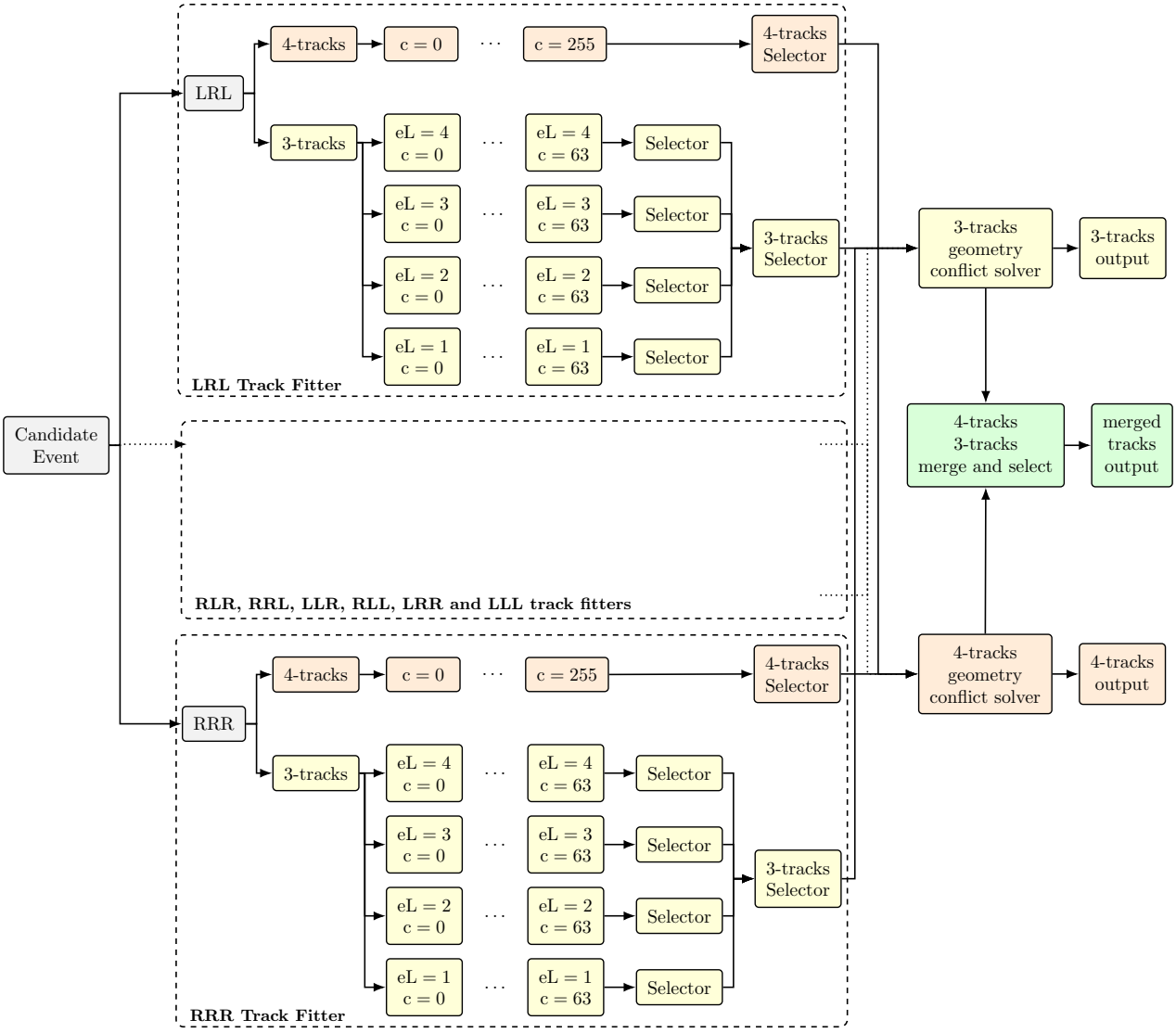


Figure 4.10: Schematic representation of the track reconstruction algorithm. For readability requirements only LRL and RRR track fitters are shown. Inside each track-fitting block (so for a fixed geometry), the algorithm performs all the combinatorial track fits for the case of 4-point tracks and for 3-point tracks. In the case of 3-point tracks, a layer is forcibly excluded each time (eL in the diagram). The algorithm can output either 3-point tracks only, 4-point tracks only or a mixture of 3-point and 4-point tracks obtained by a merging and selecting procedure.

Algorithm 2 Track fitting algorithm for a geometry G with 4-points tracks at a given head-cell H

Require: Input event E, Head-cell H, Geometry index G, Data Combination index C

function GETDATA(E, H, G, C)
    trackVolumeCells ← getTrackVolume(H, G)
    Initialize dataPoints
    for $j = 1, \ldots, 4$ **do**
        dataPoints[j] ← getEventData(E, trackVolumeCells[j])
    end for
    dataPoints ← getDataCombination(C, dataPoints)
    return dataPoints
end function

Require: Input event E, Geometry G, Head-Cell H

function GETBESTTRACKAT(E, G, H)
    $\vec{z} \leftarrow (z_4, z_3, z_2, z_1)$
    Initialize bestTrackParams
    for $C = 1, \ldots, 256$ **do**
        $x_4 \leftarrow \text{getData}(E, H, G, C)[4]$
        $x_3 \leftarrow \text{getData}(E, H, G, C)[3]$
        $x_2 \leftarrow \text{getData}(E, H, G, C)[2]$
        $x_1 \leftarrow \text{getData}(E, H, G, C)[1]$
        $\vec{x} \leftarrow (x_4, x_3, x_2, x_1)$
        fitParams ← fit($\vec{x}, \vec{z}$)
        if trackSelector(fitParams, bestTrackParams) **then**
            bestTrackParams ← fitParams
        end if
    end for
    return bestTrackParams
end function

one has:

$$\begin{cases} \Delta = NS_{zz} - S_z^2 \\ a = (S_{zz}S_x - S_zS_{xz})/\Delta \\ b = (NS_{xz} - S_xS_z)/\Delta \end{cases} \quad , \quad \begin{cases} \sigma(a) = \sigma_x\sqrt{S_{zz}/\Delta} \\ \sigma(b) = \sigma_x\sqrt{N/\Delta} \end{cases} \tag{4.7}$$

where $S_x = \sum_i x_i$, $S_z = \sum_i z_i$, $S_{xx} = \sum_i x_i^2$, $S_{zz} = \sum_i z_i^2$ and $S_{xz} = \sum_i x_i z_i$. As it will be shown in the following section, the actual implementation uses an equivalent formulation that is more computationally efficient.

Track selection procedure

In both Figure 4.10 and Algorithm 2 the concept of track selection is introduced. At first glance selecting between tracks with the same number of points would seem to require a simple χ^2 comparison. But χ^2 only quantifies how good the fit is statistically and this does not automatically imply that it is equally physically meaningful. In this case in fact one does not know a-priori which combination of points gives the true track or even if such object exists in the first place. It is then important to also consider physical information in the selection procedure. Further, in the actual algorithm implementation, some constraints force the adoption of tie-breaking protocols since it can happen that, within numerical precision, two or more tracks could have equal χ^2, leading to an undesirable situation in which the algorithm is unable to discriminate between competing objects. This is essentially due to having constant and uniform σ_x and zero uncertainty on the z component of the position.

It is again important to notice the fact that, and this will be specified even further in the next section, the logical volumes / geometries that were just introduced are different classes of geometrical hypotheses since they combine information not only on left-right ambiguity but also on cell topology

and track slope. For this reason, the algorithm executes the comparisons firstly between tracks of the same kind (same number of points and same geometry) and secondly between tracks with the same number of points but different geometries to resolve the possible conflicts. The comparison between tracks with different number of points is deferred until the very last step.

Discriminating between tracks with the same number of points The procedure described here applies equally to both 4-points tracks and 3-points tracks. The selection rule uses χ^2 as the primary discriminator but, in the case of the competing tracks having nearly equal χ^2 (within a configurable precision), the slope information is taken into account, acting as a physical bias. Specifically, a penalty to the tracks that have a slope that is larger than a limiting value that one considers as a boundary between physical and un-physical tracks with respect to the established experimental setup.

Algorithm 3 Discriminating between tracks with the same number of points

Require: reference χ^2_{ref}, slope b_{ref}, head-cell h_{ref}, candidate χ^2, slope b, head-cell h

function CHOOSECANDIDATE_SAME(χ^2_{ref}, b_{ref}, h_{ref}, χ^2, b, h)
  $\epsilon_\chi \leftarrow 10^{-5} \cdot \max(\max(\chi^2_{\text{ref}}, \chi^2), 1.0)$
  $\epsilon_b \leftarrow 10^{-6} \cdot \max(\max(|b_{\text{ref}}|, |b|), 1.0)$
  $\chi_{\text{tied}} \leftarrow (|\chi^2_{\text{ref}} - \chi^2| \leq \epsilon_\chi)$
  $\text{same}_{\text{tied}} \leftarrow [\chi_{\text{tied}} \wedge (|b_{\text{ref}} - b| \leq \epsilon_b)]$
  if $\text{same}_{\text{tied}}$ **then**
    Full degenerate case, deterministic tiebreak.
    return $(\text{h} < \text{h}_{\text{ref}})$
  end if
  if χ_{tied} **then**
    $\text{score}_{\text{ref}} \leftarrow (\chi^2_{\text{ref}} + \lambda \cdot \text{slopePenalty}(b_{\text{ref}}))$
    $\text{score} \leftarrow (\chi^2 + \lambda \cdot \text{slopePenalty}(b))$
    return $(\text{score} < \text{score}_{\text{ref}})$
  end if
  return $(\chi^2 < \chi^2_{\text{ref}})$
end function

The pseudocode for this selection rule is shown in Algorithm 3, where the function returns a boolean True when the candidate track is considered more desirable than the current reference one. Also, the rule employs a deterministic tie-break as a last resort in case of full collapse of the physical selectors (χ^2 and track slopes).

Moreover, the function uses the track slopes instead of the angles because, as it will be shown later, it is more computationally efficient. The slope penalties are furnished by a programmable function (in Algorithm 3 it is labelled *slopePenalty*), thus allowing to customize the selection rule depending on the experimental requirements. Figure 4.11 shows the function that was considered for this work. It is a box function that is described by the following equation ($b_0 > 0,\ \ b_1 > 0$):

$$\text{penalty}(b) = \begin{cases} 0, & |b| \leq b_0 \\ -\log\left[\frac{1}{2} \cdot (1 + \cos(\pi \cdot u))\right], u = \frac{b - b_0}{b_1 - b_0}, & b_0 < |b| < b_1 \\ k, & |b| \geq b_1 \end{cases} \tag{4.8}$$

The penalty is then applied, with a weight λ, to compute a score for the reference and candidate track as shown in the pseudocode. The track with the lowest score wins in case of equal χ^2, otherwise the track with the smaller χ^2 wins. This same procedure is also used when the algorithm has to compare the same number of points tracks of different geometries. This happens because the algorithm needs to select at most one track for each head-cell, making sure that there are no collisions (intersecting track volumes / geometries).

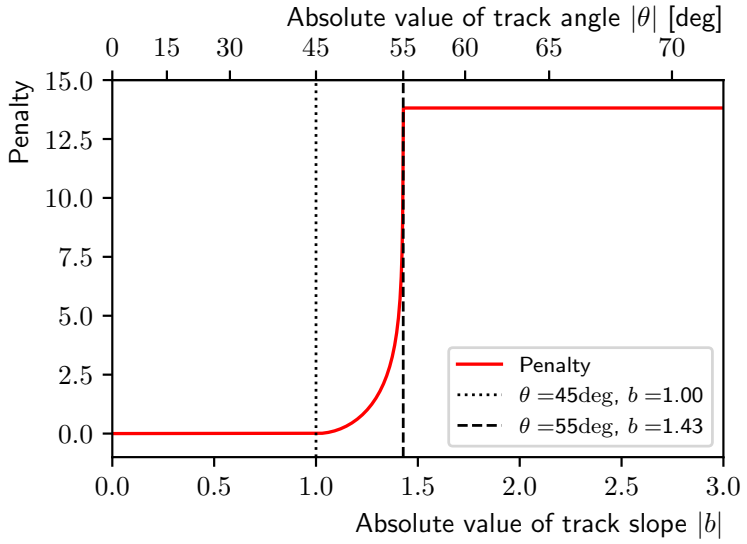


Figure 4.11: The penalty function is a box-function and it applies a non-zero, rapidly increasing penalty when the slope angle $\theta \geq 45°$ and applying the maximum penalty for all angles $\theta \geq 55°$. This is entirely configurable so that the algorithm is adaptable to the specifics of the experimental setup.

Geometrical conflict resolution As briefly introduced before, when tracks of different geometries are compared, conflicts between their respective track volumes may arise. These conflicts are evaluated firstly between the same number of points tracks and then, if requested, also between tracks with different number of points.

- In the first case, the algorithm selects, for each head-cell, only one track using the same selection rule that was described earlier in Algorithm 3. Then, it proceeds to evaluate the presence of intersecting track volumes that signal a conflict. The intersecting volumes are clusters of tracks and for each cluster, the optimal track is selected using again the same rule as before.
- In the second case, the procedure is identical with respect to the conflict resolution but this time the selection rule is different, as described in the next paragraph.

Discriminating between tracks with different number of points The algorithm can be configured to also compare tracks with different number of points. This is done at the very end when the best candidates have been produced for both the 4-points and 3-points tracks. In this case the algorithm uses the rule whose pseudocode is shown in Algorithm 4 and Algorithm 5.

The idea now is that when the algorithm compares tracks with different number of points, it needs to use the p-value and not the χ^2 as primary discriminating parameter and also it needs to consider the number of points of the fitted track. A 4-points track could be more physically meaningful than a 3-points track with the same fit quality. In this case however it is advantageous to consider the $-\log \mathrm{p}$ instead and a look-up table with the pre-computed values. Specifically one has that:

- For the 4-points tracks, there are $N - 2 = 2$ degrees of freedom and in this case the $\mathrm{p} - \mathrm{value}$ distribution is given by (χ^2 labels the reduced chi-squared):

$$p = \exp\left(-\chi^2\right) \longrightarrow -\log(p) = \chi^2$$

- For the 3-points tracks, there are $N - 2 = 1$ degrees of freedom and in this case the $\mathrm{p} - \mathrm{value}$ distribution is given by:

$$p = \mathrm{erfc}\left(\sqrt{\frac{\chi^2}{2}}\right) \longrightarrow -\log(p) = -\log \mathrm{erfc}\left(\sqrt{\frac{\chi^2}{2}}\right)$$

  and these values are pre-computed in the look-up table.

Algorithm 4 Discriminating between tracks with different number of points

Require: reference N_{ref}, χ^2_{ref}, slope b_{ref}, candidate N, χ^2, slope b

function CHOOSECANDIDATE_DIFF(N_{ref},χ^2_{ref}, b_{ref}, N, χ^2, b)
  cPscore ← trackPValueScore(N, χ^2) ▷ getting $-\log$ p for the candidate
  rPscore ← trackPValueScore(N_{ref}, χ^2_{ref}) ▷ getting $-\log$ p for the reference
  $\epsilon \leftarrow 10^{-5} \cdot \max(\max(|\text{cPscore}|, |\text{rPscore}|), 1.0)$
  $\text{score}_{\text{tied}} \leftarrow (|\text{cPscore} - \text{rPscore}| \leq \epsilon)$ ▷ checking if tie exists for the score
  thr ← 10.0 ▷ threshold to prefer a worse fitted 4-point track over 3-point
  if $N > N_{\text{ref}}$ **then**
    moreScore ← cPscore
  else
    moreScore ← rPscore
  end if
  if moreScore < thr **then**
    return ($N > N_{\text{ref}}$) ▷ choose track with more points if below threshold
  end if
  if cPscore < (rPscore $- \epsilon$) **then**
    return True ▷ choose track with less points provided its fit is not that bad
  end if
  if cPscore > (rPscore $+ \epsilon$) **then**
    return False ▷ keep the reference track
  end if
  cScore ← cPscore $+ \lambda \cdot$ slopePenalty(b) ▷ if $-\log$ p is tied then fall back to slope prior
  rScore ← rPscore $+ \lambda \cdot$ slopePenalty(b_{ref})
  if cScore < rScore **then**
    return True
  end if
  if cScore > rScore **then**
    return False
  end if
  return ($N > N_{\text{ref}}$) ▷ if also slope-tied, prefer more hits
end function

Algorithm 5 Track $-\log$(p) score retriever from pre-computed look-up table

Require: reduced χ^2, $-\log(p)$ look-up table LUPT.

function TRACKPVALUESCORE(N, χ^2)
  if N is 4 **then**
    return χ^2 ▷ in this case $-\log(p) = \chi^2$
  end if
  if N is 3 **then**
    $\chi^2_{\text{max}} \leftarrow 50$
    if $\chi^2 \geq \chi^2_{\text{max}}$ **then**
      return LUPT[size(LUPT) − 1]
    end if
    $x \leftarrow \chi^2 \cdot [\text{size(LUPT)} - 1] / \chi^2_{\text{max}}$
    $i \leftarrow \lfloor x \rfloor$
    $t \leftarrow x - i$
    $p_0 \leftarrow \text{LUPT}[i]$
    $p_1 \leftarrow \text{LUPT}[i + 1]$
    return ($p_0 + t \cdot (p_1 - p_0)$) ▷ return an interpolation
  end if
  return 10^{30} ▷ in case of invalid track
end function

Because of the logarithm of the p-value, a lower value now signals better compatibility. The selector will firstly check whether the $-\log p$ values are tied. If they don't, it will choose the track with more points even if it has a worse fit quality, provided that it does not exceed a threshold value. If on the contrary the fit quality is too degraded, the track with fewer points is preferred, provided that its $-\log p$ is not much larger than a threshold. If instead the values are tied, the slope information is added as in the previous case and the track with the lowest total score wins. If this is again tied, the track with more hits is preferred as a fall-back solution.

4.2.5 Track reconstruction algorithm implementation with CUDA kernels

As it was mentioned earlier, the combinatorial problem is a naturally parallel problem. For instance, if one considers a given geometry, in the case of 4-points tracks there are 256 possible realizations of that geometric hypothesis each corresponding to different data point values. This is replicated for each geometry, and the same happens with the 3-points tracks where instead one also has the excluded layers so that for a given geometry there are only 64 combinations for each excluded layer (again $4 \times 64 = 256$ combinations). This maps naturally to a GPU implementation which in this work was done with CUDA on NVIDIA GPU. One of the consequence of this is that it will show an interesting property of the track geometries effectively pairing them so that each geometry will appear in a doublet.

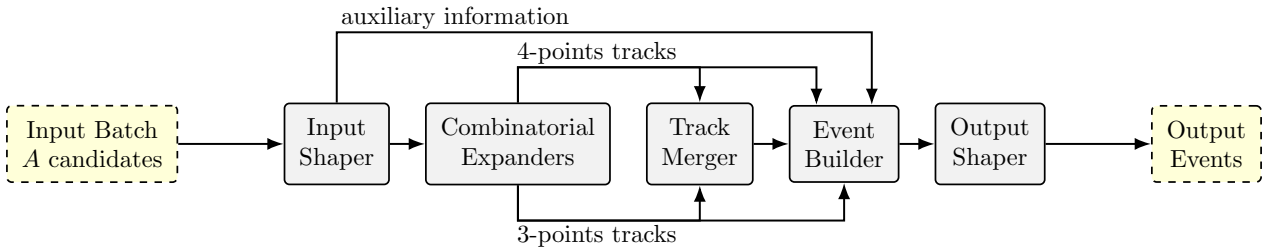


Figure 4.12: Schematic diagram of the kernel chain. The kernels are represented by the grey-filled boxes.

Kernel structure The algorithm is implemented by means of the kernel chain depicted schematically in Figure 4.12. The input batch is a subset of the pre-processed input batch containing all of the candidate events, as previously stated. The kernels are described in detail in the following.

Input shaper kernel (gridSize $= (1, 1, A)$, blockSize $= (64, 1, 1)$)

The input consists of the $(A, 64, 8)$ tensor that was described earlier. The kernel then builds, starting from this tensor, a detector-friendly representation. Each block handles a candidate event (indexed by $blockIdx.z$). Within a block, each of the 64 threads controls a detector drift cell. First the threads load the data from global memory into shared memory and then each thread checks if there is data for its own detector cell, saving up to two hit-words.

To describe the rest of the operation, one has to refer to Figure 4.13. Each thread in the figure (grey boxes) contains the thread index (i.e. T0) and the details of the detector cell location that it handles, for instance T0 handles (L4,C0) which means: cell 0 of layer 4. So along a layer, one has 16 threads, each handling its own cell. Each thread saves up to two hit-words for its cell and these are saved as doublets (L_0, R_0) and (L_1, R_1). The data is then saved to global memory using the following indexing formula that produces the schema shown in Figure 4.13 (yellow boxes):

$$\begin{cases} \text{out}_{L0} = 0 + \text{cellIdx_row} \cdot 2 + W + P \cdot 32 \\ \text{out}_{R0} = 64 + \text{cellIdx_row} \cdot 2 + W + P \cdot 32 \\ \text{out}_{L1} = 128 + \text{cellIdx_row} \cdot 2 + W + P \cdot 32 \\ \text{out}_{R1} = 192 + \text{cellIdx_row} \cdot 2 + W + P \cdot 32 \end{cases} \tag{4.9}$$

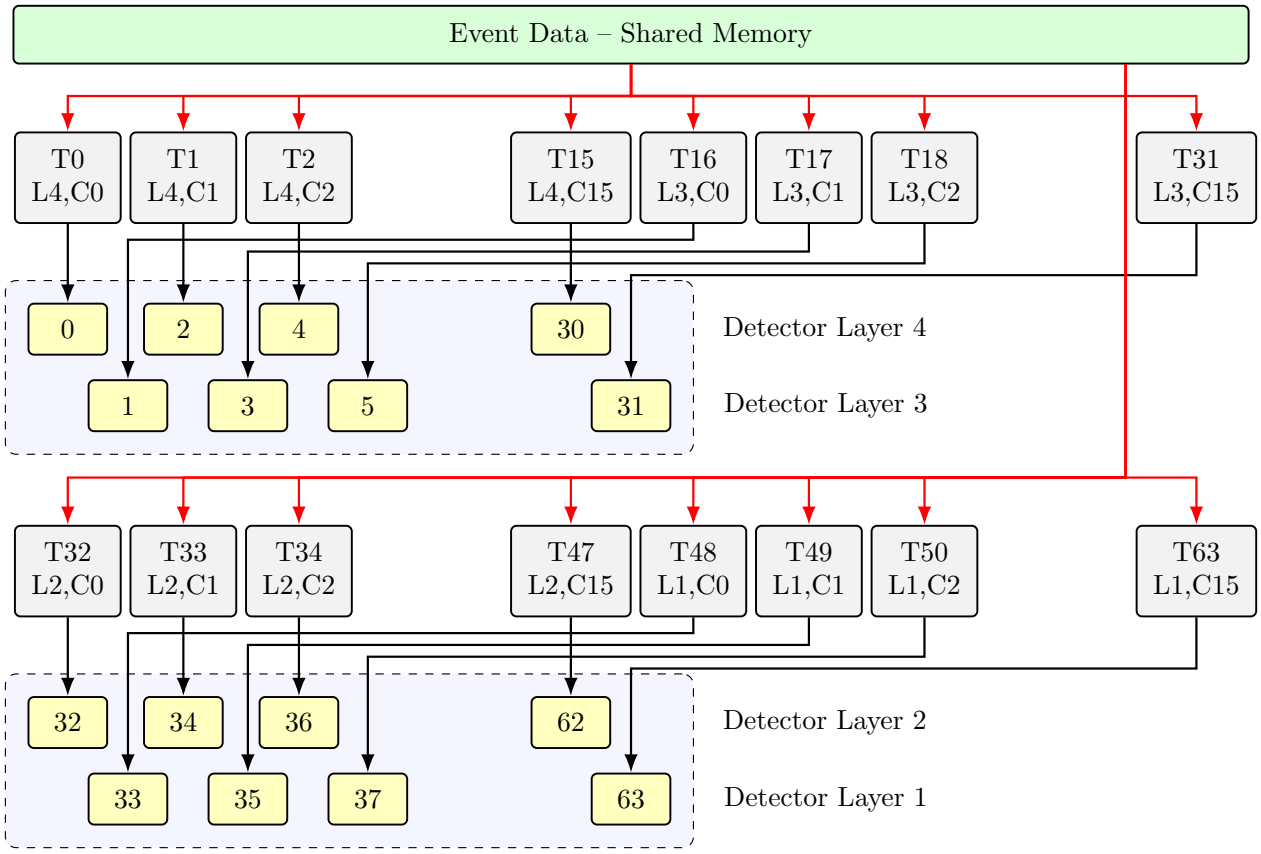


Figure 4.13: Schematic representation of the input shaper kernel. The grey boxes represent the threads, the red arrows are the data loading paths from shared memory and the black arrows are instead the data writing paths to global memory for the L_0 features whose memory locations are pictured with the yellow filled boxes.

where cellIdx_row = threadIdx.x%16, P = layerId $\gg 1$, W = layerId&1 and layerId = threadIdx.x $\gg 4$. Figure 4.13 shows, for readability, only the saving paths for L_0. The other memory indexes are fully computed in Table 4.4.

threadIdx.x	Layer	Cell	L0 address	R0 address	L1 address	R1 address
0	4	0	0	64	128	192
1	4	1	2	66	130	194
...	...	...	...	...	...	...
15	4	15	30	94	158	222
16	3	0	1	65	129	193
17	3	1	3	67	131	195
...	...	...	...	...	...	...
31	3	15	31	95	159	223
32	2	0	32	96	160	224
33	2	1	34	98	162	226
...	...	...	...	...	...	...
47	2	15	62	126	190	254
48	1	0	33	97	161	225
49	1	1	35	99	163	227
...	...	...	...	...	...	...
63	1	15	63	127	191	255

Table 4.4: Global memory indexes for the L_0, R_0, L_1, R_1 data features.

Of course these indexes have an additional offset that takes into account the event index itself. Since

each input candidate event takes 256 memory locations in global memory, the full indexing is given by:

$$\text{gMem}_X = (\text{blockIdx.z} \cdot 256) + \text{out}_X$$

where $X \in \{L_0, R_0, L_1, R_1\}$. For completeness, there are also two other helper features that are saved to two separate global memory arrays since they will be used only later at the event building stage. These features contain information regarding the drift times of the hits and their local index within the input tensor.

The reason why this global memory arrangement was used for the output is not random. It is instead crucial for the remainder of the kernel chain since it allows for the **efficient mirroring** of the detector geometry directly in the hardware organization of the GPU itself, as it will be explained in the following.

Combinatorial expander kernel $(\text{gridSize} = (64, 4, A), \text{blockSize} = (64, 4 \vee 8, 1))$

This kernel (full profile available in Appendix B.1.1) receives the input from the *input shaper* and it expands the combinatorial. As in the previous kernel, blockIdx.z indexes each pre-processed candidate. This kernel can be launched with two block sizes, depending on the desired operation:

- Building only 4-points tracks: $\text{blockSize} = (64, 4, 1)$,
- Building 3-points and 4-points tracks: $\text{blockSize} = (64, 8, 1)$,

and the default operation is with 4-points tracks only, given this specific application.

The first thing this kernel does is load the data that was previously written in global memory and because of the specific indexing that was used (Equation (4.9)), a nice structure naturally emerges.

threadIdx.x	warpIdx	reads from	layer	cellIdx row	cellIdx global
0	0	0	4	0	0
1	0	1	3	0	1
...	...	...	...	...	...
30	0	30	4	15	30
31	0	31	3	15	31
32	1	32	2	0	32
33	1	33	1	0	33
...	...	...	...	...	...
62	1	62	2	15	62
63	1	63	1	15	63

Table 4.5: Combinatorial kernel global memory access pattern.

Considering threadIdx.x first and the L_0 feature: the 64 threads represent the detector cells and each thread will read the memory sequentially, but because of the way in which the memory was indexed at the previous kernel, the reads occurs as in Table 4.5.

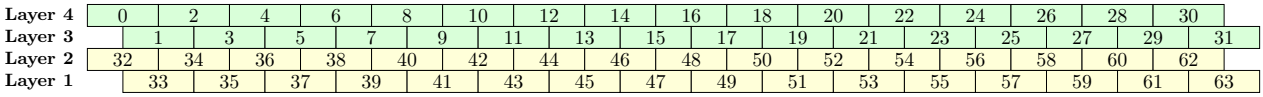


Figure 4.14: Thread to Detector cell mapping. The upper two layers and lower two layers are grouped by color to better emphasize the two warps.

This is shown graphically in Figure 4.14. One can see how the first warp effectively controls the first half of the detector (layers 4 and 3) while the second warp handles the second half of the detector (layers 2 and 1). Thanks to the memory indexing, the structure of the detector is now mirrored in the GPU execution model itself. This has crucial consequences since it allows to exploit *warp-level*

operations [22], which are very fast and efficient, to perform linear fits and geometry collision detection and solving. Further, because of the warp operations, the eight geometries that were introduced earlier are always paired in 4 geometry doublets and because of this they are processed much faster.

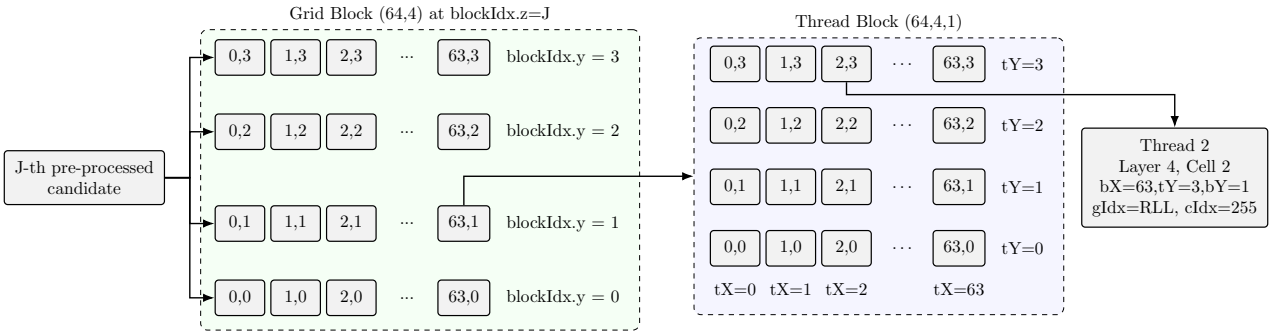


Figure 4.15: Schema depicting the threads organization in the combinatorial expander kernel for a single pre-processed candidate (of progressive index J). A number A of these schemas is therefore stacked.

Figure 4.15 shows a simplified schematic of the threads organization for this kernel for a single input pre-processed candidate. Before delving deeper into the kernel architecture and algorithm, it is important to explain what each index does.

Index threadIdx.y The index works in two different ways depending on its value.

- threadIdx.y = 4 In this case the combinatorial only considers 4-points tracks. This index acts together with blockIdx.x to determine the data distribution index, ranging from 0 to 255:

$$\text{combIdx} = \text{blockIdx.x} + 64 \cdot \text{threadIdx.y}$$

  In this way, when the kernels are launched, each thread will know which hit to choose between L_0, R_0, L_1, R_1. The logic also handles automatically the situations in which no hit-words or only one is available at any given thread/cell.

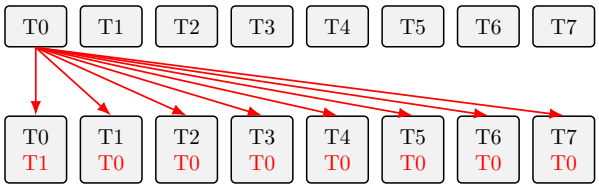
- threadIdx.y = 8 In this case, while the lower values threadIdx.y = $\{0, 1, 2, 3\}$ act as in the previous case for the 4-points tracks, the upper values threadIdx.y = $\{4, 5, 6, 7\}$ are used to determine which of the four detector layers to turn off while blockIdx.x determines the 64 possible data combinations for each excluded-layer configuration, allowing to also build the 3-points tracks.

In the following and for brevity only 4-points tracks operation will be considered since it is the one relevant for this work.

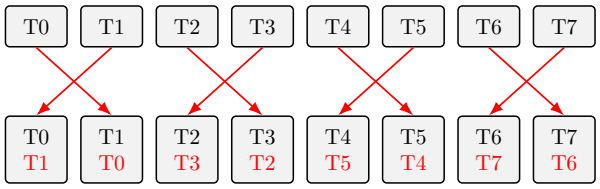
Index blockIdx.y The index is used to generate the geometry doublets. This index and how the geometries are paired in doublets is explained in the following paragraph.

Warp-level operations and geometry pairings Here the warp-level operations that are used to compute the linear fits in this kernel are described along with the geometry pairings that were introduced earlier, explaining the role of the blockIdx.y index.

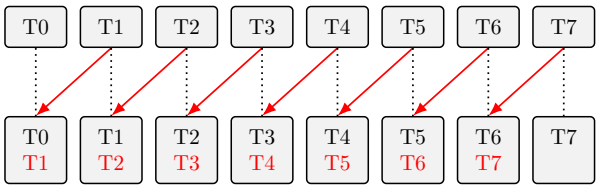
A warp-level operation occurs only within threads of the same warp. These operations are very efficient since they do not use global or shared memory but only registers [22]. In this specific case, the most frequently used operation is the *shuffling*. This exists in several flavors as shown in Figure 4.16, allowing for a thread to broadcast a value to all the others, or also to read data from up or down the threads as if they were a shift register. The idea behind shuffling is that when a thread has a local register variable a, it can also access that same variable from other threads in the warp. For instance in order for a thread to read the value of that variable from the next thread, a simple *shuffle_down*



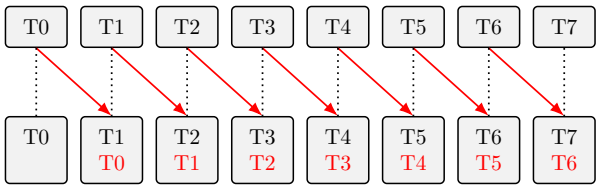
(a) Shuffle operation (broadcast): each thread in the warp can broadcast to the others.



(b) Shuffle xor (or butterfly shuffle): threads are paired in doublets and can exchange data between themselves.



(c) Shuffle down: i-th thread reads data from the $i+1$-th.



(d) Shuffle up: i-th thread reads data from the $i-1$-th.

Figure 4.16: Abstract representation of shuffle, shuffle xor, shuffle up and down operations with offset 1. With larger offsets or stacked-like operation it is possible to achieve greater complexity.

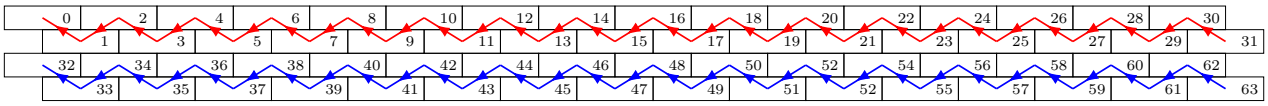


Figure 4.17: Full detector view with shuffle_down_sync operation highlighted (offset 1). The direction of the arrows highlights the data access path.

operation with offset 1 is required as shown in Figure 4.16c. Conversely to read from the preceding thread, a shuffle up is required as shown in Figure 4.16d.

However, the most useful application of these operations in the present work comes when the two warps are considered together as in Figure 4.17, where the whole detector is shown keeping in mind again that each drift cell is managed by its own thread. The arrows mark the direction of the data access path, so for instance the arrow pointing from thread-5 to thread-4 means: *thread-4 can access data from thread-5*, and so forth.

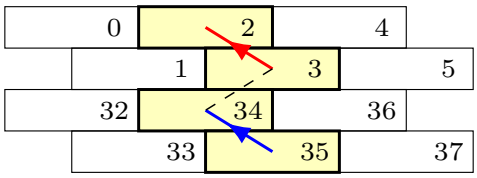


Figure 4.18: Zoomed in region around cells 2-3-34-35.

To better explain this point, Figure 4.18 shows a zoomed-in region around cells 2 and 3 of the upper half and 34 and 35 of the lower half, highlighting the four cells. As it was previously shown in Equation (4.6), computing the linear fits involves several summation operations. For instance considering $\sum_i x_i$ for each of the four highlighted cells, thanks to the shuffles, the operation is realized as detailed in the following:

- **Upper warp**: thread-2 has its x_2 (this could be either the x-coordinate of hit-word 0 or hitword-1 for instance L_0, R_0, L_1, R_1) and it can receive the x from thread-3 via the shuffle, x_3, summing it locally with x_2 inside thread-2, obtaining:

$$\sum_{\text{up}} = x_2 + x_3$$

- **Lower warp**: the same exact operation occurs in thread-2 of the lower warp (cell-34) receiving data from thread-3 of the lower warp (cell-35) and so thread-2 of the lower warp holds:

$$\sum_{\text{down}} = x_{34} + x_{35}$$

Placing $\sum_{\text{down}}$ in shared memory and reading vertically from thread-2 of the upper warp (so the upper warp thread-2 reads the corresponding thread-2 of the lower warp which is 34 in the diagram) and summing the partial sums one obtains:

$$\sum = \sum_{\text{up}} + \sum_{\text{down}} = x_2 + x_3 + x_{34} + x_{35},$$

and similarly for all the other sums ($\sum_i x_i^2$, $\sum_i z_i^2$, ...) that are required to perform the fit (it suffices to do several shuffles, one for each variable to be summed). So thread-2 of the upper warp (or cell-2) will hold the candidate track and for this reason cell-2 is called **head-cell** (as previously introduced). It is also worthy to point out how the highlighted cells form one of the eight geometrical patterns: right-left-right (RLR). Further, the head-cell is **even**. But there is more on this since another geometrical pattern appears upon considering again the very same shuffles.

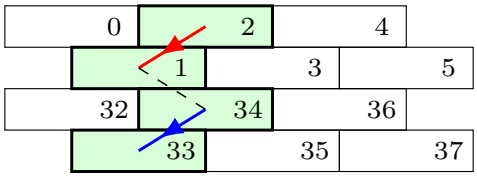


Figure 4.19: Zoomed in region around cells 2-1-34-33.

Considering again the same zoomed region as Figure 4.18 but this time, as shown in Figure 4.19, it is thread-1 of the upper warp that holds:

$$\sum_{\text{up}} = x_1 + x_2$$

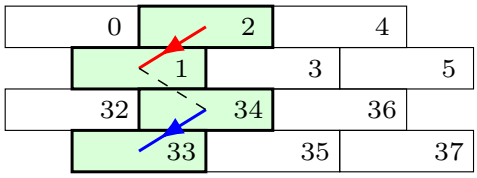
since it has read x_2 via the shuffle-down operation from thread-2. Similarly in the lower warp one has $\sum_{\text{down}}$ being held by thread-1 (thread-33 or cell-33 in the diagram). Placing the down-sum in shared memory and summing vertically in thread-1 of the upper warp one has:

$$\sum = \sum_{up} + \sum_{down} = x_1 + x_2 + x_{33} + x_{34},$$

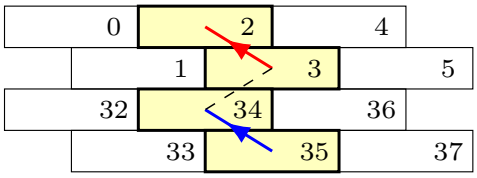
This time the head-cell is 1 which is **odd**. It is important to notice how the geometrical pattern now is left-right-left (LRL). The **key result** is that with a single shuffle operation of offset 1 and summing vertically the half-sums between the two warps, two kinds of fits with two geometrical hypotheses are obtained: LRL and RLR. **The two geometries appear then as doublets**.

One interesting property of the doublets is that the right-beginning geometries have an even head-cell, while the left-beginning ones have an odd head-cell. This allows even threads to hold right-beginning candidate tracks and odd threads to hold left-beginning tracks. When considering tracks of the same geometry doublet, they will share at least one cell and so it is necessary to choose only one since they have to be mutually exclusive (no two distinct tracks may share one or more cells). This will be considered in the next paragraph.

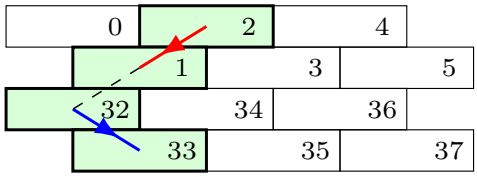
Similar constructions, combining up and down shuffles and different offsets can be done to obtain the other geometries that are paired as shown in Figure 4.20. For instance in the case of LLR and RLL (Figure 4.20c and Figure 4.20d) the construction is as follows:



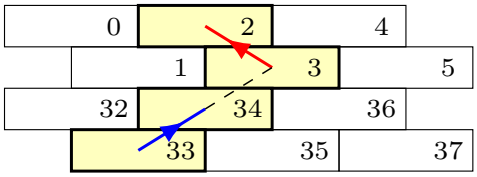
(a) Left-Right-Left (LRL)



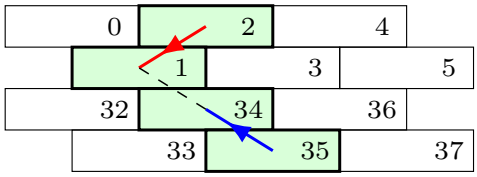
(b) Right-Left-Right (RLR)



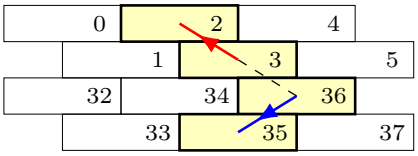
(c) Left-Left-Right (LLR)



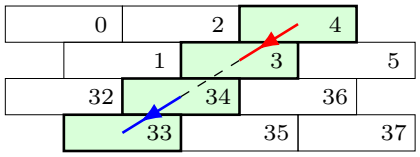
(d) Right-Left-Left (RLL)



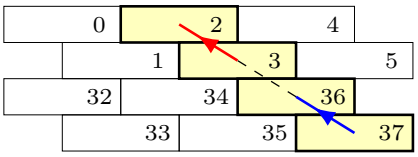
(e) Left-Right-Right (LRR)



(f) Right-Right-Left (RRL)



(g) Left-Left-Left (LLL)



(h) Right-Right-Right (RRR)

Figure 4.20: The four geometry doublets, where pairing is row-wise.

- both warps initially execute the same instructions so both the upper and the lower warp do a shuffle_down_sync. This is requested for the computation of the RLR and LRL geometries.
- referring to Figure 4.20d, the head-cell is cell-2 and summing vertically so one needs to have the $x_{34} + x_{33}$ sum in cell-34 (thread-2 of warp-2).
- having first performed the initial down shuffle, $x_{34} + x_{33}$ is actually present in cell-33. So to access it one needs to perform a shuffle up operation only in warp-2, so that cell-34 will hold the summation and so one can finally sum vertically to obtain the requested $x_2 + x_3 + x_{34} + x_{33}$.

A fitted-track object consists in 8 features as follows: where the χ^2 is as always the reduced one, N is

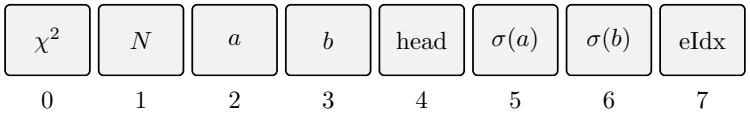


Figure 4.21: Track object with its 8 features. The underlying numbers are the progressive index for each feature.

the number of points in the track, a and b are the fit parameters with their errors $\sigma(a)$ and $\sigma(b)$, head is the head-cell value and eIdx is a local index that is used later by another kernel to assemble the final result. There is also an auxiliary object that saves the geometry and data combination indexes for each track.

The geometry doublets are indexed through *blockIdx.y* index as it was previously introduced. The

blockIdx.y	Geometry Doublets
0	RLR, LRL
1	RLL, LLR
2	RRL, LRR
3	RRR, LLL

Table 4.6: Geometries mapping to blockIdx.y index.

specific mapping rule is specified in Table 4.6.

Track fitting and selection In the following only the 4-points tracks configuration is considered since it is the relevant one for this work. When the combinatorial expander kernel is launched, it creates multiple realizations of the detector, varying the data distribution in the layers (threadIdx.y and blockIdx.x indexes) and the geometrical hypothesis on the tracks (blockIdx.y index). For each realization, and for each of the homogeneous track volumes (within a given realization the tracks all share the same geometry), the linear fits are performed for each head-cell by means of the warp-level operations described in the previous paragraph.

The algorithm uses **float32** arithmetic since it offers superior performances on GPUs with respect to *float64*. It however introduces numerical precision issues that become challenging when dealing with the operations involved in the fits calculations (repeated summations, subtractions, divisions). In order to partially mitigate this problem, compensated accumulation via *Kahan sums* is used as shown in the following snippet:

Algorithm 6 Kahan compensated summation (C-pseudocode)

```
1: function KAHAN_ADD(float x, float& sum, float& comp)
2:     float y = x - comp;
3:     float t = sum + y;
4:     comp = (t - sum) - y;
5:     sum = t;
6: end function
```

Tracks with less than 4-points are suppressed as well as tracks with reduced χ^2 larger than a maximum threshold value (50). After the initial track fitting, selection must be applied. Specifically the kernel executes the operations that are described in the following, implementing what was described in Section 4.2.4:

- **initial track selection within same-geometry tracks**: for instance LRL and RLR tracks are compared whenever they share the same even head-cell. For instance an LRL track with head-cell 1 and an RLR with head-cell 2 share at least cell-2 and so only one of them can survive.
- **check for collisions within same-geometry tracks**: for instance a RLR track with head-cell 2 can collide with a LRL track with head-cell 3. Details on how this collision detection and resolution works are given in the following paragraph.
- a **second selection** is performed with varying threadIdx.y index, comparing same-geometry tracks with different data distributions.
- these operations are performed for each block and so at the end there are 64 blocks for each of the 4 geometry doublets, each with a single realization of the detector with the best surviving tracks. This partial data is then written out to global memory where it will be picked-up by the next kernel.

The next kernel will load this data and compare the tracks at fixed blockIdx.y (keeping the same geometry) and considering the 64 different blockIdx.x values. Finally it will do the last comparison between

the different geometries (varying blockIdx.y values), producing a single final detector realization with the best tracks for that particular input candidate event.

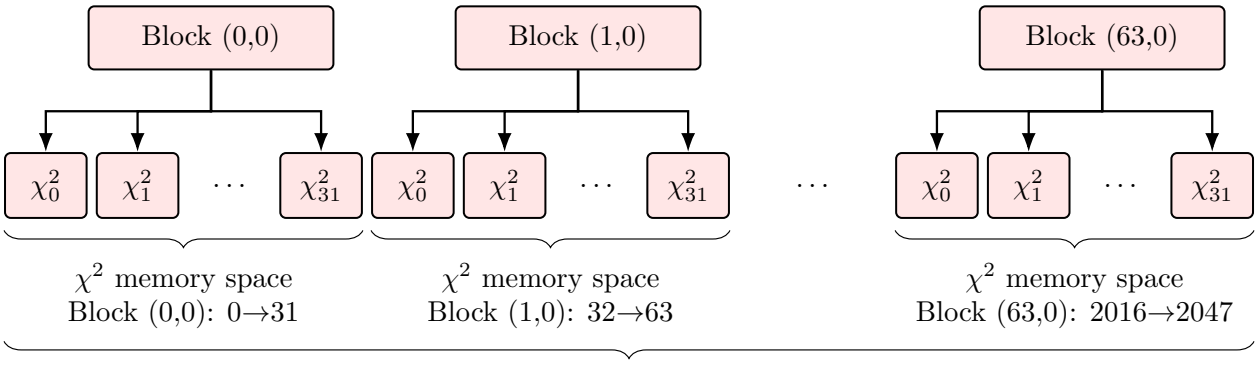


Figure 4.22: Simplified schema of output memory organization. The memory indexes specify a memory location and each location is a fp32 number. Here only the χ^2 is shown and only the 64 blocks for blockIdx.y = 0.

Figure 4.22 shows a partial view of how the output memory is organized for the combinatorial kernel. Only the first 64 blocks for blockIdx.y = 0 are shown and only for the χ^2 feature. The memory indexes $0 \rightarrow 2047$ are used to save the χ^2 for each of the 32 possible head cells of each of the 64 blocks at that given value of blockIdx.y. Since this index ranges from $0 \rightarrow 3$, the χ^2 memory space ranges from $0 \rightarrow 8191$. After that, the same construction is repeated for all the other seven features, so that, schematically one has an output memory organization as shown in Figure 4.23.

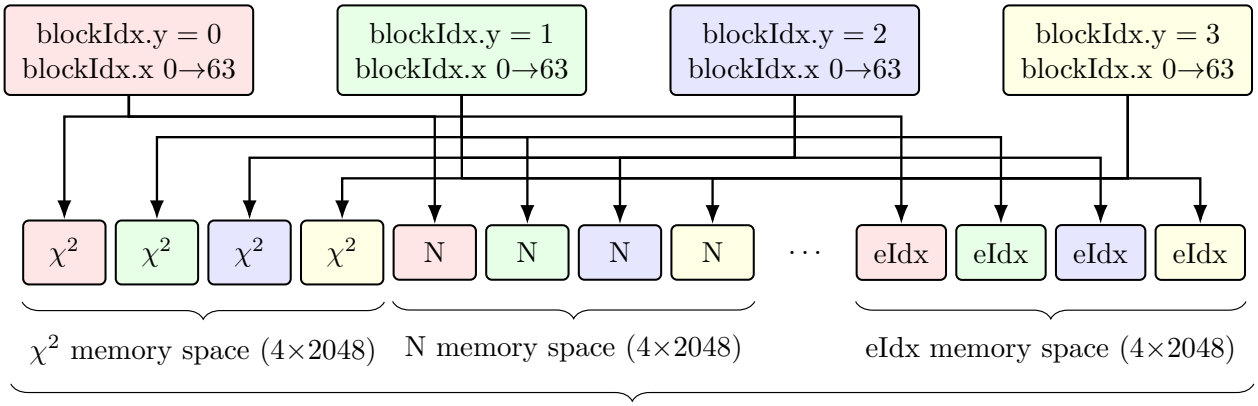


Figure 4.23: Output memory organization for the combinatorial kernel for a single pre-processed input candidate.

The figure shows how the tracks features are written to global memory. The 64 blocks at blockIdx.y=0 write their tracks' χ^2 values, followed by the 64 blocks at blockIdx.y=1 and so forth until blockIdx.y=3, after which the 64 blocks at blockIdx.x=0 write the second feature N and so forth until the last. For each pre-processed input candidate, one has $8 \times 4 \times 2048 = 65536$ memory locations. This data is what is read by the following kernels.

Note on the conflict detection and resolution With respect to the collision detection between tracks of the same geometry, the algorithmic way by which they are compared is again realized by means of shuffles. For the geometry doublets 0, 1 and 2 this is accomplished by means of the shuffle_xor_sync with offset 1 depicted in Figure 4.16b. For the doublet 3 (RRR and LLL) the technique is slightly different but in the end achieves the same goal. The xor sync technique is hereby described.

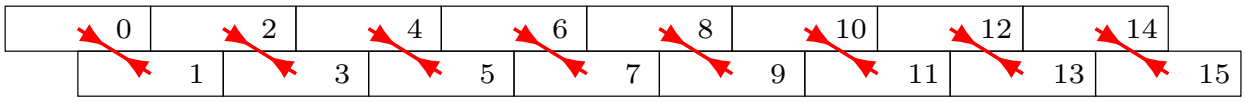


Figure 4.24: Graphical representation of the xor-shuffle operation showing only threads 0 to 15 for clarity.

Figure 4.24 shows the xor-shuffle operation (only half of the upper detector layers is shown for clarity reasons). The data is exchanged between threads 0-1, 2-3, 4-5, ..., 30-31. The interest is only towards the upper warp since these are the threads that hold the track information. When this procedure is applied to same-geometry tracks, one has that, in the case of RLR and LRL for instance, the RLR track with head-cell 0 is compared with the LRL track wit head-cell 1, the RLR of head-cell 2 with the LRL of head-cell 3 and so forth. For each track pair, only one is chosen since they share at least one common cell. So conflict detection and resolution is automatically solved for each geometry doublet.

Track selection and Event builder kernels

As it was briefly mentioned earlier, these kernels now complete the track selection procedure, extending also the conflict detection and resolution to tracks of different geometries. The **track selection** kernel completes the selection that was partially executed in the combinatorial kernel and the **event builder** assembles all the information, producing the final output. The full profiling is available in Appendix B.1.2 and Appendix B.1.3.

Track selection kernel (gridSize $= (1, 1, A)$, blockSize $= (32, 4, 1)$) This completes the track selection procedure. Each of the 32 threads along the x dimension (threadIdx.x) represents a cell of the upper half of the detector while the y dimension (threadIdx.y) this time indexes the geometry doublets (which previously were indexed by blockIdx.y). These 32 cells are the possible head-cells and so each of them, for a given threadIdx.y, loads from memory all of the 64 previous results for that head-cell. Out of these 64 values, each head-cell selects the best candidate. Within the same geometry doublet (same threadIdx.y) the conflict-detection and resolution is performed again as previously described and, at the very end, the surviving track candidates for each head-cell and geometry are compared.

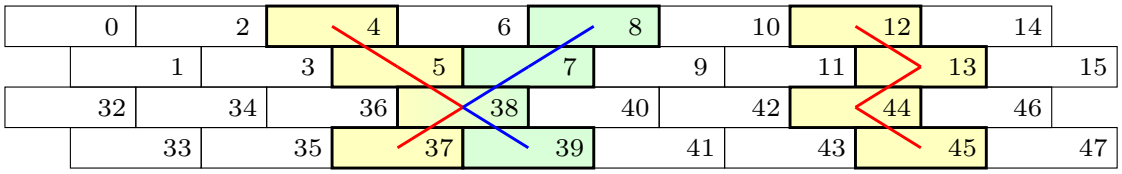


Figure 4.25: Example of a final state with two colliding track volumes (RRL head-cell 4 and LLR head-cell 7) and an isolated track volume (RLR head-cell 12).

This final step requires for conflict-detecting logic since situations as the one exemplified in Figure 4.25 may arise. In this case cell-38 is shared between two tracks of different geometries (RRL and LLR).

To solve this issue, the algorithm uses **cell masks** to detect intersections between neighboring tracks. The idea behind this is that each geometry has a fixed cell topology. So, given a head-cell it is always possible to predict which cells constitute the volume of a given geometry and then to encode this in a binary 64 bit mask (1 bit for each of the 64 cells) where the occupied cells are marked with ones and the empty with zeros. Considering for instance the RLR track volume shown in Figure 4.25 with head-cell 12. The topology of the RLR volume is: $+1, -1 + 32, +1 + 32$. Starting from 12 then the volume consists in:

$$V_{\text{RLR},12} = [12, 13, 44, 45]$$

and so the cell mask is obtained by (1ULL is the unsigned integer 1 in 64 bit):

$$\text{cellMask}_{12} = (\text{1ULL} \ll 12) \mid (\text{1ULL} \ll 13) \mid (\text{1ULL} \ll 44) \mid (\text{1ULL} \ll 45)$$

which yields the mask with the ones in positions 12, 13, 44, 45 starting from the right:

$$\text{cellMask}_{12} = 00000000000000000011000000000000000000000000000011000000000000$$

Repeating this construction for the other two track volumes one has:

$$\text{cellMask}_4 = 00000000000000000000000011000000000000000000000000000000110000$$
$$\text{cellMask}_7 = 00000000000000000000000110000000000000000000000000000110000000$$

By means of a simple bit-wise logical and, one can detect the intersections. For instance:

$$\begin{aligned} \text{cellMask}_{12} \,\&\, \text{cellMask}_4 &= 0ULL, && \text{no intersection between 12 and 4} \\ \text{cellMask}_{12} \,\&\, \text{cellMask}_7 &= 0ULL, && \text{no intersection between 12 and 7} \\ \text{cellMask}_4 \,\&\, \text{cellMask}_7 &\neq 0ULL, && \text{intersections between 4 and 7} \end{aligned}$$

At this point, each head-cell that has a track can look at the cell masks of other threads both forward and backward in the warp, using broadcast shuffles (Figure 4.16a). Each thread then reads the cell masks of the others within a maximum radius and checks for a non-null intersection with their own mask. If there is an intersection, each thread decides, by looking at the other conflicting threads' tracks, if its own candidate is better. If it is it maintains the track, otherwise it suppresses it. At the end, only one track will remain for each intersecting group.

This kernel then writes to memory a final detector realization with the surviving tracks. Since there are up to 32 head-cells, the output memory is organized so that the first 32 indexes are occupied by the χ^2, the second 32 by the number of points N, and so forth until the last of the eight features as before. This time the output memory will be a 8×32 array of fp32 numbers for each of the pre-processed input candidates.

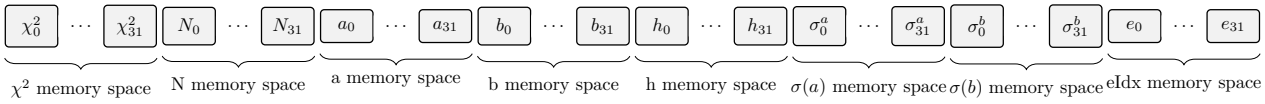


Figure 4.26: Output memory organization for the track selection kernel for a single pre-processed input candidate. Each feature has numerosity 32.

Event builder kernel (gridSize $= (1, 1, A)$, blockSize $= (64, 1, 1)$) This kernel reads back from global memory the final output of the track selection kernel and it assembles the final events by piercing together the auxiliary information. Specifically this kernel:

- Takes the auxiliary hit-words information from the input shaper kernel as shown in Figure 4.12 and extracts the drift times of each hit position that was used to compute the particular track.
- It computes the θ angle of the track as:

  $$\theta = \arctan b$$

  where b is the slope that was computed before in the fit. As it was previously pointed out, the angle is computed with respect to the perpendicular with respect to the detector plane, that is the z-axis. This is shown in Figure 4.27.
- It writes to global memory eight output features for each pre-processed input candidate as detailed in the following.

The output is organized in a similar way as Figure 4.26 but this time the features are ordered as per Table 4.7. The highlighted rows are the ones that are used by the Data Quality Monitoring (DQM) block of the pipeline as described in the appropriate section.

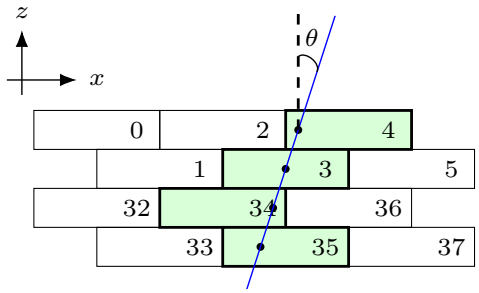


Figure 4.27: Track angle θ definition. The blue line represents a fitted track belonging to the LLR geometrical hypothesis.

Index	Feature	Note
0	combIdx	the data combination index that was used to compute the track
1	t_drift_4	the drift time of the hit used at layer 4 to compute the track
2	t_drift_3	the drift time of the hit used at layer 3 to compute the track
3	t_drift_2	the drift time of the hit used at layer 2 to compute the track
4	t_drift_1	the drift time of the hit used at layer 1 to compute the track
5	θ	the track angle
6	geomIdx	the geometry index of the track
7	χ^2	the reduced χ^2 of the track

Table 4.7: Final output features.

Output shaper kernel (gridSize $= (64, 4, A)$, blockSize $= (64, 4 \vee 8, 1)$)

This kernel simply performs a transposition in order to obtain an output that is more suitable for the final DQM block of the pipeline. Since the DQM expects a PyTorch tensor as input, this transposition is used to obtain an Array-of-Structures that is C-contiguous in the jargon of PyTorch. This means that the output of the GPU kernels that is a CuPy array can be re-cast as a PyTorch tensor with a zero-copy operation. Thus the final output has the same features as shown in Table 4.7 but they are now saved to memory as array of structures, where each structure has the eight features.

4.2.6 Performance Analysis

The performance of the chain is evaluated together with that of the pre-processor block since both blocks always operate together and the track reconstruction kernels depend on the pre-processor block output. The two blocks have been joined in a partial, two-block pipeline. The main performance metric is again the throughput and in order to measure it, the pipeline is defined as **synchronous**. This means that a single unit of work of the pipeline corresponds to having *an exact correspondence between the input data batch and the reconstructed track output.*

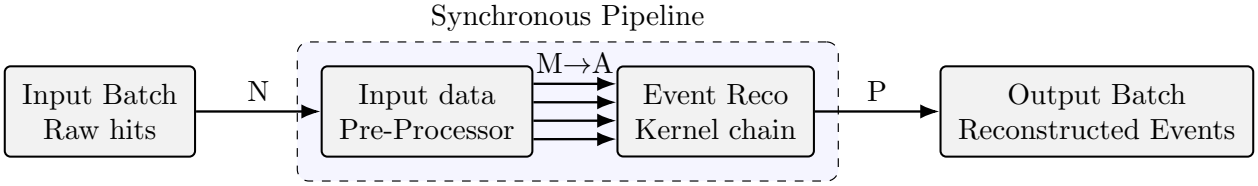


Figure 4.28: Schematic of the partial synchronous pipeline with the pre-processor and event reconstruction kernel chain.

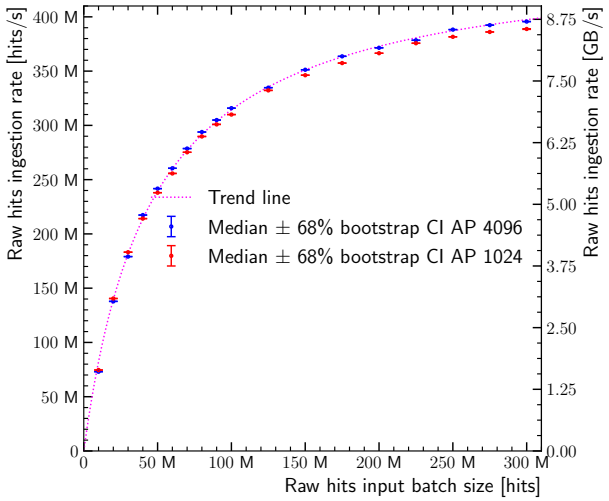
A schematic of the pipeline is shown in Figure 4.28. When an input batch with N raw hits is presented at the input, the pre-processor produces M candidates which are fed to the kernel chain in sub-batches of A candidates each to which the kernels correspond an output which is accumulated over the sub-

batches in a final output of size P. So in the end one has:

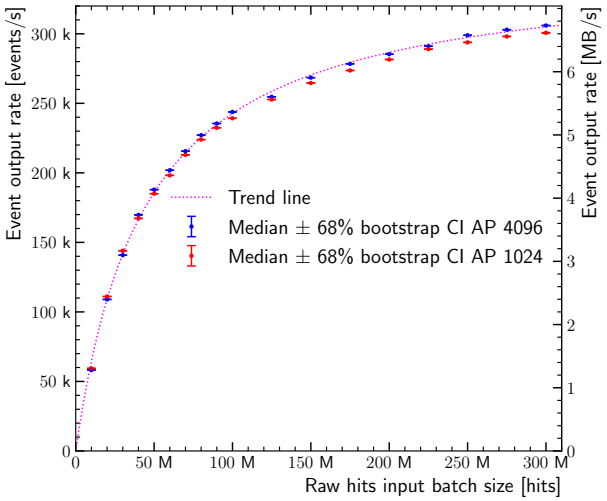
$$\mathrm{P}_{\mathrm{events}} = F(\mathrm{N}_{\mathrm{hits}}), \qquad F : \mathbb{R}^{6N} \longrightarrow \mathbb{R}^{8P}$$

where F represents the action of the pipeline.

Measurement setup The measurement setup is the same as in the case of the pre-processor. Execution time is measured by means of CUDA events and synchronization, obtaining the time interval between the start of the pre-processing and the instant at which the kernel chain finishes to process it. In order to smooth out the large time fluctuations that are induced by GPU initialization, measurements for a given input batch are repeated 100 times, each time measuring 10 repetitions and averaging out. As previously stated, this pipeline is using the RMM allocator so that cuDF (pre-processor) and CuPy (preprocessor and kernels) can share the same memory pool.



(a) Input throughput of the pre-processor and track-reconstruction blocks, measured in hits/s.



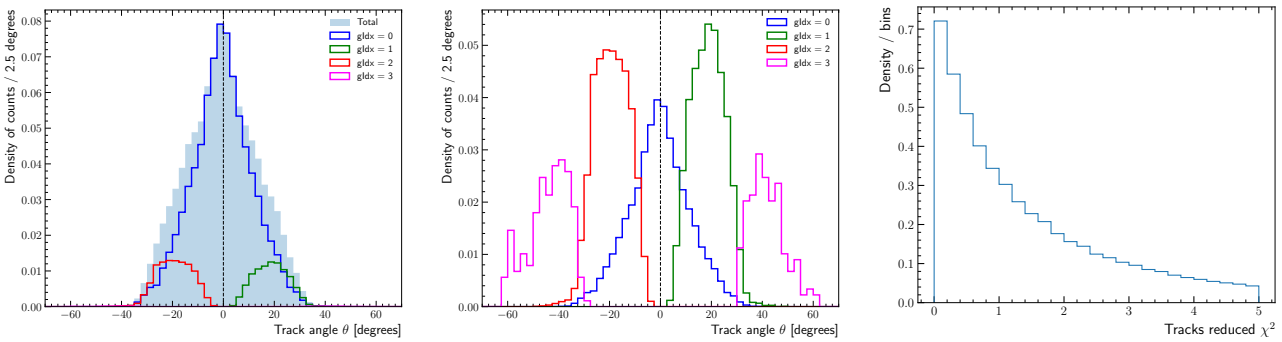
(b) Output throughput of the pre-processor and track-reconstruction blocks, measured in events/s.

Figure 4.29: Input and output throughput measurements for the pre-processor and kernel chain blocks as a function of the input batch size. The measurements are shown with two different gpu apertures: 4096 and 1024 events per kernels launch.

The results for the input and output throughputs are shown in Figure 4.29. At the left in Figure 4.29a the input throughput as a function of input batch size is shown. This is the rate with which the raw hits are ingested by the pipeline. Conversely, at the right in Figure 4.29b one has the output throughput as function of input batch size. This is the rate at which the pipeline outputs reconstructed events. Following the same considerations that were earlier put forth for the pre-processor, also this time the median and the bootstrap CI have been used to estimate the central value and its confidence interval. A similar slowdowns study to the one of the pre-processor has been conducted for this partial pipeline too and is briefly presented in Appendix A.1 (Figure A.1a and Figure A.1b).

Angular distribution of the tracks and Geometry doublets angular covering The angle distribution of the reconstructed tracks is shown in Figure 4.30a and in Figure 4.30b while Figure 4.30c shows the reduced χ^2 distribution.

In Figures 4.30a and 4.30b, the different contributions to the total distribution by the four paired geometrical hypotheses is shown. In Figure 4.30b the shapes of the contributions are shown in greater detail. The gIdx = 0 doublet (RLR and LRL geometries), as expected, covers the most vertical tracks and as the track angle become larger and larger, the gIdx = 1 and gIdx = 2 doublets (RLL,LLR and RRL,LRR respectively) produce better candidates. Finally the gIdx = 3 (RRR and LLL) covers the outer angular region which is however scarcely populated as the plot shows. The magnitude of each contribution is of course a property of the dataset, since it depends on the experimental conditions.



(a) Common normalization. (b) Individual normalization. (c) Reduced χ^2 distribution.

Figure 4.30: The angular distribution of the reconstructed tracks. In Figure 4.30a the total distribution is represented by the shaded area while the lines represent the contributions from each geometry doublet's angular distributions. The distributions are commonly normalized. In Figure 4.30b shows the angular distributions for each geometry doublet with individual normalization to show the shape of each contribution, highlighting where each geometrical hypothesis produces the most tracks and also the overlaps between them. In Figure 4.30c the reduced χ^2 distribution for the tracks is shown.

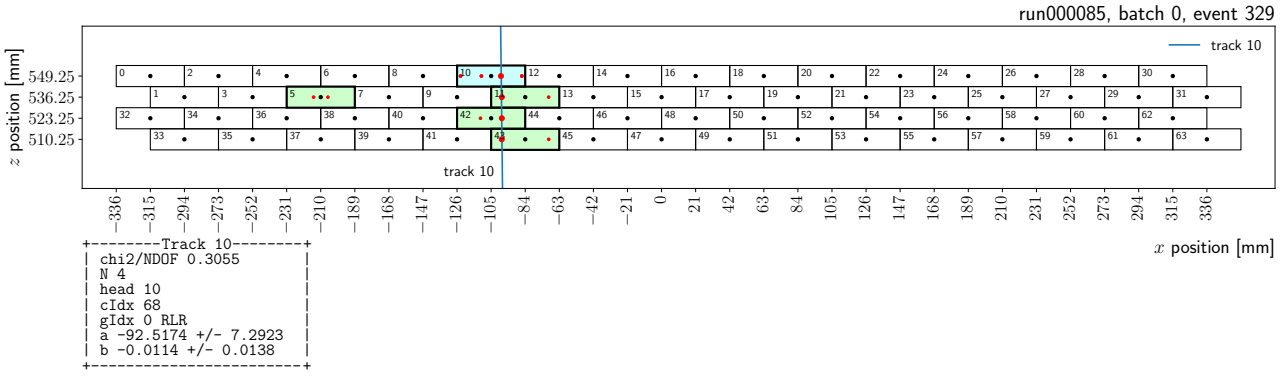


Figure 4.31: Event with a single track and 6 total recorded hit-words.

Figure 4.31 depicts a single track fit with an isolated hit-word recorded in cell 5. Because of the way in which the track reconstruction algorithm works, isolated cells have no impact on it. This is an advantage over more conventional approaches that partition the detector volume into rigid windows, allowing also potentially spurious information coming from isolated hits to affect the track fitting and selection procedures. The cyan-filled detector cells are those where two hit-words have been recorded.

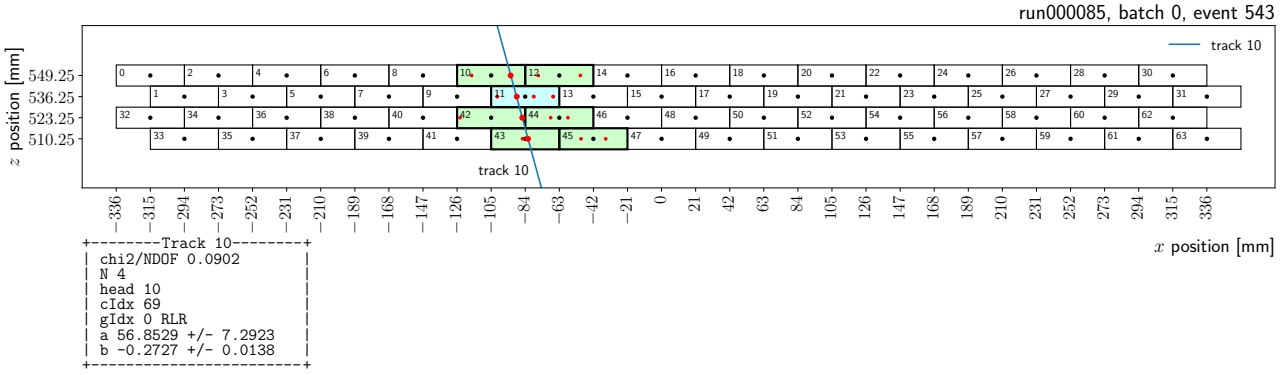


Figure 4.32: Event with a single track and 8 total recorded hit-words.

Figure 4.32 shows another event with a single track but with three cells that are not participating in the fit. This is another situation in which the reconstruction algorithm showcases its versatility. For instance one can spot that cells 12, 11, 44 and 45 (LRR geometry) also show nicely aligned points if

one keeps the L,R,R and L hits respectively. These four points would indeed produce a 4-points track with a reduced χ^2 of 1.05. This track has been pruned not only because the χ^2 is slightly larger than the one that track-10 has, but because the two tracks share a common cell that is cell 11. This is one of the cases where the conflict resolution procedure has suppressed a track letting the one with the better quality survive.

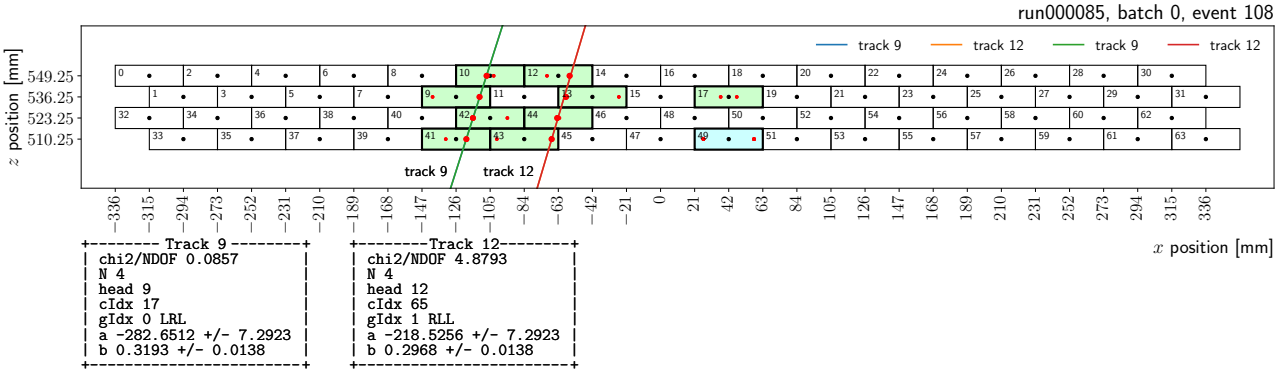


Figure 4.33: Event with two parallel tracks and 11 total recorded hit-words.

Figure 4.33 shows an example of a reconstructed event with two parallel tracks. In this case a LRL track (head cell 9) and a RLL track (head cell 12) have been found. This is a nice example of the interplay between the different geometrical hypotheses: even if the tracks have nearly the same angle, given how the hits points are distributed, one has been captured by the LRL geometry and the other by the RLL.

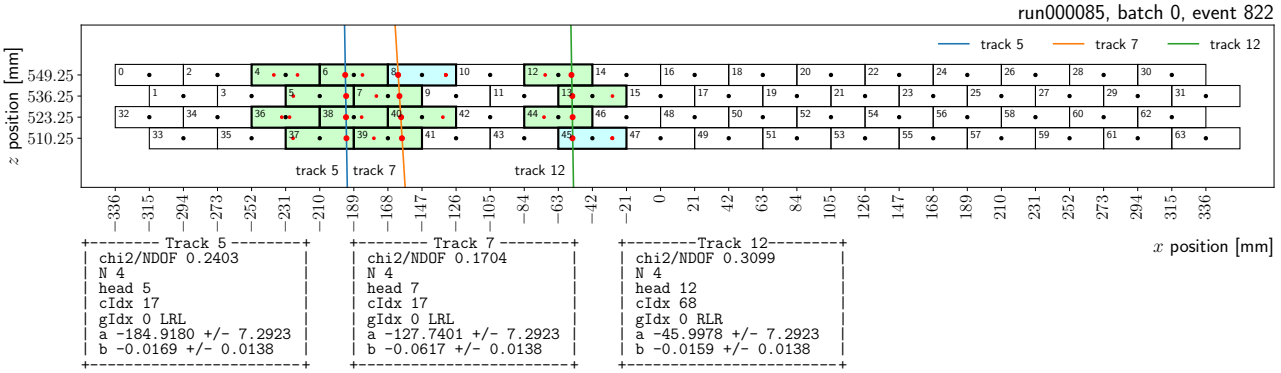


Figure 4.34: Example of a reconstructed event with three almost parallel tracks and 16 total recorded hit-words.

Another more complex example is shown in the following Figure 4.34. In this specific case, the event carried 3 tracks all of the same geometry doublet (RLR and LRL) and there are also additional cells (4 and 36) that do not participate in the fits of any of the reported tracks.

Finally Figure 4.35 showcases an event with 4 tracks, two of which are 3-points one and many cells contain two hit-words. One of these tracks (track 17) has a large reduced χ^2 and so it has been later pruned. Many other examples are shown in Appendix A.2.

Comparison between GPU kernels approach and a simple CPU algorithm The simple CPU algorithm solves the combinatorial using a sequential approach. For each candidate event, an algorithm tries all the possible data combinations and chooses the best track or tracks depending on the situation. For this comparison, the following simplifying assumptions were made: only events with at most one active cell per layer were considered and only events with exactly four total active cells. This is the simplest event topology and $\approx 58\%$ of the dataset consists of events of this kind. Then, both the simple Python fitter and the GPU kernels chain have been used to do the reconstruction.

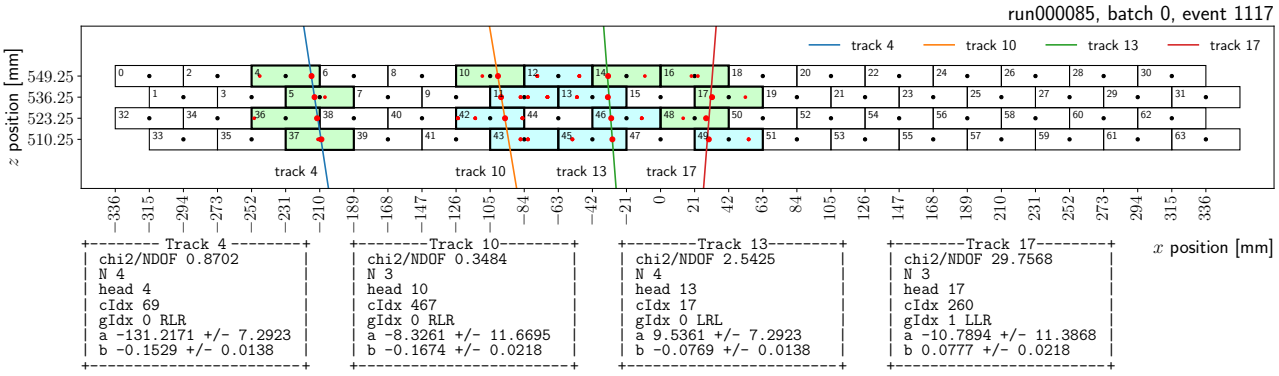
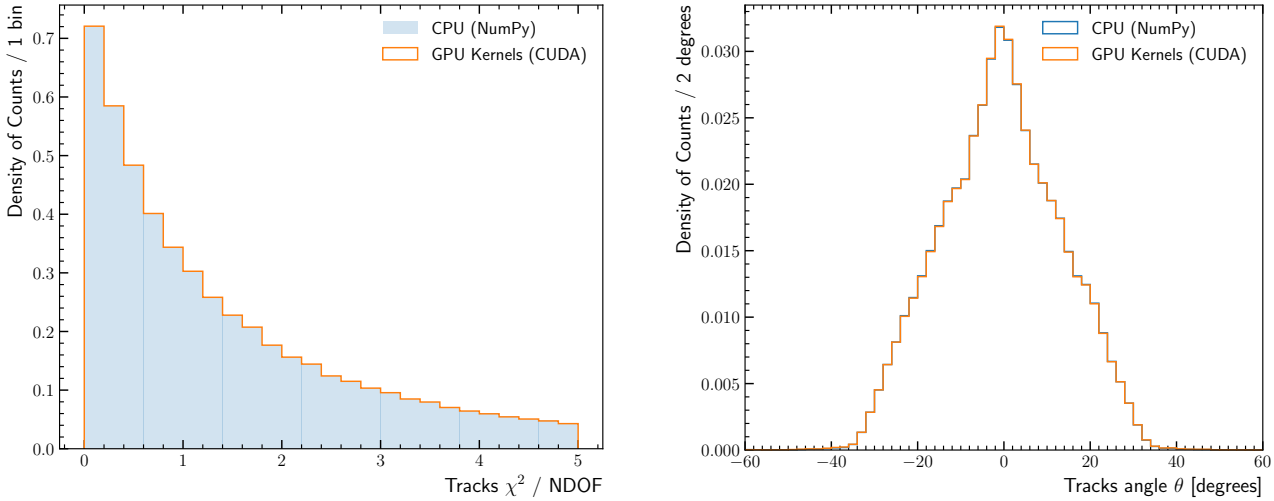


Figure 4.35: Example of a reconstructed event with four tracks and 25 total recorded hit-words.

When considering the time required to fit each track / reconstruct each event the scores are as follows:

$$t_{\text{CPU}} = (3444.50 \pm 30.14)\ \mu s, \qquad t_{\text{GPU}} = (2.10^{+0.04}_{-0.02})\ \mu s \tag{4.10}$$

where the measurements have been taken over 100 repetitions and for the GPU case the measurements were performed considering the median and the 16th and 84th percentiles. One can thus see how the GPU approach is faster by a factor of ≈ 1600. Also, this comparison has been done with the simplest of the event topology and while the GPU kernels do not care about event complexity (the required time is exactly the same whether the candidate event has 4 or 64 hit-words), the CPU approach instead does. The reason why t_{GPU} does not depend on the event complexity is due to its architecture and the choice of keeping execution divergence at a minimum.



(a) CPU vs GPU χ^2 distributions

(b) CPU vs GPU angle distributions

Figure 4.36: Comparison between the CPU and GPU reconstruction outcomes. At the left the reduced χ^2 distributions are reported while at the right the angular distribution of the θ angle of the tracks is shown.

As a further comparison, the angular and χ^2 distributions obtained with the CPU algorithm are plotted together with the ones obtained via the GPU approach. These are shown in Figure 4.36. The agreement is quantified by means of several quantities:

- The two distributions contain different number of samples, so they were normalized yielding:

$$p_i^{\text{CPU}} = \frac{N_i^{\text{CPU}}}{N^{\text{CPU}}}, \qquad p_i^{\text{GPU}} = \frac{N_i^{\text{GPU}}}{N^{\text{GPU}}}$$

- For each normalized histogram, the following quantity was computed:

$$F_k = \sum_{i \leq k} p_i$$

which resembles a cumulative distribution function and from this the following quantity was computed:

$$D = \max_k \left| F_k^{\text{CPU}} - F_k^{\text{GPU}} \right|$$

This is called the **CDF maximum difference**.

- The first Wasserstein distance was also computed. This measures how far probability mass would have to be moved along the x-axis to transform one distribution into the other:

$$W_1(P, Q) = \int |F_P(x) - F_Q(x)| \, dx$$

and the discrete version is already implemented in the `scipy.stats` module in Python.

- The total variation distance defined as:

$$D_{\text{TV}} = \frac{1}{2} \sum_i \left| p_i^{\text{CPU}} - p_i^{\text{GPU}} \right|$$

It ranges from 0 to 1 and it measures the fraction of probability mass that would need to be reassigned between bins to make the histograms match.

The results are shown in the following Table 4.8.

Quantity	Value (45 bins)	Value (60 bins)	Value (120 bins)
D	4.78×10^{-4}	4.85×10^{-4}	4.89×10^{-4}
W_1	0.0240	0.0244	0.0243
D_{TV}	1.48×10^{-3}	1.47×10^{-3}	1.53×10^{-3}

Table 4.8: Results of the agreement indicators between the CPU and GPU algorithms outputs. The number of CPU samples is $N_{\text{CPU}} = 190544$ while the number of GPU samples is $N_{\text{GPU}} = 190054$.

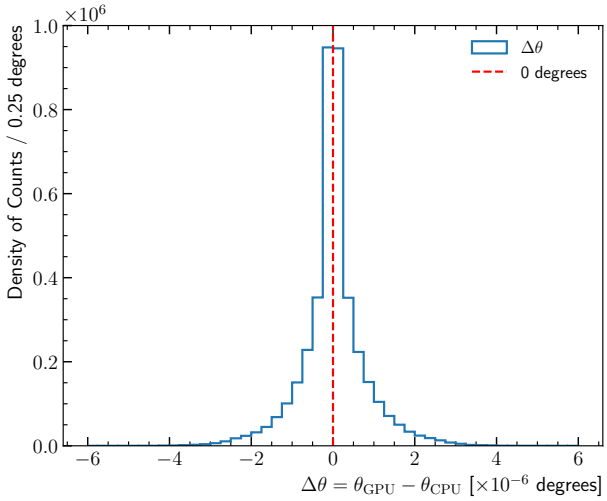
The CPU and GPU implementations were evaluated on the same input data using identical track-selection criteria. Their normalized output distributions exhibit very close agreement, with a maximum cumulative difference of 4.85×10^{-4}, a Wasserstein distance $W_1 = 0.0244$ deg, a total variation distance $D_{\text{TV}} = 1.47 \times 10^{-3}$. These values indicate sub-per-mille to per-mille-level differences in the distribution shapes. The comparison was repeated using several histogram binnings, including 45, 60, and 120 bins, yielding consistent results and demonstrating that the observed agreement is robust against the choice of binning.

The results demonstrate excellent distribution-level agreement between the GPU and CPU implementations, and the added value of the GPU implementation that is $\approx 10^3$ times faster and able to deal effortlessly with more topologically complex events. Another comparison that again demonstrates the agreement between the CPU and GPU outputs that instead uses statistical methods (Falkon as it will be described later) is given in Appendix A.3.

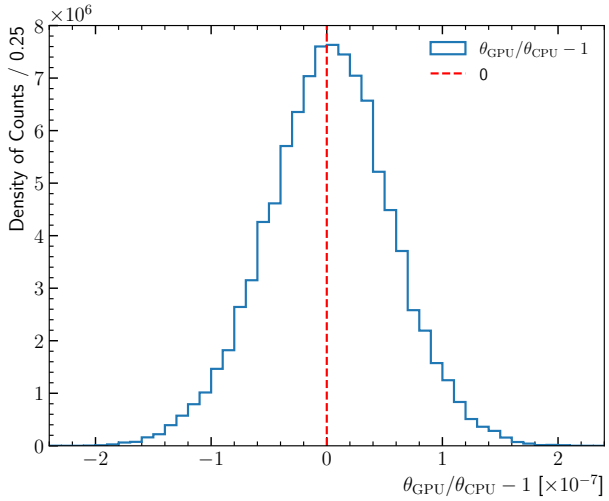
Event-per-Event comparison To further explore the agreement or disagreement between the reconstruction outputs of the GPU and CPU algorithms, an event-per-event test was conducted. The same events that were employed in the previous analysis have been also used here. In this case, each track has been uniquely indexed so that the outputs of the two algorithms could be joined and the event-per-event comparison could be executed.

Figure 4.37 shows the results of the comparison for track angles of up to $\pm 60°$. In Figure 4.37a and Figure 4.37b the angular differences $\Delta\theta = \theta_{\text{GPU}} - \theta_{\text{CPU}}$ and angular ratios (offset by 1) $\theta_{\text{GPU}}/\theta_{\text{CPU}} - 1$ of matching tracks are displayed. One can see that the distributions are centered around 0 and are mostly symmetrical, showing widths in the order of 10^{-6} degrees for the angular and for the χ^2 differences.

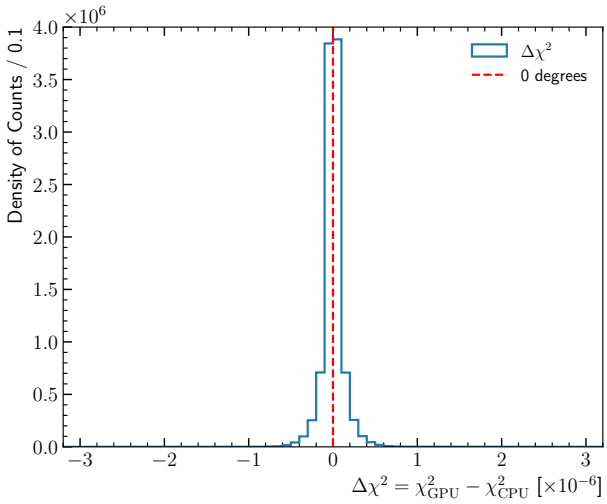
The numerical results are shown in Table 4.9. As a final note, the possible reason for the discrepancy between N_{CPU} and N_{GPU}, even if it is small with respect to the total number of tracks ($\approx 2.6 \times 10^{-3}$)



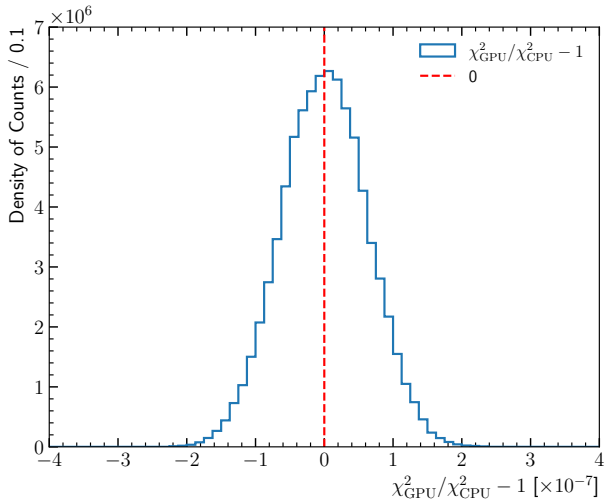
(a) Track angle differences between matching tracks reconstructed via the GPU algorithm and via the NPY script.



(b) Track angle ratios between matching tracks reconstructed via the GPU algorithm and via the NPY script.



(c) Track χ^2 differences between matching tracks reconstructed via the GPU algorithm and via the NPY script.



(d) Track χ^2 ratios between matching tracks reconstructed via the GPU algorithm and via the NPY script.

Figure 4.37: Event-per-event comparison between the GPU and CPU track reconstruction algorithms for track angles of up to ±60°, showing both track angle and track χ^2 related quantities.

Angular window	Median $\Delta\theta$	$q_{84} - q_{16}$ $\Delta\theta$	Median $\Delta\chi^2$	$q_{84} - q_{16}$ $\Delta\chi^2$
±5°	-3.01×10^{-11}	1.86×10^{-7}	7.16×10^{-11}	1.02×10^{-7}
±10°	-2.705×10^{-10}	4.18×10^{-7}	6.71×10^{-11}	1.09×10^{-7}
±20°	-3.23×10^{-11}	8.55×10^{-7}	7.05×10^{-11}	1.27×10^{-7}
±30°	1.29×10^{-10}	1.08×10^{-6}	5.77×10^{-11}	1.36×10^{-7}
±40°	2.64×10^{-10}	$1.11e \times 10^{-6}$	5.50×10^{-11}	1.37×10^{-7}
±50°	2.69×10^{-10}	1.12×10^{-6}	5.5×10^{-11}	1.37×10^{-7}
±60°	2.70×10^{-10}	1.12×10^{-6}	5.41×10^{-11}	1.37×10^{-7}

Table 4.9: Event-per-event comparison between the GPU and NumPy reconstruction implementations for progressively wider angular windows. Differences are computed as GPU minus NumPy. The quantity $q_{84} - q_{16}$ gives the width of the central 68% interval of the difference distribution.

could point to inefficiencies in either of the two approaches. Further investigations on this are left for future work.

4.3 Anomaly detection and DQM block

This is the final block of the pipeline that takes as input the dataset produced by the previous track reconstruction stage. The features are the highlighted ones reported in Table 4.7. The core of this block is constituted by the `FalkonML` framework, which is a machine-learning framework implementing scalable kernel methods using GPU-accelerated training. The final objective is to produce, for each input batch, a **single scalar value**, that is the **p-value** associated to the computed *test statistic*, given the properly calibrated reference distribution.

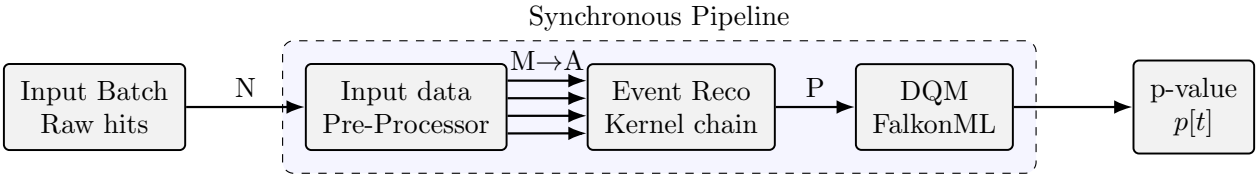


Figure 4.38: Complete synchronous pipeline, showing all the stages from the raw input batch to the final test statistic. DQM is the acronym for Data Quality Monitoring.

This is shown in Figure 4.38. If one represents the pipeline's collective action as a function F, then:

$$F : \mathbb{Z}_{+}^{6N} \longrightarrow \mathbb{R}$$
$$B \longmapsto p[t] = F(B)$$

where, referring also to Figure 4.38, N is the number of raw hits in the input batch B. So, to each input batch there corresponds a single scalar value. This is the p-value associated to the test statistic that was computed in the comparison between the reference data and the incoming data batch. The underlying idea is that, as described in Chapter 2, the p-value signals a departure from the reference conditions, quantifying it. In this way the algorithm can flag the input batch as anomalous whenever the deviation becomes large enough (thresholds). Also, the value itself of the computed test statistic can also be used to tentatively infer the origin of the anomaly as it will be described in the following.

4.3.1 FalkonML implementation

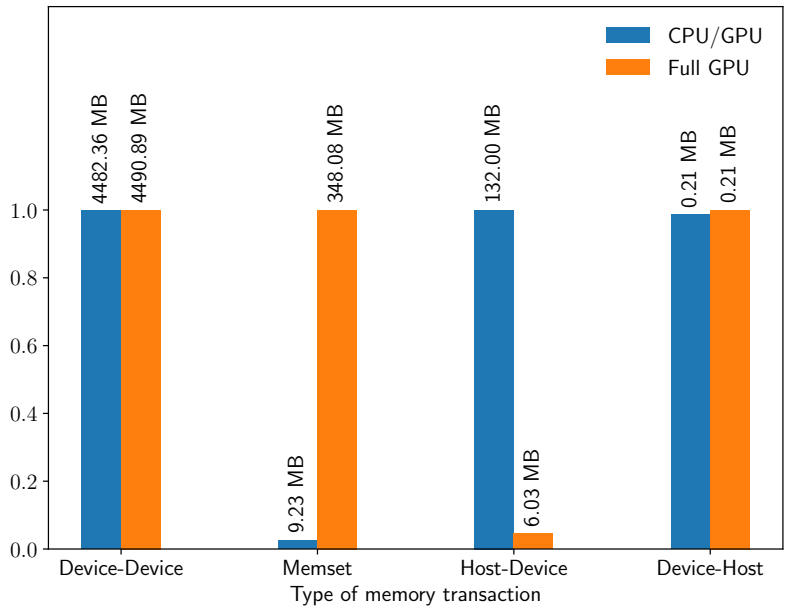
FalkonML is built on top of `PyTorch`, the popular machine-learning framework. In order to fully integrate this block in the pipeline, the same memory management paradigm that was earlier introduced for the two preceding blocks must also be used here. This introduces some major issues since on the one hand PyTorch supports external memory allocators offering an interface that is much more limited with respect to its native memory manager. On the other hand, FalkonML relies on PyTorch's native memory manager interface to perform many of its internal functions. For this reason, it was necessary to write a compatibility layer between FalkonML and PyTorch so that FalkonML's calls to PyTorch's allocator interface are properly intercepted when the external RMM allocator is in use.

FalkonML and PyTorch compatibility layer

While the modifications to FalkonML are not extensive, they are scattered across multiple source files. For this reason the full list and details of modifications is provided in the Appendix B.2. In this section the modifications are briefly described in words as follows:

- The core of the compatibility layer consists in three functions that are called within the function that FalkonML uses to get information on the gpu device in use on the system. When the external allocator (RMM) is in use, these functions furnish Falkon the data that it would get from PyTorch's internal memory manager. Conversely, when no external allocator is in use, the functions returns the internal manager's call transparently.
- In other parts of the source code, GPU data residency is enforced by explicitly setting the GPU device as the default one to hold the tensors. While this behaviour can also be triggered by setting a flag in FalkonML itself, it has been found that it is sometimes ineffective.

With these modifications, the whole pipeline acts like a coherent entity with respect to memory management. It is in fact important to keep in mind that the three blocks are natively heterogeneous in that respect.



CPU/GPU

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
4,482.358	748,930	0.006	0.000	0.000	8.000	0.207	[memcpy Device-to-Device]
9.228	5,000	0.002	0.002	0.000	0.008	0.002	[memset]
132.004	2,019	0.065	0.000	0.000	0.220	0.091	[memcpy Host-to-Device]
0.207	114,362	0.000	0.000	0.000	0.000	0.000	[memcpy Device-to-Host]

Full GPU

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
4,490.893	760,186	0.006	0.000	0.000	8.000	0.205	[memcpy Device-to-Device]
348.084	9,000	0.039	0.002	0.000	0.220	0.060	[memset]
6.032	1,020	0.006	0.000	0.000	6.028	0.189	[memcpy Host-to-Device]
0.210	115,982	0.000	0.000	0.000	0.000	0.000	[memcpy Device-to-Host]

Figure 4.39: Variation in the memory usage when switching from CPU/GPU FalkonML to fully resident GPU version using FalkonML as stand-alone module acting on a reduced input dataset.

Figure 4.39 shows what happens when one switches from the typical FalkonML implementation to the modified fully GPU resident one. The important variation consists in the fact that the Host-to-Device memory copies are drastically reduced, signalling how the data now is not anymore moved back and forth between the system and the device memories.

Input data format and on the nature of the data

Since PyTorch's native data format is that of PyTorch tensors while the output of the track reconstruction stage is a CuPy matrix, it is mandatory to execute a transformation. Thanks to the unified memory approach and the fact that, as it was explained earlier, the output matrix of the kernel chain is C-contiguous, the operation becomes very simple and happens without triggering a memory copy:

```
torch_tensor = torch.from_dlpack(kernel_output_cupy)
```

The data that is passed to FalkonML consists in 5 features: the four drift times and the track angle. While the angular distribution has already been shown before, it is now interesting to see how the drift time distributions look for each of the four detector layers and how they compare to the ones that have been shown at the pre-processing stage in Figure 4.2 and also recalled here below.

Looking at Figure 4.40 one immediately sees how the shape of the timebox for the inner layers 3 and 2 has changed with respect to the initial timebox distributions for the raw hits that presents instead

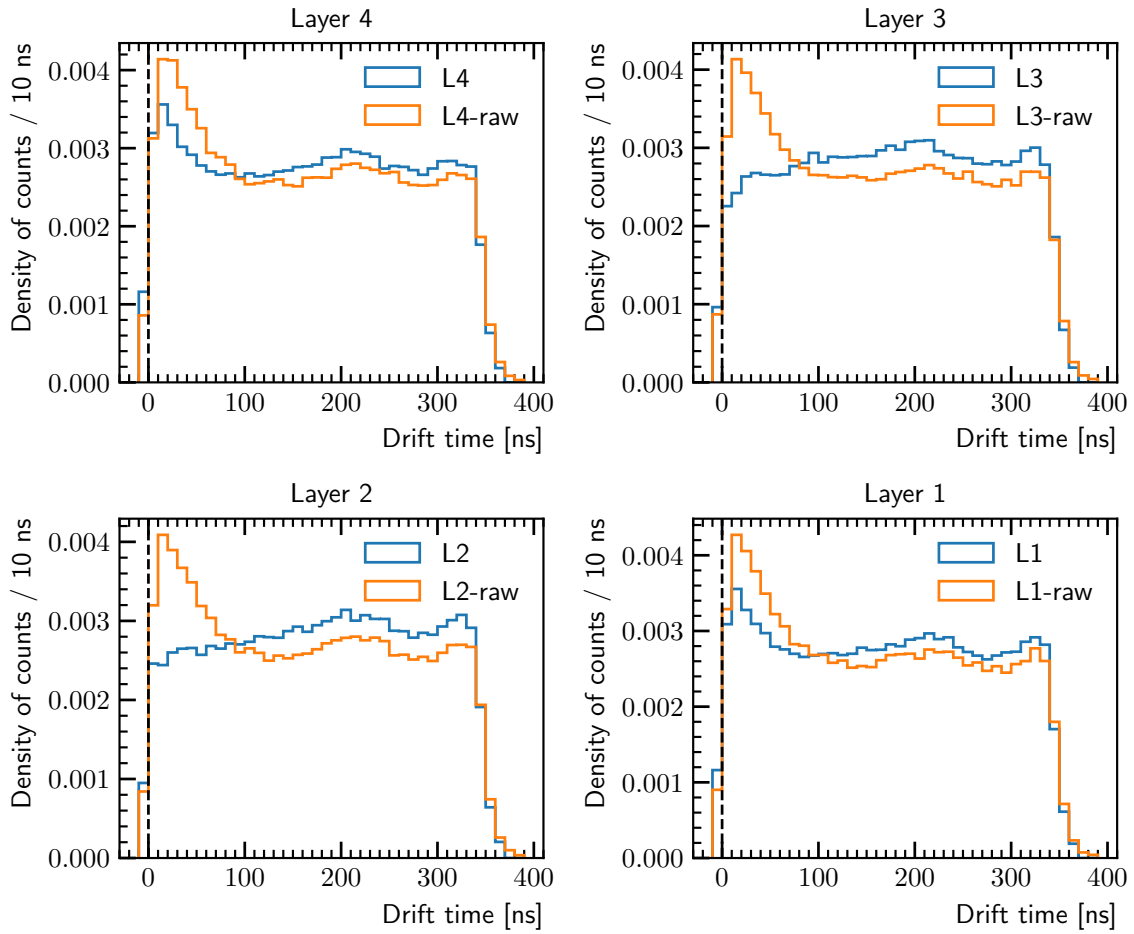


Figure 4.40: Drift times distributions for the four layers.

the sharp leading edge, the fall to the plateau region and the final drop towards zero at the right edge. The reason for this is that now the timebox is a **different observable**.

This behavior can be understood by noting that the drift times associated with the hits of a reconstructed track are not independent quantities. Since the four hit points originate from the same particle trajectory, their positions within the corresponding drift cells are geometrically correlated. Owing to the staggered arrangement of the sense wires, the distance of the track from the wire in one layer constrains the distances measured in the neighboring layers. As a consequence, selecting hits that are compatible with a reconstructed track introduces non-trivial correlations among their drift times, which can modify the shape of the corresponding one-dimensional timebox distributions.

In a way, if the earlier timeboxes obtained on the raw input data represented $P(t_i)$ for the i-th layer, now after the track recontruction stage the timeboxes represent $P(t_i|T)$ where T is the track constraint that is imposed. In fact, if it where that $P(t_i|T) = P(t_i)$, that is the distribution is independent of the track constraint, it would mean that the hit points associated to the track really have no relation to it. So, track reconstruction effectively reweights the input timeboxes.

To investigate these correlations, the drift time measured in each layer is studied as a function of the reconstructed track angle θ.

The resulting plots are shown in Figure 4.41. These plots represent the joint-distributions $P(t_i, \theta|T)$ and the one 1-dimensional timeboxes shown Figure 4.40 are just the integral over θ:

$$P(t_i|T) = \int P(t_i, \theta|T) d\theta$$

It is possible to spot immediately the diamond-shaped regions which represent the allowed regions in

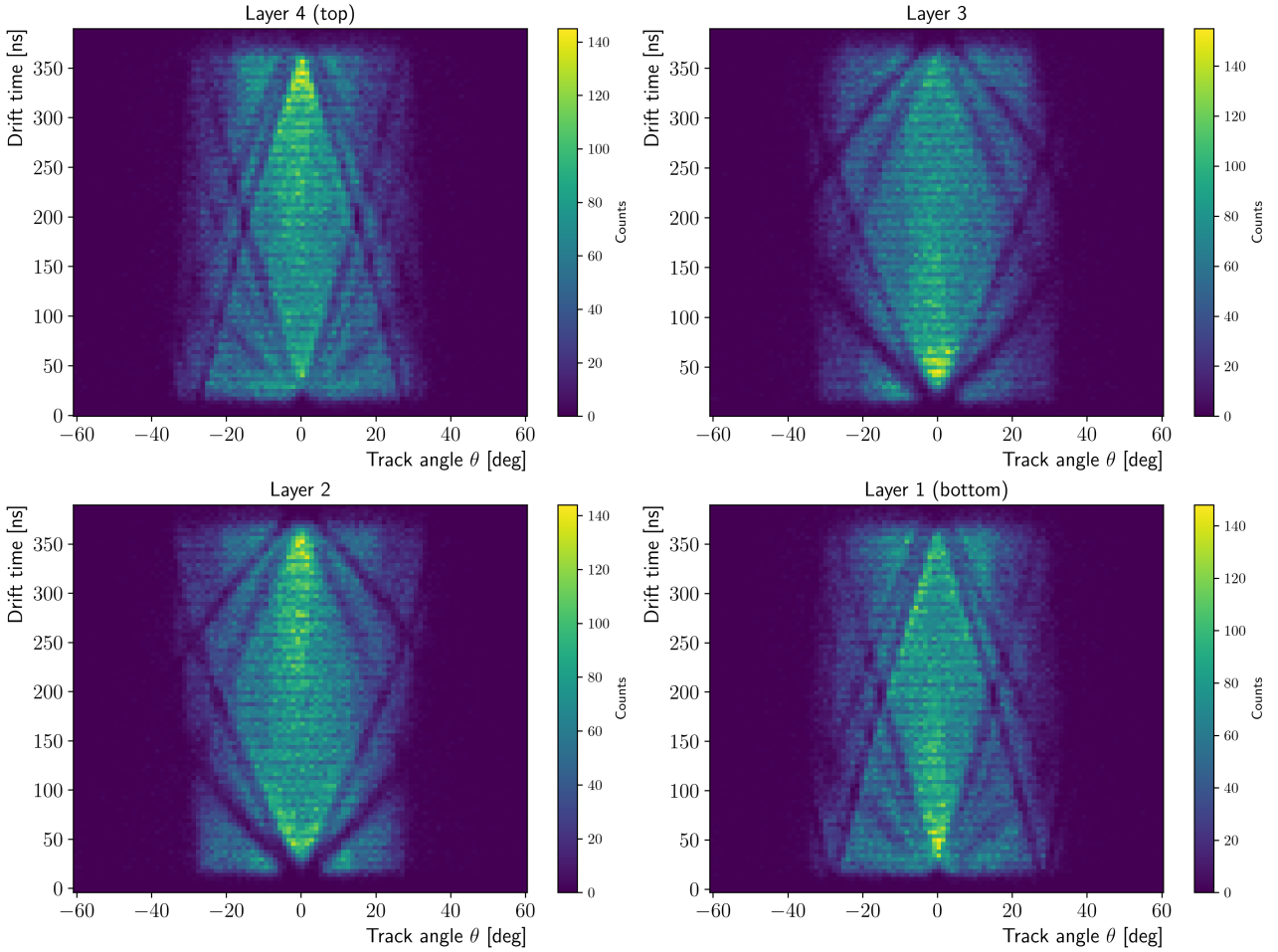


Figure 4.41: Drift time as a function of track angle for each layer.

phase space: given a certain θ angle, only certain drift time combinations can form a valid track. This demonstrates that, after track reconstruction, the drift time associated with a given layer is strongly constrained by the inclination of the particle trajectory. This behavior originates from the geometrical constraints imposed by the four-layer track topology. In the x-z projection, a reconstructed trajectory can be approximated by:

$$x(z) = a + \tan(\theta) \cdot z$$

For fixed θ, requiring the same trajectory to cross all four staggered detector layers restricts the range of possible values of the intercept a. Consequently, only a limited range of distances between the trajectory and the sense wire of a given layer is geometrically accessible. Since the measured drift time is monotonically related to this distance, the same restriction appears in the (θ, t_i) plane. The boundaries of the populated regions therefore represent the limiting drift times compatible with a reconstructed four-layer trajectory at a given track angle. Near $\theta = 0$, corresponding to tracks approximately parallel to the z-axis, a broad range of drift distances can be sampled. As the track inclination increases, the transverse displacement of the trajectory across the detector $\Delta x \propto \Delta z \tan\theta$, progressively restricts the allowed positions of the track relative to the staggered wires, producing the slanted boundaries visible in the two-dimensional distributions.

To further investigate the correlations between hits belonging to the same reconstructed trajectory, the drift times measured in adjacent detector layers are considered pairwise. Figure 4.42 shows the joint distributions for the three neighboring-layer pairs (L4,L3), (L3,L2), and (L2,L1).

A pronounced anti-correlation is observed in all three cases. Small drift times in one layer are preferentially associated with large drift times in the neighboring layer, and vice versa. This behavior is a consequence of the staggered sense-wire geometry: for a given trajectory, approaching the wire of one

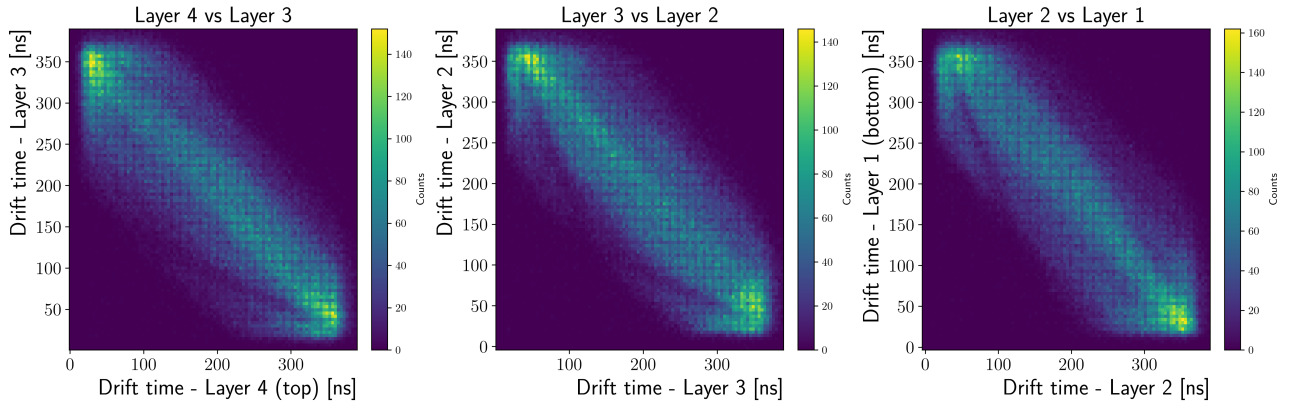


Figure 4.42: Adjacent layer correlations.

layer generally corresponds to increasing the distance from the wire of the adjacent layer.

The anti-correlation observed between adjacent-layer drift times suggests that their sum may be approximately constrained. To test this hypothesis, the quantities $S_{43} = t_4 + t_3$, $S_{32} = t_3 + t_2$, $S_{21} = t_2 + t_1$ are considered. For nearly vertical tracks, the staggered geometry makes the two drift distances approximately complementary, so that $S_{i,i+1} = t_i + t_{i+1} \approx t_{\max}$.

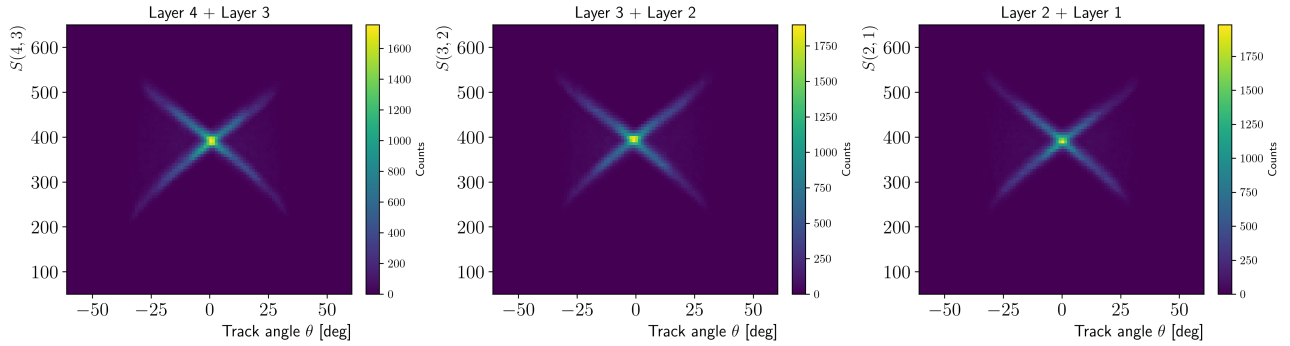


Figure 4.43: Adjacent layer sums.

The results are shown in Figure 4.43 and tabulated in Table 4.10.

Layer pair	$< t_{\max} >$	$\sigma(t_{\max})$
L4,L3	392.06	13.19
L3,L2	393.41	13.67
L2,L1	392.16	13.05

Table 4.10: Adjacent layer sums numerical results.

These are compatible with the $\approx 390ns$ value which is close to the maximum drift time of the drift cell. The observed correlations therefore indicate that the modified timebox shapes arise naturally from the detector geometry and track selection, rather than from an anomalous response of the individual layers.

4.3.2 Falkon's theory recall: hyperparameters and calibration procedure

Model hyperparameters

While Falkon was introduced in Chapter 2, it is useful to recall some of its properties. The input samples for Falkon are the reconstructed tracks which are represented by a 5-dimensional feature vector:

$$\vec{x} = (t_4, t_3, t_2, t_1, \theta) \in \mathbb{R}^5$$

Recalling what was said in the preceding chapter, Falkon's main hyperparameters are:

$N_{\mathcal{R}}$ determines the number of reference tracks used in each two-sample comparison. Increasing $N_{\mathcal{R}}$ improves the statistical representation of the reference distribution, reducing fluctuations due to finite sampling. However, it also increases the computational cost of each Falkon run.

$N_{\mathcal{D}}$ defines the number of reconstructed tracks collected from the monitored data stream before computing one test statistic. It therefore controls the trade-off between statistical precision and time resolution. Smaller batches allow faster detection of changes but lead to broader fluctuations. Larger batches produce more stable test statistics at the cost of increased latency.

M is the number of Nyström centers. This controls the effective capacity of the kernel classifier and the accuracy of the low-rank kernel approximation. A larger number of centers increases the ability of the model to resolve fine structures in feature space, but also increases computational cost and may make the test more sensitive to statistical noise or reconstruction artifacts. Therefore M must be chosen together with the regularization parameter λ. For instance a very large M with a too small λ can yield an unstable configuration.

Kernel length scale σ defines the length scale of the kernel in the standardized feature space. Small values of σ make the classifier sensitive to very local differences between the two samples. Larger values instead produce a smoother comparison by correlating points over larger distances. The value of σ is therefore chosen after feature standardization, inspecting the distribution of pairwise distances between reference tracks.

Regularization parameter λ controls the smoothness and stability of the learned classifier. Smaller values allow the model to fit finer differences between the two samples, increasing sensitivity but also making the statistic more vulnerable to finite-sample fluctuations and reconstruction artifacts. Larger values suppress these fluctuations and produce a more stable null distribution, but may reduce sensitivity to small distributional shifts. In this work, λ is selected by requiring the reference-vs-reference null distribution to be stable and approximately compatible with a χ^2-like law, while preserving separation between reference and altered data samples.

Calibration of the test statistic

The procedure and the importance for the calibration is already described in depth in Chapter 2 and exemplified in Figure 2.4. As an additional note the calibration is also used to validate the choice of hyperparameters. Because the test statistic distribution is expected to be well behaved, if strong deviations appear, such as bimodality or unstable tails, then the selected hyperparameters may not be optimal. For instance the model could be too sensitive to finite-sample fluctuations or reconstruction artifacts and so instability would result since the test statistic will likely "jump" each time Falkon is run. So while **hyperparameters define the test, it is calibration that validates it**.

4.3.3 Calibration results (reference-vs-reference)

Sampling procedure implementation

In order to build the samples, typically data standardization is performed first via the following Code 4.1:

```
    # Standardizes the input data using in-place zero copy operations.
    def training_standardize_in_place(X: cp.ndarray, means: cp.ndarray | None = None, stds:
    cp.ndarray | None = None):
        if means is None:
            means = X.mean(axis=0)
        if stds is None:
            stds = X.std(axis=0)
```

```
        X -= means            # subtract column means
        X /= stds             # divide by column stds
        return X
```

Listing 4.1: Function to do in-place data standardization.

As it was shown earlier in Chapter 2, the standardization constants μ_{ref} and σ_{ref} are computed one time and then recalled during the pipeline operation. Then the samples are built, again using a pure GPU approach, via the function shown in Code 4.2:

```
    # Builds up the reference-vs-reference samples assigning the correct labels
    def training_build_training(
        reference: cp.ndarray,
        data: cp.ndarray,
        n_ref: int,
        n_data: int,
        std_flag: bool = True,
        dtype: cp.dtype = cp.float64,
        ref_means: cp.float64 | None,
        ref_stds: cp.float64 | None
    ):

    # Sample independently from reference and data
    ref_perm = cp.random.permutation(reference.shape[0])
    data_perm = cp.random.permutation(data.shape[0])

    ref_idxs = ref_perm[:n_ref]
    data_idxs = data_perm[:n_data]

    # Allocate
    train_set = cp.zeros((n_ref + n_data, reference.shape[1]), dtype=dtype)

    # Fill
    train_set[:n_ref] = reference[ref_idxs]
    train_set[n_ref:] = data[data_idxs]

    # Labels
    train_labels = cp.zeros((n_ref + n_data, 1), dtype=dtype)
    train_labels[n_ref:] = 1.0

    # Standardization
    if std_flag:
        train_set = training_standardize_in_place(train_set, means=ref_means, stds=ref_stds)

    # Convert to torch (zero-copy via DLPack)
    return torch.from_dlpack(train_set), torch.from_dlpack(train_labels)
```

Listing 4.2: Function to build the reference-vs-reference samples.

So the function assigns labels 0 to the reference points and 1 to the data points that, in the case of calibration runs, are again taken from the same reference distribution as explained earlier. This same function is also used later when real data is used as it will be shown later.

Results

On the σ hyperparameter The RBF kernel width was chosen by inspecting the distribution of pairwise Euclidean distances in the standardized reference feature space.

For a subsample of 10^4 reference tracks, the median pairwise distance was 2.85, while the 90-th percentile was 4.63. The adopted value is:

$$\sigma = 4.5$$

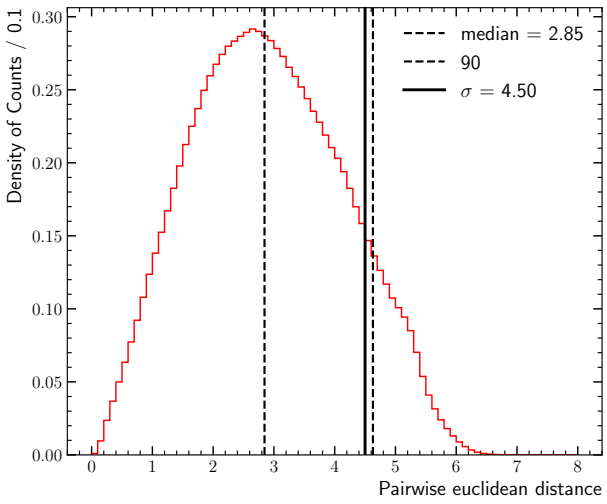


Figure 4.44: Pairwise euclidean distance distribution on the standardized input dataset.

and is therefore close to the upper quantiles of the typical reference distance scale. With this choice, the RBF kernel remains sizeable over most reference-track separations, with $k(d_{50}) \simeq 0.82$ and $k(d_{90}) \simeq 0.59$. The comparison is therefore performed on a smooth feature-space scale, reducing sensitivity to very local statistical fluctuations. This is the same value as reported in [2, 9].

On the model parameters The model parameters were chosen keeping several things into consideration:

- **Model complexity** the model should not be excessively large so as to require very large input datasets. In this specific application, some of the non reference datasets are quite small and a large model would preclude their usage. It is important to keep in mind that any given dataset must have a number N of items that has to be such that $N \geq N_{\mathcal{D}}$.
- **Computational cost** the model should not be unmanageable with respect to the computing resources available. Large models, especially with a large number of Nyström centers, saturate the available GPU memory very fast.
- **Latency requirements** Falkon should respond within a certain amount of time that has to be consistent with the experimental requirements. In a live system where an input batch arrives at the pipeline input every Δt seconds, if the time required by the pre-processor and track-reconstructor to produce an input dataset for Falkon is t_{lat}, then naming the time required for Falkon to produce a test statistic from that input dataset t_F one has:

  $$t_{\text{F}} + t_{\text{lat}} \leq \Delta t$$

  This is analyzed in detail in Chapter 5.
- **Test stability and sensitivity** Falkon should be able to produce a stable test statistic, as previously said, and sufficiently sensible to keep the distributions of the test statistics due to non-reference data well separated from the reference one.
- **Numerical precision** Since Falkon works in `fp64` by default while the output of the track-reconstruction stage is in `fp32`, the dataset is usually promoted to fp64 before being fed to Falkon. This of course does not recover the lost precision, but the stability of the test was checked, as it will be shown in the following, to ensure that fp32-related artifacts did not induce abnormal behaviour in the test statistic distribution. Further, Falkon was also run in fp32 mode, so in the calibration has been performed both for fp64 and fp32 running modes.

Calibration results - FP64 The model parameters have been found via a grid-search. For each set of parameters, 1000 samples of the test statistic in reference-vs-reference mode have been sampled

in order to estimate the null distribution. This has then be fitted with a scaled χ^2 model. The fit is then validated via a Kolmogorov-Smirnov test which returns both the maximum distance between empirical CDF and fitted CDF and the goodness-of-fit p-value in order to check whether the test statistic null distribution is compatible with the χ^2 model.

This is done in Python using the `scipy.stats` module from which one imports `chi2` and `kstest`:

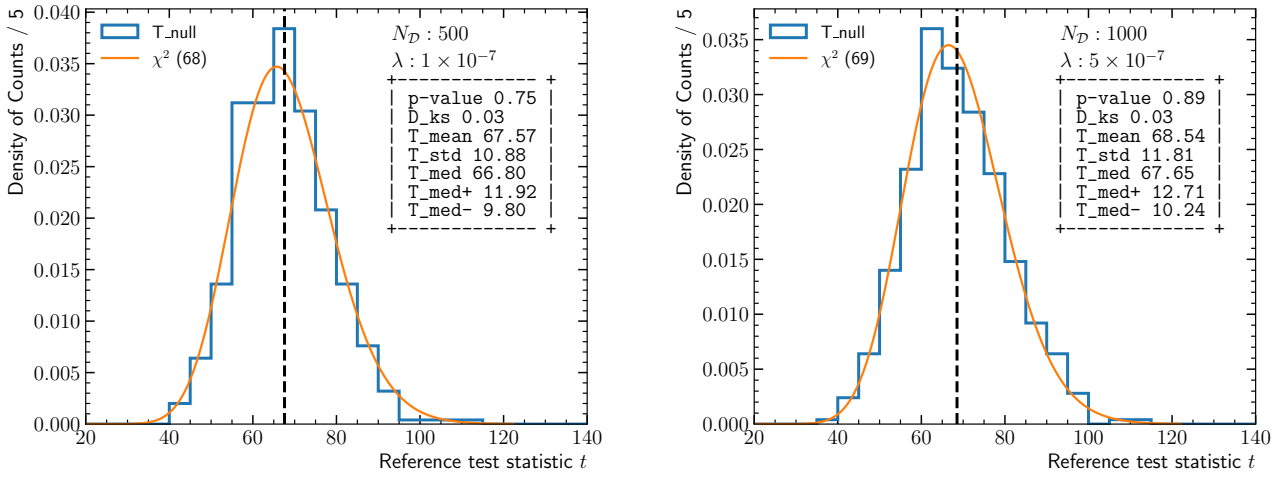
```
# Fit scaled chi-square with loc fixed to zero and scale 1
df, loc, scale = chi2.fit(T_null, floc=0, fscale=1)
# KS goodness-of-fit diagnostic
ks_D, ks_p = kstest(T_null, "chi2", args=(df, loc, scale))
```

The following Table 4.11 shows the selected models when running Falkon in native FP64 mode.

idx	$N_{\mathcal{R}}$	$N_{\mathcal{D}}$	M	σ	λ	dof	T_mean	T_std	ks_D	ks_p-value
0	2000	500	2000	4.5	1×10^{-8}	98	97.79	12.66	0.037	0.49
1	2000	500	2000	4.5	1×10^{-7}	68	67.57	10.89	0.029	0.75
2	2000	1000	2000	4.5	1×10^{-8}	134	133.97	17.10	0.032	0.68
3	2000	1000	2000	4.5	5×10^{-7}	69	68.54	11.82	0.025	0.89

Table 4.11: Selected models in `fp64` mode. M denotes the number of Nyström centers. The acronym `ks` in the table header stands for Kolmogorov-Smirnov, so `ks_D` and `ks_p-value` result from the χ^2 model fitting procedure to the test statistic distribution.

Out of the models listed in Table 4.11 only two were retained: one for $N_{\mathcal{D}} = 500$ (idx=1) and one for $N_{\mathcal{D}} = 1000$ (idx=3). These were chosen because of the better agreement (larger p-value and smaller KS distance) with a χ^2 model. Also, larger models (e.g. with $N_{\mathcal{D}} > 1000$) were not considered in this study since the search was aimed towards smaller models. In fact, some of the altered-data samples that will be later used in the reference-vs-data tests contain very little data for each batch, and increasing $N_{\mathcal{D}}$ beyond ≈ 1000 would result in many batches being dropped.



(a) Model with $N_{\mathcal{D}} = 500$ and $\lambda = 1 \times 10^{-7}$.

(b) Model with $N_{\mathcal{D}} = 1000$ and $\lambda = 5 \times 10^{-7}$.

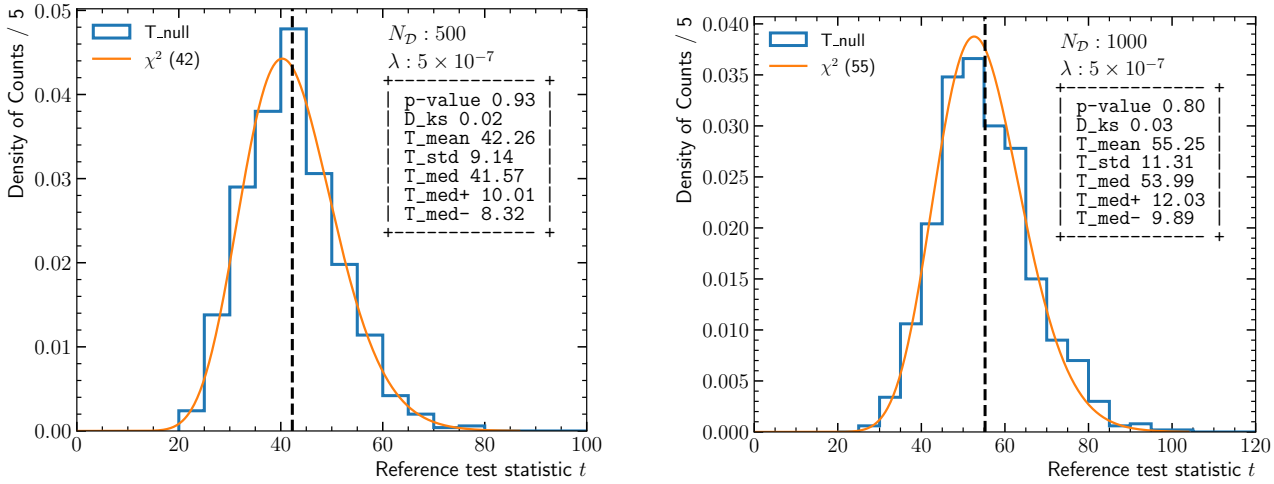
Figure 4.45: Test statistic distributions for reference-vs-reference tests in `fp64` for the $N_{\mathcal{D}} = 500$ and $N_{\mathcal{D}} = 1000$ models.

Figure 4.45 shows the χ^2 models fit to the test statistic distributions for the two models. The two models show similar shapes (the number of inferred degrees of freedom is nearly the same) but the hyper-parameters are different. This will have consequences when the reference-vs-data tests are performed, as it will be shown. What is found is that the larger model is indeed able to capture more details from the data, resulting in higher sensitivity which manifests with stronger separation between various test statistic distributions with respect to both the reference-vs-reference distribution and the other reference-vs-data distributions.

Calibration results - FP32 A similar procedure was conducted also in `fp32` mode. It is important to note that the kernel width σ was kept fixed also in the `fp32` configuration, since it is determined by the geometry of the standardized feature space rather than by the numerical precision used in the Falkon solver. In particular, $\sigma = 4.5$ lies close to the 90-th percentile of the pairwise Euclidean distances between reference tracks after standardization, corresponding to a smooth comparison scale in feature space. Since the reconstructed features and their standardized distance distribution are unchanged when switching from `fp64` to `fp32` Falkon arithmetic, the same kernel scale remains well motivated. In `fp32` the main issue is the numerical precision so in this case $\lambda \geq 5 \times 10^{-7}$ were considered.

idx	N_REF	N_DATA	N_CENTERS	σ	λ	dof	T_mean	T_std	ks_D	ks_p-value
0	2000	500	2000	4.5	5×10^{-7}	42	42.26	9.14	0.017	0.93
1	2000	500	2000	4.5	1×10^{-6}	38	38.26	8.67	0.019	0.86
2	2000	1000	2000	4.5	5×10^{-7}	55	54.72	10.98	0.028	0.80
3	2000	1000	2000	4.5	1×10^{-6}	38	37.99	8.96	0.030	0.73

Table 4.12: Selected models in `fp32` mode.



(a) Model with $N_{\mathcal{D}} = 500$ and $\lambda = 5 \times 10^{-7}$.

(b) Model with $N_{\mathcal{D}} = 1000$ and $\lambda = 5 \times 10^{-7}$.

Figure 4.46: Test statistic distributions for reference-vs-reference tests in `fp32` for the $N_{\mathcal{D}} = 500$ and $N_{\mathcal{D}} = 1000$ models.

The results are reported in Table 4.12. For the `fp32` configuration, an **a-posteriori** study showed that the loss of sensitivity is not controlled by the regularization parameter λ alone. After fixing $\lambda = 5 \times 10^{-7}$, which was chosen as the lowest value retained for numerical-stability reasons, the comparison between $N_{\mathcal{D}} = 500$ and $N_{\mathcal{D}} = 1000$ shows that the larger monitored sample produces significantly larger test-statistic separations during reference-vs-data comparisons. This suggests that, in reduced precision, the statistical power provided by $N_{\mathcal{D}}$ plays an important role in recovering sensitivity to weak distributional shifts. The test statistics with their χ^2 model fits for the $N_{\mathcal{D}} = 500$ and $N_{\mathcal{D}} = 1000$ models are shown in Figure 4.46.

4.3.4 Altered-data results (reference-vs-data)

In order to check the behavior of the test statistic when altered-data is injected in the pipeline, six additional datasets were considered. These datasets have been collected under degraded detector operating conditions. In three of them, the driving voltage of the drift tubes was altered and in the other three the threshold levels in the readout logic were modified. Each of the dataset was collected under only one of these perturbations (i.e. they were not mixed).

As it was previously said, when data that is not drawn from the reference distribution enters the pipeline (i.e. from an altered distribution), the test statistic produced by the reference-vs-data tests

will shift. When one runs multiple experiments and injects data batches drawn from a distribution that was obtain during a degraded operation condition, a shifted distribution of test statistic appears. The separation between the reference distribution and each altered distribution and between the altered distributions themselves is quantified by the following quantity:

$$S_{i,j} = \frac{|\mu_i - \mu_j|}{\sqrt{\sigma_i^2 + \sigma_j^2}}, \tag{4.11}$$

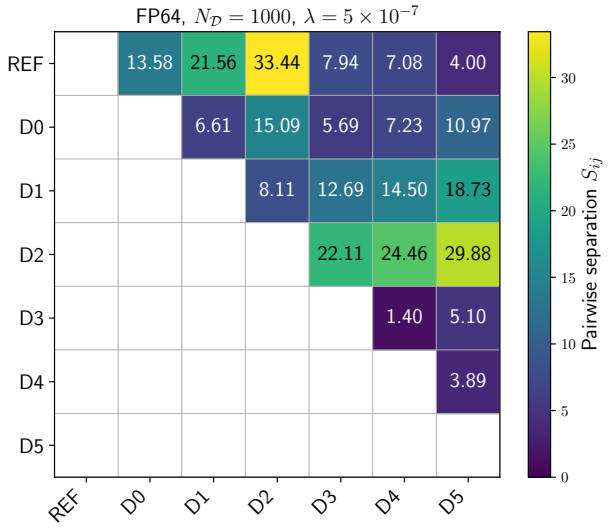
where μ_i and μ_j are the centers of their respective distributions and σ_i and σ_j are their standard deviations. This is called **combined-width separation**. It measures the separation between distributions i and j in units of their combined standard deviations so that it is possible to assert whether two distributions are well separated compared to their widths. This quantity is then used to measure the sensitivity of the test. A companion quantity to Equation (4.11) is given by:

$$\tilde{S}_{i,j} = \frac{\left|\tilde{T}_i - \tilde{J}_j\right|}{\sqrt{w_i^2 + w_j^2}}, \qquad w_i = \frac{q_{84,i} - q_{16,i}}{2} \tag{4.12}$$

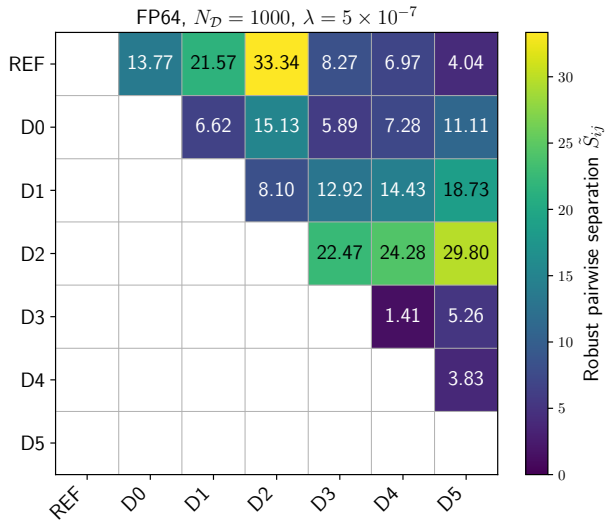
which uses the medians and the difference between the 84th and 16th quantiles and could be more robust with respect to the mean and standard deviation.

FP-64 models

To evaluate the model behavior, the pairwise separation between the centers of each distribution according to Equation (4.11) and Equation (4.12) has been computed and the resulting matrices are shown here in Figure 4.47 and Figure 4.48.



(a) Combined width-separation $N_{\mathcal{D}} = 1000$, $\lambda = 5 \times 10^{-7}$.

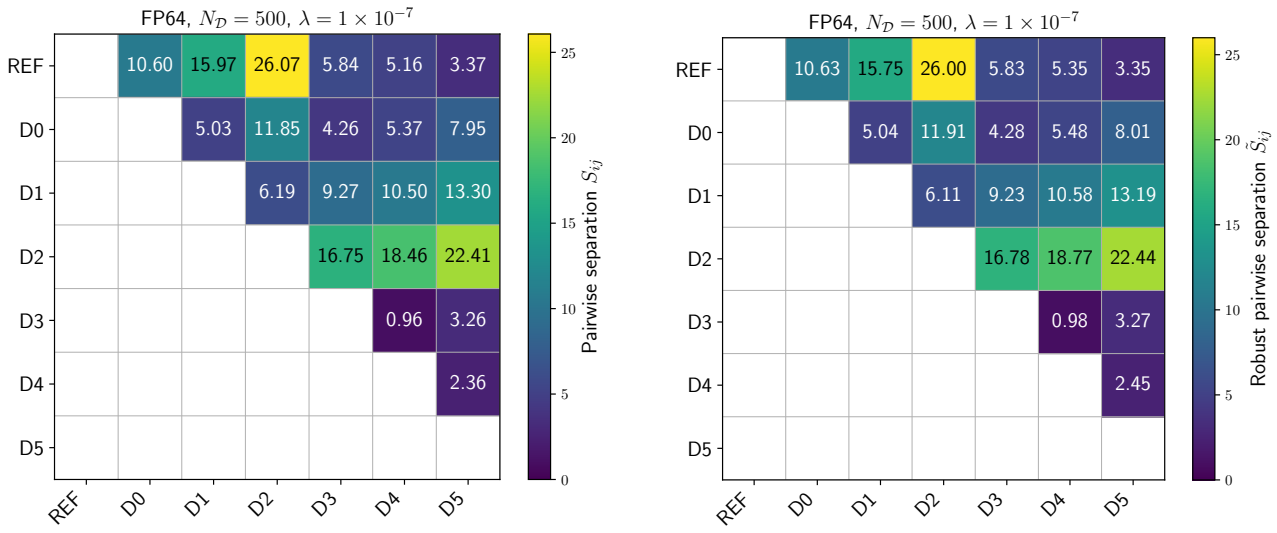


(b) Robust separation $N_{\mathcal{D}} = 1000$, $\lambda = 5 \times 10^{-7}$.

Figure 4.47: Pairwise separations between test statistic distributions for the `fp64` $N_{\mathcal{D}} = 1000$, $\lambda = 5 \times 10^{-7}$ model obtained using both the combined-width separation and the more robust approach based on the medians and quantile differences.

The entries involving the reference distribution quantify the sensitivity of the model to each altered condition, while the altered–altered entries quantify the diagnostic separability between different anomaly classes. For this reason, the matrix will also be referred to as a sensitivity/separation matrix.

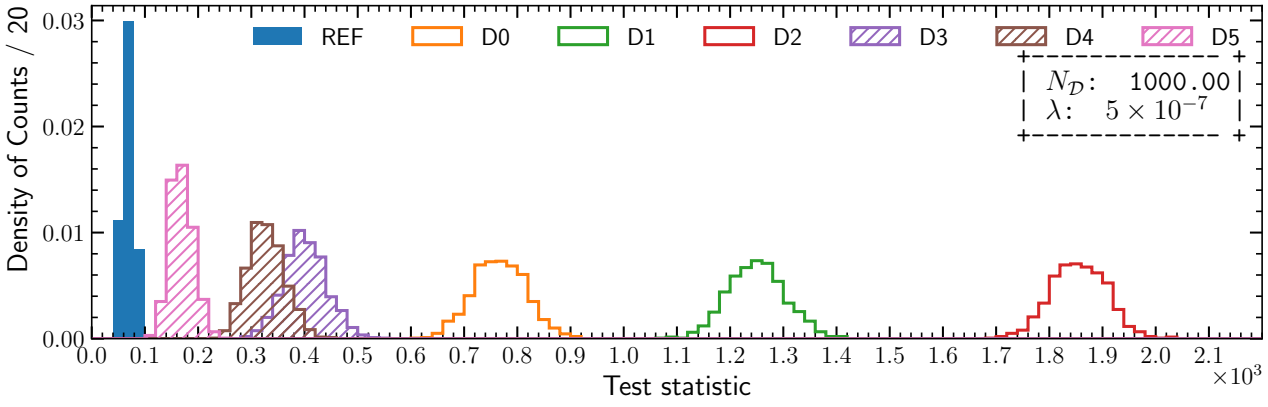
Comparing Figure 4.47a with Figure 4.48a, and Figure 4.47b with Figure 4.48b one can see how the larger model with $N_{\mathcal{D}} = 1000$ yields higher separation scores. On the first row the reference-vs-data separations are reported, which are the sensitivities. Conversely, the other rows report the data inter-distributional separations. Overall, the larger model supersedes the smaller one.



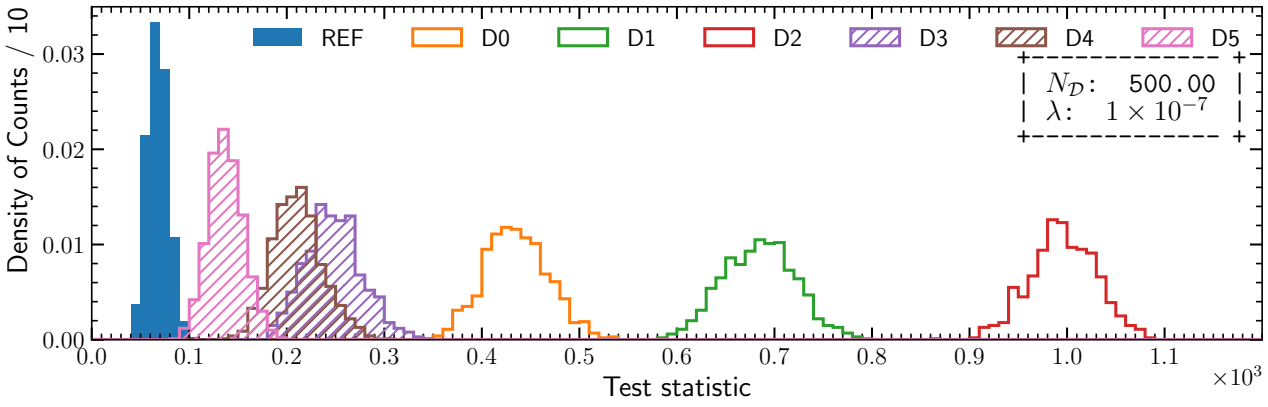
(a) Combined width-separation $N_{\mathcal{D}} = 500$, $\lambda = 1 \times 10^{-7}$.

(b) Robust separation $N_{\mathcal{D}} = 500$, $\lambda = 1 \times 10^{-7}$.

Figure 4.48: Pairwise separations between test statistic distributions for the `fp64` $N_{\mathcal{D}} = 500$, $\lambda = 1 \times 10^{-7}$ model obtained using both the combined-width separation and the more robust approach based on the medians and quantile differences.



(a) $N_{\mathcal{D}} = 1000$ and regularization $\lambda = 5 \times 10^{-7}$.



(b) $N_{\mathcal{D}} = 500$ and regularization $\lambda = 1 \times 10^{-7}$.

Figure 4.49: Overview of the test statistic distributions in the case of reference-vs-data comparisons for the $N_{\mathcal{D}} = 500$ and $N_{\mathcal{D}} = 1000$ models in `fp64`. In each of their respective figures, the histograms are plotted with the same binning to enforce comparability. Further, etch-filled and empty-filled histograms identify the two kinds of anomalies from which the altered-distributions have been drawn varying the parameters.

Figure 4.49 shows how the test statistic distributions move for the considered models. A common issue that affects all models is the generally small separation between the `D4` and `D3` distributions. Also, the larger model shows a better ability in separating the altered-data distributions from the reference one, explaining graphically what was already observed in the separation/sensitivity matrices Figure 4.47 and Figure 4.48.

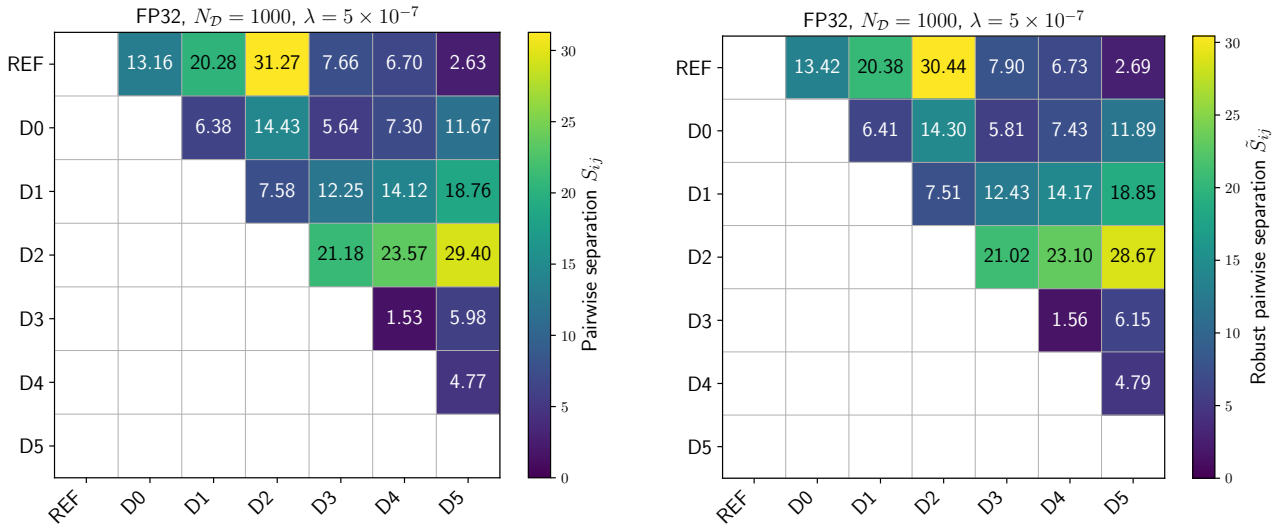
The final step of the algorithm is the association of the p-value to the test statistic that the model outputs for each reference-vs-data test. The p-value has been computed in the two ways that were outlined in Equation (2.4) and Equation (2.5). The results are presented in Table 4.13.

Anomaly	$N_{\mathcal{D}} = 500$, `emp`	$N_{\mathcal{D}} = 500$, χ^2-`fit`	$N_{\mathcal{D}} = 1000$, `emp`	$N_{\mathcal{D}} = 1000$, χ^2-`fit`
D0	$< 10^{-5}$	$< 10^{-5}$	$< 10^{-5}$	$< 10^{-5}$
D1	$< 10^{-5}$	$< 10^{-5}$	$< 10^{-5}$	$< 10^{-5}$
D2	$< 10^{-5}$	$< 10^{-5}$	$< 10^{-5}$	$< 10^{-5}$
D3	$< 10^{-5}$	$< 10^{-5}$	$< 10^{-5}$	$< 10^{-5}$
D4	$< 10^{-5}$	$< 10^{-5}$	$< 10^{-5}$	$< 10^{-5}$
D5	$< 10^{-5}$	$< 10^{-5}$	$< 10^{-5}$	$< 10^{-5}$

Table 4.13: Median p-values for each anomaly kind for the `fp64` models.

FP-32 models

In the `fp32` case, the same procedure is applied and the resulting separation/sensitivity matrices are shown in Figure 4.50 and Figure 4.51. One can again spot that the larger model achieves higher scores overall with respect to the smaller model. It is also worth noticing that the $N_{\mathcal{D}} = 1000$ `fp32` model produces higher separation scores in some pairs with respect to the corresponding `fp64` model.

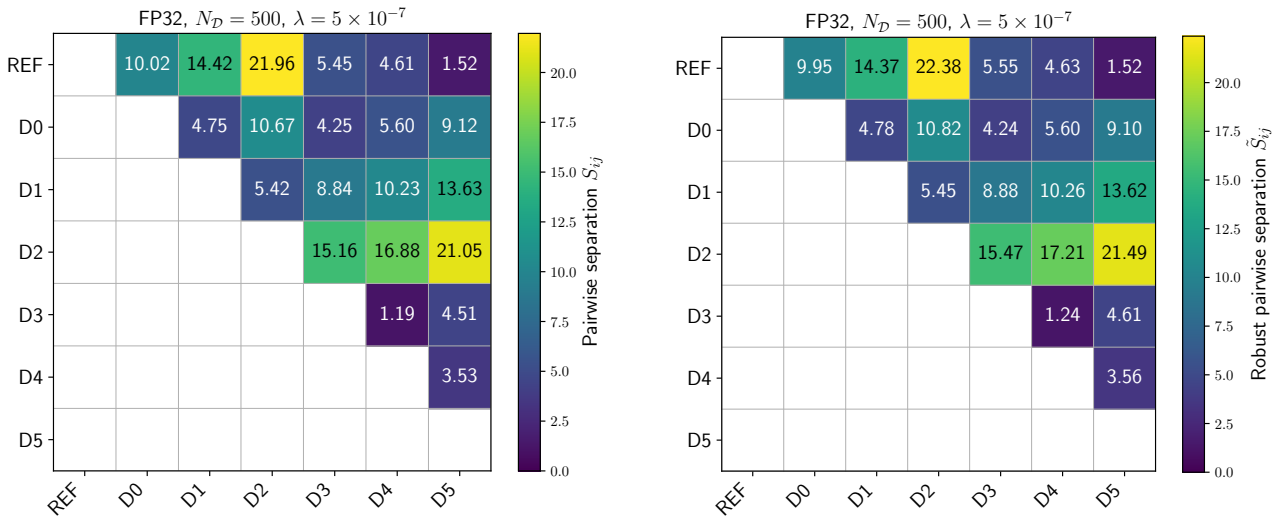


(a) Combined width-separation $N_{\mathcal{D}} = 1000$, $\lambda = 5 \times 10^{-7}$.

(b) Robust separation $N_{\mathcal{D}} = 1000$, $\lambda = 5 \times 10^{-7}$.

Figure 4.50: Pairwise separations between test statistic distributions for the `fp32` $N_{\mathcal{D}} = 1000$, $\lambda = 5 \times 10^{-7}$ model obtained using both the combined width-separation and the more robust approach based on the medians and quantile differences.

Finally Figure 4.52 shows the full overview on how the test statistic distributions move for the considered models. It is possible to see how the ordering of the distributions is maintained with respect to the `fp64` plot counterparts. In this case however, the separation between the reference and D5 distribution is particularly poor, signalling worsened sensitivity with respect to the double precision case for both the large and small model. In this case in fact, one has that the separation is only 1.52 and 2.69 for $N_{\mathcal{D}} = 500$ and $N_{\mathcal{D}} = 1000$ respectively, whereas in the `fp64` case these were significantly higher at 2.54 and 4.04 respectively. On the contrary, the `fp32` models seem to be able to better

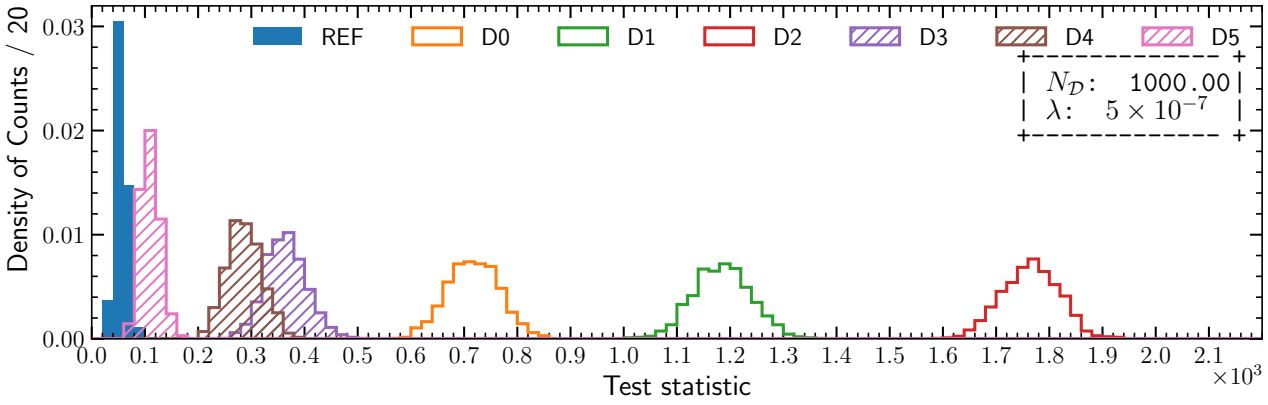


(a) Combined width-separation $N_{\mathcal{D}} = 500$, $\lambda = 5 \times 10^{-7}$.

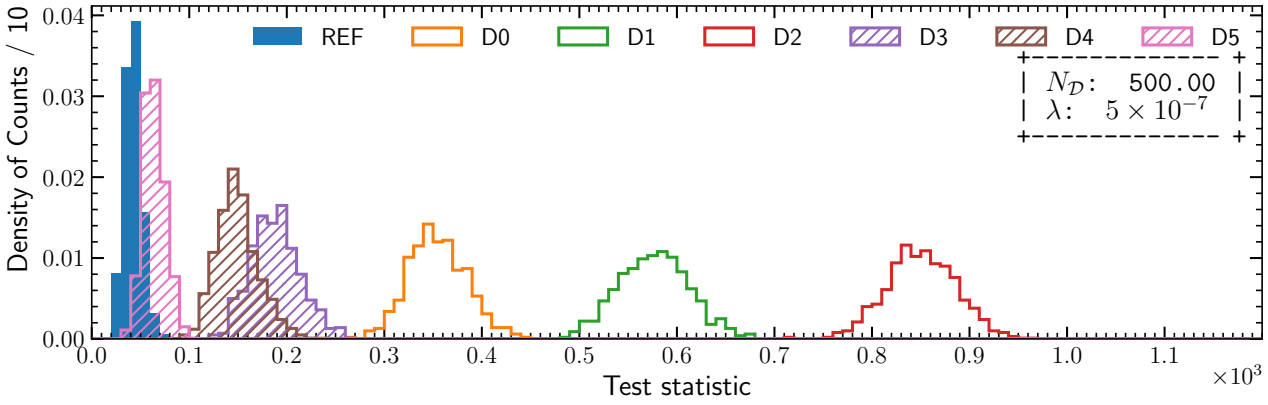
(b) Robust separation $N_{\mathcal{D}} = 500$, $\lambda = 5 \times 10^{-7}$.

Figure 4.51: Pairwise separations between test statistic distributions for the `fp32` $N_{\mathcal{D}} = 500$, $\lambda = 5 \times 10^{-7}$ model obtained using both the combined width-separation and the more robust approach based on the medians and quantile differences.

separate D4 and D5.



(a) $N_{\mathcal{D}} = 1000$ and regularization $\lambda = 5 \times 10^{-7}$.



(b) $N_{\mathcal{D}} = 500$ and regularization $\lambda = 5 \times 10^{-7}$.

Figure 4.52: Overview of the test statistic distributions in the case of reference-vs-data comparisons for the $N_{\mathcal{D}} = 500$ and $N_{\mathcal{D}} = 1000$ models in `fp32`. In each of their respective figures, the histograms are plotted with the same binning to enforce comparability. Further, etch-filled and empty-filled histograms identify the two kinds of anomalies from which the altered-distributions have been drawn varying the parameters.

Anomaly	$N_{\mathcal{D}} = 500$, `emp`	$N_{\mathcal{D}} = 500$, χ^2-`fit`	$N_{\mathcal{D}} = 1000$, `emp`	$N_{\mathcal{D}} = 1000$, χ^2-`fit`
D0	$< 10^{-5}$	$< 10^{-5}$	$< 10^{-5}$	$< 10^{-5}$
D1	$< 10^{-5}$	$< 10^{-5}$	$< 10^{-5}$	$< 10^{-5}$
D2	$< 10^{-5}$	$< 10^{-5}$	$< 10^{-5}$	$< 10^{-5}$
D3	$< 10^{-5}$	$< 10^{-5}$	$< 10^{-5}$	$< 10^{-5}$
D4	$< 10^{-5}$	$< 10^{-5}$	$< 10^{-5}$	$< 10^{-5}$
D5	5.7×10^{-2}	1.98×10^{-2}	6×10^{-3}	1.8×10^{-5}

Table 4.14: Median p-values for each anomaly kind for the `fp32` models.

The p-values for the various anomalies are reported in Table 4.14.

Concluding remark on FP64 vs FP32

The direct comparison shows how, unsurprisingly, single-precision operation yields a less sensitive system. But it is also worth noticing that with the sole exception of D5 test statistic distribution, no appreciable worsening of the separation/sensitivity occurs in all the other cases. This is also confirmed by the p-values reported in Table 4.13 and Table 4.14 for the `fp64` and `fp32` models respectively. With respect to the p-values, the one obtained from the fitted model was truncated at the same level of the empirical p-value since it was not possible to test the agreement further along the upper tail.

The system remains adequately sensible to properly push the various distribution sufficiently apart so as to not confuse them. If for instance one is interested only in altered data of the kind of D0, D1 or D2 then using double precision or single precision yields absolutely no difference, as the scores in matrices show, and the system could run in `fp32` mode. This possibility of running in single-precision depending on the experimental requirements could be a great advantage not only because of the diminished memory footprint, but also because GPU operations in `fp32` precision are much faster than `fp64` (this is architecture dependent and could range anywhere from a $\times 2$ to a $\times 16$ speed-up).

4.3.5 Performance analysis

Introduction

Falkon's response time is largely independent of the input batch size. In fact, Falkon's runtime t_F depends almost exclusively on the model complexity. Specifically, t_F is made up of two components:

$$t_F = t_{\text{sample}} + t_{\text{train}}$$

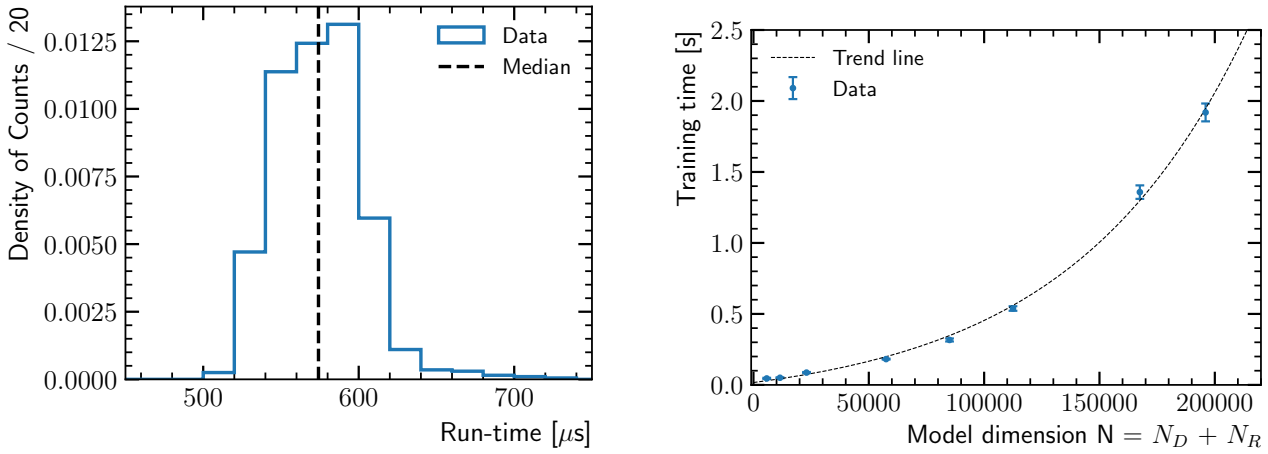
The t_{sample} is the time required to build the samples (to perform either reference-vs-reference or reference-vs-data tests) from the input data batches that are coming from the pre-processor and track-reconstruction blocks. These are built via the Python function shown in Code 4.2 and its variation to allow for reference-vs-data sample building.

It has been found that (Figure 4.53a), in the specific case of this application, regardless of model complexity and input batch size to Falkon, the time required to run this function on the GPU is always $\approx (0.5 \to 0.6)$ ms which is much faster than Falkon's typical run-time for each comparison which lies in the tens to hundreds of milliseconds range.

For this reason one can safely have:

$$t_F \approx t_{\text{train}}$$

This does not necessarily hold for arbitrary-size problems and was not true in the previous implementation [10] where instead sample-building relied on a CPU-running function that performed some operations sequentially rather than in a fully vectorized way.



(a) Distribution of sampling times (median is 574.13μs with a width of 34.66μs). Times are in microseconds.

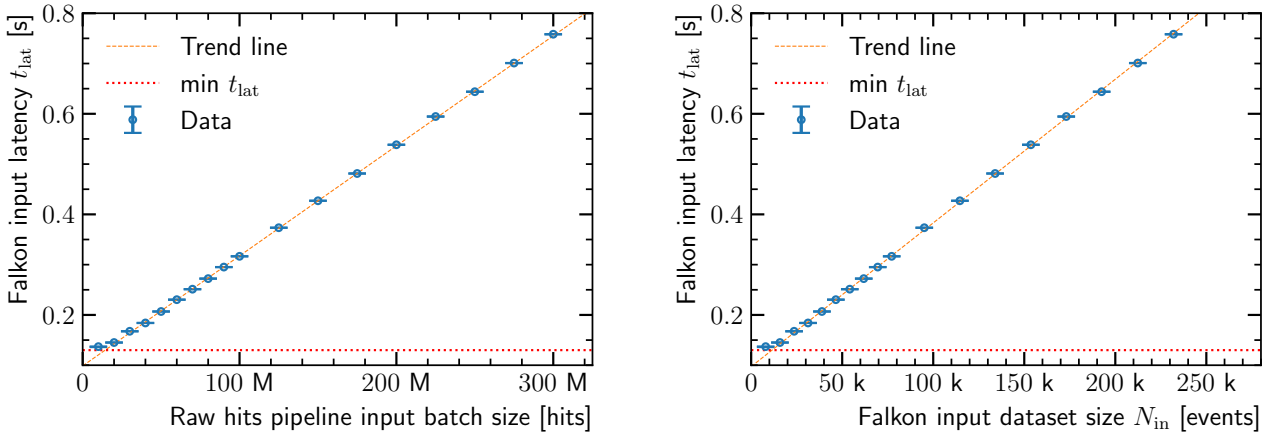
(b) Distribution of training times as a function of the model dimension. Times are in seconds.

Figure 4.53: Sampling and training times for Falkon.

The t_{train} is instead the time required for Falkon to run a test a single time. This is instead heavily dependent on model complexity and is particularly evident when considering a broad amplitude of model dimensions. The larger the model is and the slower Falkon will run as Figure 4.53b shows: each point in the figure represents the median over repeated measurements for a given model configuration, so each point is essentially a different model.

Falkon performance within the pipeline

When inside the pipeline, Falkon operates at a **fixed** operating point and so the model does not change between repeated comparisons.



(a) Latency vs pipeline input batch size.

(b) Latency vs Falkon input dataset size.

Figure 4.54: Falkon input latency t_{lat} as a function of the pipeline input batch size (left) and Falkon input dataset size (right). This is the time it takes the pre-processing and track-reconstruction blocks together to produce the input dataset for Falkon.

At a given raw input batch size, the pre-processing and track-reconstruction blocks build an input dataset for Falkon requiring a time t_{lat} that is called the **input latency**. This is shown in Figure 4.54 where a linear trend line acts as a visual guiding line.

The input latency thus grows in a mostly linear fashion especially for larger raw input batches. Looking now at Table 4.15 and Table 4.16 it is possible to see how, with these small models, the latency is comparable to Falkon training times and sometimes is the dominating time component. If one

$N_{\mathcal{D}}$	λ	t_F
500	1×10^{-7}	$120.38^{+0.55}_{-0.34}$ ms
1000	5×10^{-7}	$88.88^{+0.41}_{-0.09}$ ms

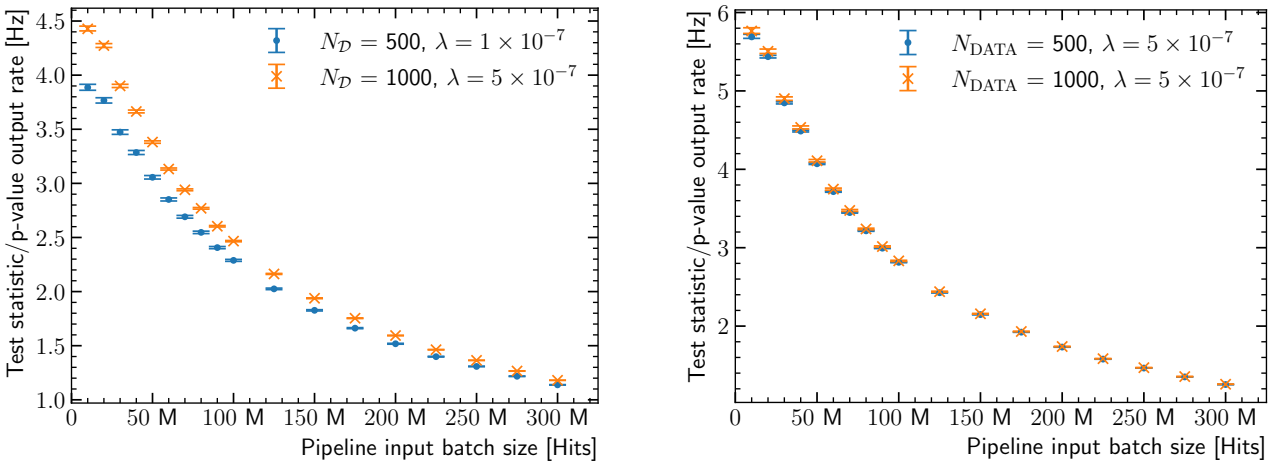
Table 4.15: Values of t_F for each of the selected `fp64` models. The fact that larger models take less time to reach convergence can seem counterintuitive but this has mainly to do with the action of the regularization λ since the number of Nyström centers is the same. With less regularization, Falkon needs more iterations to reach convergence.

considers, for instance, a raw input batch size to the pipeline of 40M raw hits which, looking at Figure 4.54a, this would yield a $t_{\text{lat}} \approx 0.2s$ which, in the case of the `fp64`, $N_{\mathcal{D}} = 1000$ model is the dominating component when compared to $t_{\text{F}} \sim 90$ms.

$N_{\mathcal{D}}$	λ	t_F
500	5×10^{-7}	$38.80^{+0.15}_{-0.14}$ ms
1000	5×10^{-7}	$36.52^{+0.34}_{-0.14}$ ms

Table 4.16: Values of t_F for each of the selected `fp32` models. This time the models differ only by their $N_{\mathcal{D}}$ values and since they have the same regularization, the runtimes are comparable. The slight edge that the larger model has in terms of training time could be due to the larger data representation, allowing Falkon to converge faster.

Falkon is in fact quite fast in solving these small models. If instead one were to select a larger model, for instance referring to Figure 4.53b, selecting a dimension $N \approx 180 \times 10^3$ with $N_{\mathcal{D}} = 80 \times 10^3$ which, looking at Figure 4.54b would impose a latency of $\geq 0.35s$ to have enough data, the value of t_F would in this case be $\geq 1.5s$, and so for larger models the dominating component would be t_F.



(a) `fp64` models.

(b) `fp32` models.

Figure 4.55: Test statistic/p-value output throughputs as a function of the pipeline input batch size, for both the `fp64` (left) and `fp32` (right) models.

The final throughput measurement is shown in Figure 4.55. These figures represent the end-to-end measurement of the pipeline, showing how many tests per unit of time can be executed for each selected model as a function of the pipeline raw input batch size. It is important to stress again the fact that $N_{\mathcal{D}}$ is the number of feature vectors that Falkon samples from the input batch that is present to it (the one produced by the pre-processor and track-reconstructor stages). So if $N_{\mathcal{D}} = 1000$ but one presents, for instance, 10^4 feature vectors, it does not matter. The important thing is that *at least* $N_{\mathcal{D}}$ of them are present.

4.4 Anomaly classification head

A visual inspection of Figure 4.49 and Figure 4.52 shows that the test statistic distributions under the different anomalies are well separated from each other, with the sole exception of the D3 and D4 anomalies that tend to overlap. For these reasons the p-values that are reported in Table 4.13 and Table 4.14 show largely saturated values. Then the following idea suggests itself: while the p-value is the main statistical tool answering the question of whether the monitored batch is anomalous or not, the test statistic value can help in understanding what could have been the source of the observed anomaly. As it was said earlier, the test statistic value has no intrinsic meaning by itself but it gains value once the system has undergone calibration. Specifically, one might introduce a **multiclass classifier** that assigns a label to each test statistic with the most probable origin for the input batch to which that test statistic corresponds. The labels could then be `REF` for the reference and `D0` to `D5` for each anomaly class. Further, through a rejection mechanism, it is also possible to define the `unknown` and `ambiguous` classes, signalling situations in which the classifier recognizes an out-of-support value of the test statistic and situations in which a unique label could not be determined with sufficient confidence respectively.

Regarding the classifier itself, two versions of it have been considered: a simple Bayesian density-based classifier and XGBoost [23] classifier.

4.4.1 Bayesian density-based classifier

Introduction

The first classifier considered in this work is a Bayesian density-based classifier. As it was said earlier, when one calibrates the reference test statistic distribution (fitting a χ^2 model) then it becomes possible to compute the p-values of each monitored batch's test statistic with respect to the reference. In this way the algorithm can decide whether the monitored batch is compatible with the nominal detector condition. Given a significance threshold α, the batch is labelled as compatible with the reference condition if

$$p_{\text{ref}}(T_{\text{obs}}) > \alpha.$$

In this case, no anomaly-type classification is required and the assigned label is `REF`. If instead

$$p_{\text{ref}}(T_{\text{obs}}) \leq \alpha,$$

the batch is considered anomalous with respect to the reference condition, and a density-based Bayesian classifier is used to associate the observed statistic with one of the known injected anomaly classes.

Let c denote one of the known classes. For each class c, an empirical distribution of the test statistic is obtained from calibration samples. A one-dimensional kernel density estimator is fitted to these values, providing an estimate of the class-conditional density

$$p(T \mid c).$$

Given an observed value T_{obs}, the posterior probability of class c is computed using Bayes' theorem,

$$P(c \mid T_{\text{obs}}) = \frac{p(T_{\text{obs}} \mid c)\, P(c)}{\sum_{c'} p(T_{\text{obs}} \mid c')\, P(c')},$$

where $P(c)$ is the prior probability assigned to class c. The most likely known class is therefore

$$\hat{c} = \arg\max_{c} P(c \mid T_{\text{obs}}).$$

The classifier is augmented with two rejection rules. First, a density-support criterion is introduced in order to avoid assigning a known label in regions of the test-statistic axis where none of the known classes has support. Defining

$$d(T) = \max_{c} p(T \mid c),$$

an anomalous observation is labelled as `unknown` whenever

$$d(T_{\text{obs}}) < d_{\text{min}}.$$

The threshold d_{min} is chosen empirically from the validation set as a low quantile of the distribution of $d(T)$. In this analysis, the 0.1% quantile was used, since it retained almost all valid known-class samples while still rejecting clearly unsupported regions between the known test-statistic distributions.

For anomalous observations lying inside the support of the known classes, the posterior probabilities are used to determine whether the anomaly-type assignment is sufficiently confident. If

$$\max_c P(c \mid T_{\text{obs}}) < p_{\text{min}},$$

the observation is labelled as `ambiguous`. Otherwise, it is assigned to the class with the largest posterior probability.

The complete decision rule can therefore be written as

$$\text{label}(T_{\text{obs}}) = \begin{cases} \texttt{REF}, & p_{\text{ref}}(T_{\text{obs}}) > \alpha, \\ \texttt{unknown}, & p_{\text{ref}}(T_{\text{obs}}) \leq \alpha \ \wedge \ d(T_{\text{obs}}) < d_{\text{min}}, \\ \texttt{ambiguous}, & p_{\text{ref}}(T_{\text{obs}}) \leq \alpha \ \wedge \ d(T_{\text{obs}}) \geq d_{\text{min}} \ \wedge \ \max_c P(c \mid T_{\text{obs}}) < p_{\text{min}}, \\ \arg\max_c P(c \mid T_{\text{obs}}), & \text{otherwise.} \end{cases}$$

In this scheme, the reference p-value provides the calibrated anomaly-detection decision, while the Bayesian classifier provides an interpretation of anomalous batches in terms of the known injected conditions. The `unknown` label corresponds to values of the statistic lying outside the density support of all known anomaly classes, whereas the `ambiguous` label corresponds to values lying inside the known support but in regions where the posterior probabilities do not allow a sufficiently confident assignment.

Classifier performance

With respect to the performance of the classifier, the following is reported.

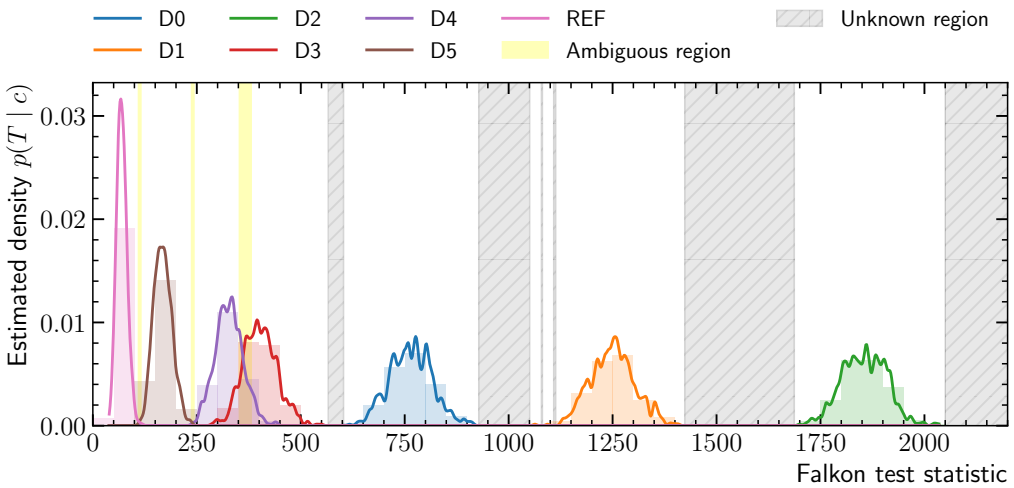


Figure 4.56: Test statistic distributions for the $N_{\mathcal{D}} = 1000$ model in `fp64`. The continuous lines represent the kernel density estimates of the various $p(T|c)$. The shaded regions represent the ambiguous (yellow filled) and unknown (etched grey filled) regions where the classifier does not output a valid reference or anomaly class.

In Figure 4.56 one can see how the density classifier has built the density estimates for the various classes. Also in the regions of the t-space where no support exists, the assigned label is `unknown` since

no known failure modes of the detector induce shifting of the test statistic over those regions. Then the regions where the classifier cannot produce a label with sufficient confidence (for instance two or more competing classes, none of which exceeds a required probability threshold) the label is `ambiguous`.

T_{obs}	Decision	Label	Prob
72.5	`reference compatible`	`REF`	1.00
290.0	`accepted known class`	`D4`	0.95
375.0	`ambiguous low confidence`	`ambiguous`	n/a
1000.0	`unknown low density support`	`unknown`	n/a
1800.0	`accepted known class`	`D2`	1.00

Table 4.17: Classifier output for different values of the observed test statistic T_{obs}.

As an example, looking again at Figure 4.56, some values of the test statistic have been passed to the classifier obtaining the results shown in Table 4.17. The confusion matrix is shown in Figure 4.57.

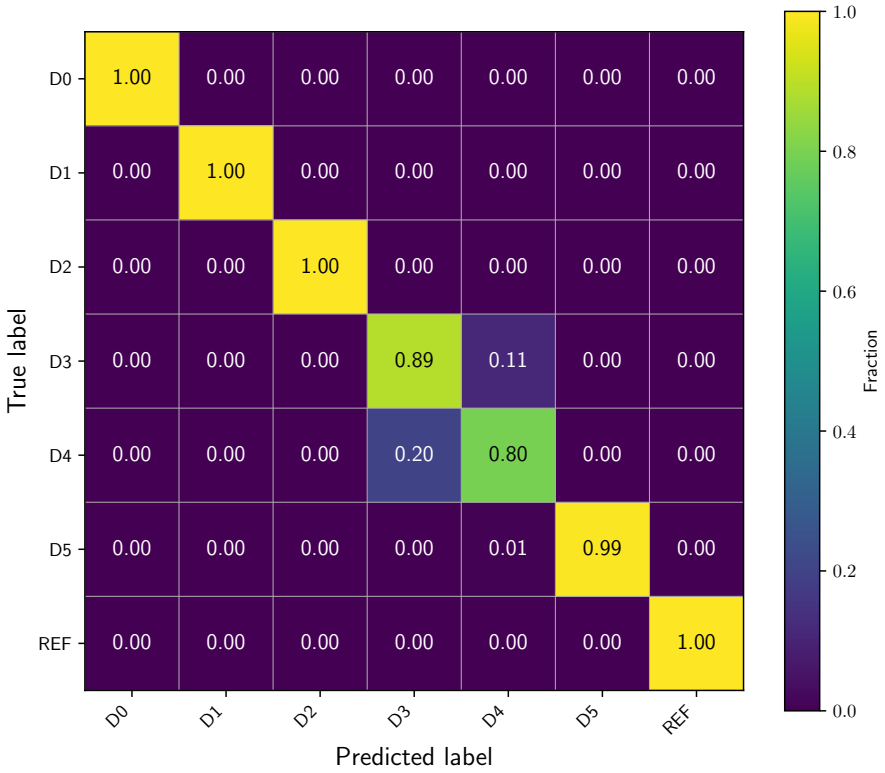


Figure 4.57: Confusion matrix for the Bayesian density-based classifier.

The matrix shows how the classifier performs well overall with the expected difficulty in correctly separating D3 and D4. The full evaluation of the model on the validation set is shown here below:

```
Density-based Bayes classifier evaluation
============================================================
Coverage                  : 0.9457
Total rejection fraction  : 0.0543
Unknown fraction          : 0.0014
Ambiguous fraction        : 0.0529
------------------------------------------------------------
Accepted accuracy         : 0.9675
------------------------------------------------------------
n_total                   : 1400
n_accepted                : 1324
n_rejected                : 76
n_unknown                 : 2
```

```
    n_ambiguous               : 74
    ------------------------------------------------------------
    Decision-status counts:
    accepted_known_class          :   1324  (0.9457)
    ambiguous_low_confidence      :     74  (0.0529)
    unknown_low_density_support   :      2  (0.0014)
```

When the classifier operates online as a classification head attached to the pipeline output, one must make sure that it does not introduce significant latency. The classifier in fact depends only on the scalar value of the test statistic T. After the calibration phase, the class-conditional density estimates $p(T \mid c)$, the posterior probabilities $P(c \mid T)$, and the corresponding decision regions are fixed. Therefore, the classifier response can be precomputed on a dense grid of values of T, producing a lookup table that maps a possible value of the test statistic to the corresponding monitoring label.

For each grid point T_i, the lookup table stores the estimated densities $p(T_i \mid c)$, the posterior probabilities $P(c \mid T_i)$, the maximum density $d(T_i)$ that were defined earlier. In this way when the DQM pipeline outputs the test statistic T_{obs}, the classification stage only requires locating the corresponding position in the precomputed lookup table, or interpolating between neighboring grid points, and returning the stored decision. This avoids repeated online evaluations of the density estimators and makes the classification step deterministic and lightweight. This is reported in Algorithm 7.

Algorithm 7 Lookup-based online classification

Offline calibration:
Fit the reference null distribution of the test statistic.
Fit the class-conditional densities $p(T \mid c)$.
Choose the thresholds $d_{\min}$ and $p_{\min}$.
Build a dense grid $\{T_i\}$ covering the calibrated range of the statistic.
for each grid point T_i **do**
    Compute $p(T_i \mid c)$ for all classes c.
    Compute $P(c \mid T_i)$ for all classes c.
    Assign the corresponding label using the rejection rule.
    Store the posterior probabilities and label in the lookup table.
end for

Online operation:
for each monitored batch **do**
    Compute the observed statistic T_{obs}.
    Locate or interpolate T_{obs} in the lookup table.
    Return the corresponding label and posterior information.
end for

In this way one can obtain the label corresponding to a produced test statistic in a time of $\approx 80\mu$s whereas without the lookup table simplification the same classifier would require ≈ 2.3ms on average. While both delays are small with respect to the typical latencies of the pipeline, the sub-millisecond delay introduced by the faster approach might be more desirable in a broader range of applications.

4.4.2 XGBoost classifier

As a comparison, XGBoost [23] classifier was also considered. The model was trained only on the known detector conditions, using the scalar test statistic T as input. Since XGBoost does not estimate class-conditional densities, unsupported regions were identified with an external support rule based on the empirical ranges of the known training distributions. Samples outside this support were labelled as `unknown`, while samples inside the support but with competing class probabilities were labelled as `ambiguous`.

The XGBoost classifier achieved a performance comparable to the density-based Bayesian classifier in

Classifier	Coverage	Unknown	Ambiguous	Accepted accuracy	Latency
Bayesian lookup	0.945	0.001	0.054	0.966	$\sim 80\ \mu$s
XGBoost	0.928	0.007	0.065	0.966	~ 25 ms

Table 4.18: Comparison between the density-based Bayesian lookup classifier and the XGBoost baseline. Both methods achieve essentially the same accepted accuracy, but the Bayesian classifier provides higher coverage and significantly lower online inference latency.

terms of accepted accuracy. However, it rejected a larger fraction of samples and its online inference latency was significantly larger. In the tested implementation the XGBoost classifier required ≈ 25 ms per statistic which is much larger than the $\approx$ 2ms or $\approx 80\mu$s of the density-based classifier. Since the accepted accuracies of the two methods were essentially equivalent, the density-based Bayesian classifier was preferred for online use because of its higher coverage, simpler interpretation, and lower latency.

Chapter 5

Pipeline memory management and latency

5.1 Memory management

5.1.1 Introduction

The processing pipeline that is the object of this work is based upon three different libraries: cuDF [18], CuPy [19] and PyTorch [24]. While passing objects between these libraries does not pose complications since all of them support DLpack [20] (which is a standard protocol for exchanging tensor data between different libraries), the way in which each of them handles memory needs to be examined. In fact, while DLpack enables for objects to be passed over without copying the underlying memory, each library allocates its own memory with its own memory management system that interfaces with the device driver.

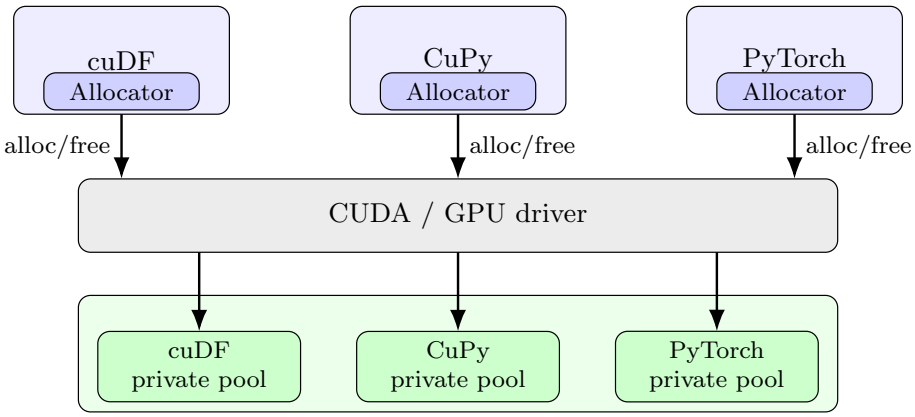


Figure 5.1: Schematic view of independent GPU-memory allocation. Each high-level library uses its own allocator and issues allocation/free requests to the CUDA/GPU driver. As a result, device memory can be partitioned into library-specific private pools or allocation regions.

This is exemplified in the simplified diagram in Figure 5.1. Each library manages its own memory: if it needs more space, it asks the driver for more, when it no longer needs a chunk of memory, it signals the driver to de-allocate it.

It is a well known fact [21, 25] that memory allocations and de-allocations are costly operations, severely impacting not only performance but also memory fragmentation. The macroscopic effect of this was shown earlier in the left plot in Figure 4.3, where the throughput performance of the pre-processor (which makes heavy use of cuDF) was severely impacted by the lack of an improved memory management strategy and this is where Rapids Memory Manager (RMM) enters.

5.1.2 Rapids Memory Manager (RMM)

The idea now is the following: instead of having each library allocate/de-allocate their own memory, a common memory allocator layer is used. Each library interfaces with this layer which, in turn, handles all of the driver calls to allocate/de-allocate memory. This common layer is the Rapids Memory Manager (RMM). Further, RMM allocates memory in large blocks so that subsequent requests for allocations/de-allocations by the libraries are not necessarily routed to the drivers, but trigger internal sub-allocations and sub-releases of memories which are intensively re-used [21].

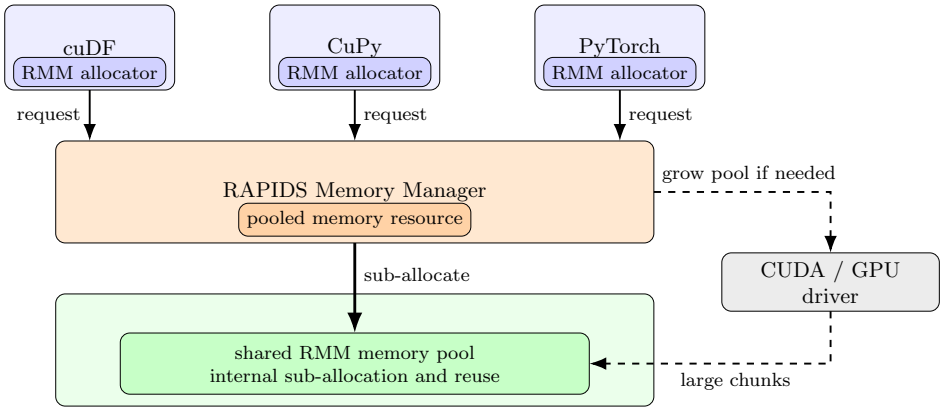


Figure 5.2: Unified GPU-memory allocation through RMM. cuDF, CuPy, and PyTorch route device-memory requests to a common RMM memory resource. In the nominal path, RMM serves these requests by sub-allocating memory from a shared pool already reserved in device memory. Only when the pool cannot satisfy a request does RMM interact with the CUDA/GPU driver to obtain additional large memory regions.

This is shown in Figure 5.2. RMM then does not replace the CUDA driver as the mechanism through which device memory is ultimately obtained. Rather, it changes the allocation pattern seen by the driver. The libraries in fact receive ordinary device pointers, which are used by their CUDA kernels in the usual way. In this way RMM reduces the number of costly driver-level allocation operations while preserving normal GPU memory access semantics.

For instance, if CuPy asks to allocate a certain amount of memory to RMM and RMM checks that that memory is available within the one it has already allocated, it will sub-allocate that, returning immediately to CuPy without having to call the driver. Instead, if the RMM pre-allocated memory pool does not contain sufficient free memory to further allocate what CuPy is requesting, RMM will call the driver for a new allocation, expanding the pool. In this way, one eliminates the many small allocation/de-allocation requests that each library might issue, with few large ones. This has direct impact on performance [25] as it is also shown in the right plot in Figure 4.3.

Known issues with PyTorch RMM allocator

While cuDF and CuPy implement a fully operational allocator to interface with RMM, PyTorch does not at the time of this writing. This is the reason why it was necessary to implement the modifications if Falkon's source code in order to create a compatibility layer as described in Section 4.3.1.

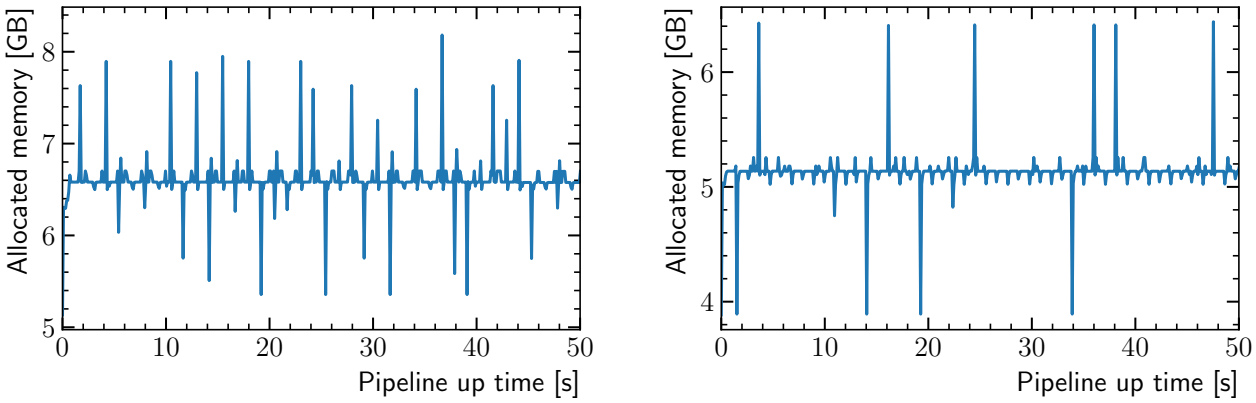
5.1.3 Pipeline memory stability analysis

The role of RMM in this DQM pipeline is particularly important. In fact, the pipeline is supposed to be a live system that can be deployed and kept running for long periods of time and so it is important to ensure that memory usage and management reach a steady state so that the system does not incur in an Out Of Memory (OOM) error or similar issues. Also, given the amount of different operations that are performed, namely:

- pre-processor: makes heavy use of cuDF to do dataframe operations, shuffling, joining and indexing,

- track-reconstructor: uses computationally intensive and memory-intensive CUDA kernels launched via CuPy,
- anomaly detection and DQM: model fitting that consists in heavy computational tasks,

without RMM the GPU driver will receive a large amount of memory allocation/de-allocation requests.



(a) Memory allocations/de-allocations without the usage of RMM.

(b) Memory allocations/de-allocations with RMM within the shared pre-allocated pool.

Figure 5.3: Pipeline memory pressure with and without the usage of RMM.

This is shown explicitly in Figure 5.3a that depicts how device memory varies in time when RMM is not used. Every spike represents a memory allocation (when higher than baseline) and a memory release (when lower than baseline) event. Each of these spikes roughly correspond to a driver call and it is easy to imagine that in a DQM pipeline that needs to execute cyclically for a long period of time this is not ideal and besides the performance problems that have already been pointed out, there is also the issue that this continuous allocations/de-allocations could induce memory fragmentation eventually leading to OOM situations which are of course undesirable.

The situation is instead different in Figure 5.3b. In this case one can see that there are still many allocations a de-allocations and this is completely expected since using or not using RMM does not change the nature of the algorithm. But the difference here is fundamental: while without RMM each spike represent a potential new driver call, in this case instead all of these allocations and de-allocations happen within the already pre-allocated pool and so do not require a driver alloc/de-alloc each time. These are the sub-allocations that were referred to earlier in Figure 5.2.

Figure 5.4 shows the **time-trace** of memory events when RMM is enabled. This plot is produced using a monitoring thread that samples GPU memory usage at fixed intervals (every 1ms) and by having the main program change a stage indicator variable that is read by the monitoring thread. No thread-locking device has been used since only the main thread changes the state of the variable at any given time, plus having Python `threading.Lock` objects can interact in unpredictable ways with CUDA workloads that require synchronization themselves.

Considering now the plot: each colored band depicts a stage of the pipeline and the width of the band is the time required to fully execute it. The red band is the pre-processing stage, the green band is the track-reconstruction stage and the blue band is Falkon stage. There is also a magenta-colored band which is a forced delay that has been placed for aesthetic purposes and has no technical reasoning behind it. A **pipeline cycle** consists in the time between two successive low memory peaks, just before the beginning of each red-band. In this case the blue band has been expanded considerably because Falkon was running multiple times (many toys) within a pipeline cycle since it was performing reference-vs-reference toy generation.

Since we can subdivide the pipeline life cycle, from the memory point of view, as a succession of small identical cycles where the same instructions are performed every time, it is reasonable to assume that **if no net memory balance** occurs over a cycle, the memory usage will remain in a stationary state.

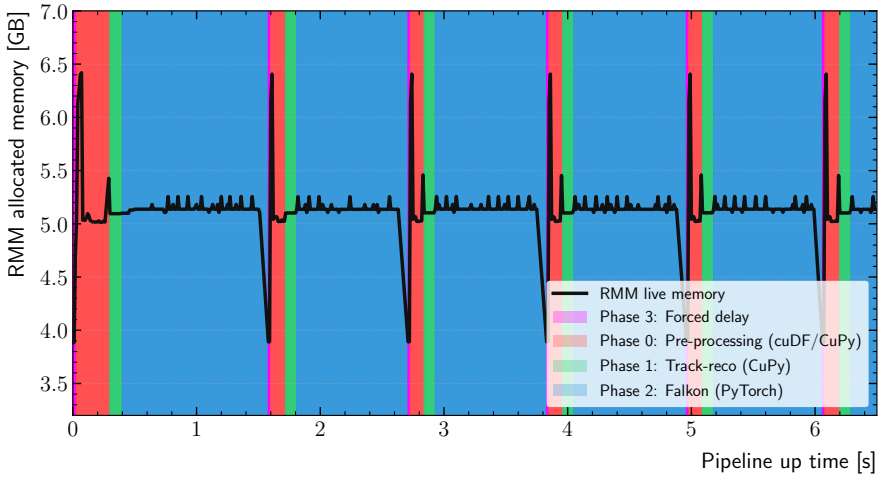


Figure 5.4: Pipeline time-trace of memory events with RMM enabled. The different colors encode the different states of the algorithm, allowing to trace memory changes linking them to their origin.

Let A_k be the allocated memory at cycle k. This is defined as the large memory swing that occurs just at the start of each red-band. We can write it as:

$$A_k = P_k - B_{k-1},$$

where B_{k-1} is the low point from which the spike starts and P_k is instead the high peak where the spike stops. Let then R_k be the released memory over a cycle k:

$$R_k = P_k - B_k$$

With these defitions we can operationally "integrate" over all the small allocations/de-allocations that are evident in the memory trace that occur during a cycle. What is important is what happens overall as if the cycle were a blackbox: the state of the memory before and after a cycle. Requiring that no net memory balance occurs is like asking that:

$$A_k = R_k$$

so all the memory that was allocated must also be released. In fact we have that:

$$A_k - R_k = P_k - B_{k-1} - P_k + B_k = B_k - B_{k-1} = \Delta B_k$$

And so one would require that $\Delta B_k = 0$. This condition is true on paper but in reality this is not necessarily true for each cycle for several reasons. On the one hand there are RMM and Python's own garbage collector that are not necessarily synchronized. On the other hand, the monitoring thread that was used to sample the memory can be slightly out of phase, making some measurement artifacts to appear.

What is more important operationally is the overall sum over N cycles:

$$\sum_k \Delta B_k = B_N - B_0$$

In fact if one finds that $\sum_k \Delta B_k \sim 0$, then it would mean that the occasional $\Delta B_k \neq 0$ are compensated over successive cycles. If instead a systematic drift was present, this condition could never hold: in this case in fact one would have that not all the memory could be released ($\Delta B_k > 0$) and subsequent accumulations of repeated cycles could result in the drift. What is expected then for a stable pipeline is that the $\sum_k \Delta B_k$ to be a quantity centered around 0 with confined fluctuations.

Regarding the peaks P_k If we consider the high-peaks instead, the situation is different. A peak is less representative than the base values B_k for the following reasons. During the pipeline execution, or in any application that is using RMM, it can happen that the shared memory pool is increased in its size. In the case of this pipeline for instance, the pool increases in a wide jump at the beginning of the operation after the initial runs / initializations of the various components. This of course is accompanied by a large peak that is not necessarily re-absorbed anytime soon if the pool is required to stay in that enlarged state, and so generally speaking $\sum_k \Delta P_k \neq 0$.

Both the hypothesis on $\sum_k \Delta B_k$ and on $\sum_k \Delta P_k$ have been studied and will be shown in the following.

Studying the behavior of the lower peaks B_k In order to test this hypothesis, many long runs have been conducted (each one of approximately 1 hour) during which the pipeline was performing reference-vs-data tests as it would in nominal operating conditions.

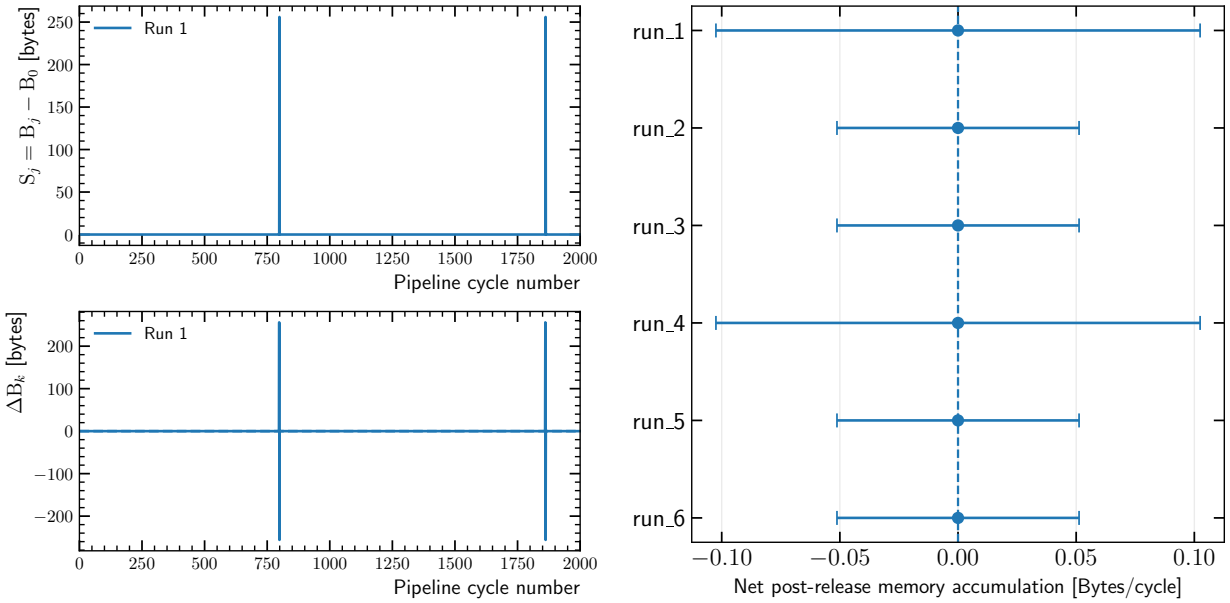


Figure 5.5: Stability analysis for the B_k. For practical reasons, only a small portion of the time-trace is shown.

In Figure 5.5 several results are reported. Starting from the left figure, the upper plot shows the following quantity:

$$S_j = \sum_{k=1}^{j} \Delta B_k = B_j - B_0,$$

where j is the current pipeline cycle number. It clearly shows, together with the bottom figure that instead displays ΔB_k for each cycle k, how sometimes $\Delta B_k \neq 0$ but also how S_j eventually returns and oscillates around 0. Further the bottom figure shows how these non zero increments occur in couples that compensate one another. These are extremely small with respect to the overall memory since their size is of 256 bytes. At the right of the figure a forest plot is shown. In this case, for each of the 6, 1-hour long runs taken with exactly the same settings, the net post-release memory accumulation is shown that is ΔB_k. These values and the error bars have been obtained via bootstrapping, so for this reason one has apparently "un-physical" sub-bytes allocations. The plots clearly suggest a contained behavior which supports the hypothesis.

More detailed results are reported also in Table 5.1. For each run, an explicit drift-estimate was obtained via a least-squares linear fit over the whole range. These results, within the studied time scale ($\approx$ 1 hr), support stability, not finding any evidence of a systematic drift. Further investigations on these are however left for future work.

run	$B_N - B_0$	$\sum \Delta B$	$\max \Delta B$ [bytes]	fit slope [MB/cycle]	fit p-value
1	0	0	256	-1.96×10^{-5}	0.997
2	0	0	256	-3.98×10^{-5}	0.239
3	0	0	256	$+1.07 \times 10^{-5}$	0.752
4	0	0	256	-4.15×10^{-5}	0.540
5	0	0	256	$+9.41 \times 10^{-5}$	0.650
6	0	0	256	$+8.41 \times 10^{-5}$	0.634

Table 5.1: Linear fits for each run to detect systematic drifts.

Studying the behavior of the upper peaks P_k For the upper peaks P_k, the following quantity was considered:

$$\Delta P_k = P_{k+1} - P_k$$

and while $\sum_k \Delta P_k$ is not necessarily 0 for the aforementioned reasons, it is still important to verify that the ΔP_k are well behaved, in the sense that they do not vary arbitrarily during a run. In other words it is requested that peak memory does not systematically increase with each cycle so if one considers again a linear fit:

$$P_k \simeq a + bk$$

the idea could be to estimate the slope to see whether it supports the hypothesis of a systematic increase or not. In order to do this, the following procedure was followed:

- the peaks P_k are extracted and the increments ΔP_k are computed.
- the increments are then bootstrapped via moving-block bootstrap. This was done to preserve part of the local temporal correlation structure of the measurements (for example, if the memory is slightly higher for a few consecutive cycles, a block bootstrap can preserve that local structure better than a completely random shuffle).
- the result of this operations is a sequence of synthetic sequence of increments ΔP_0^*, ΔP_1^*, ...
- for each synthetic sequence, a pseudo memory trace is considered:

$$P_k^* = P_0^* + \sum_{j=0}^{k-1} \Delta P_j^*, \qquad P_0^* = P_0$$

  so that each bootstrap iteration generates one possible pseudo peak-memory trace compatible with the observed increment structure.
- for each pseudo-trace a linear fit is performed so that $P_k^* \simeq a^* + b^* k$ and the slopes b^* are collected and finally displayed in a histogram.

The histogram of the bootstrapped slopes is therefore an empirical uncertainty distribution for the peak-memory drift and, since bootstrap cannot generate new physical information, it tells what trends are most supported by the information contained in the measured sequence.

The distributions for the bootstrapped peak-memory slopes are shown in Figure 5.6 for each of the six runs considered. The histograms show the resulting distributions of b^* in MB/cycle, while the dashed vertical line marks zero drift. The distributions are centered close to zero, indicating that the measured peak-memory fluctuations do not correspond to a systematic increase of the peak memory over pipeline cycles.

Table 5.2 reports more detailed quantities for each distribution. The table reports the median bootstrap slope, the central 95% bootstrap interval, and the fraction of bootstrap replicas with positive slope. All confidence intervals include zero, and $P(b^* > 0)$ is approximately 0.5 for all runs, indicating that the observed peak-memory fluctuations are not biased toward a systematic positive drift.

Finally the forest plot is shown in Figure 5.7 in order to compare the peak-memory stability across repeated runs. Each marker represents the central bootstrap estimate of b^*, while the horizontal bar

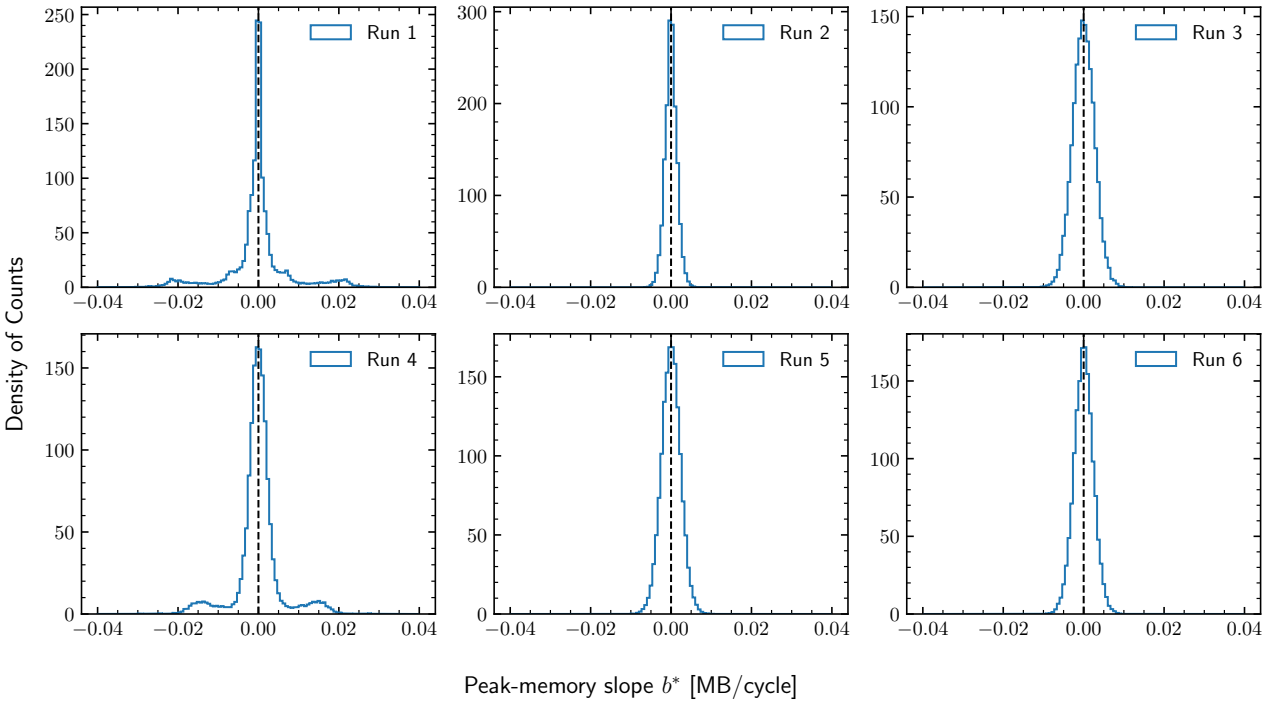


Figure 5.6: Bootstrap distributions of the peak-memory slopes.

Run	Median b^* [MB/cycle]	95% CI [MB/cycle]	$P(b^* > 0)$
Run 1	-4.59×10^{-4}	$[-2.05 \times 10^{-2}, 2.03 \times 10^{-2}]$	0.476
Run 2	-1.91×10^{-7}	$[-2.99 \times 10^{-3}, 2.94 \times 10^{-3}]$	0.497
Run 3	-3.02×10^{-5}	$[-5.42 \times 10^{-3}, 5.39 \times 10^{-3}]$	0.495
Run 4	-2.33×10^{-6}	$[-1.48 \times 10^{-2}, 1.49 \times 10^{-2}]$	0.498
Run 5	-1.47×10^{-6}	$[-4.74 \times 10^{-3}, 4.72 \times 10^{-3}]$	0.499
Run 6	1.74×10^{-6}	$[-4.78 \times 10^{-3}, 4.78 \times 10^{-3}]$	0.501

Table 5.2: Summary of the moving-block bootstrap distributions of the peak-memory drift.

shows the corresponding 95% bootstrap interval. The vertical line at $b^* = 0$ represents the case of no peak-memory drift.

One can see that all bootstrap intervals are compatible with zero drift, and the corresponding slope distributions are centered close to zero. This indicates that the peak memory fluctuates from cycle to cycle, but these fluctuations are not associated with a systematic increase of the peak working-set size. The result is therefore consistent with stable reuse of the RMM-managed memory pool during repeated pipeline execution.

Without RMM

When RMM is disabled, the post-release memory baseline (the B_k) no longer behaves as a stable pooled-memory reference level and one can no longer claim that:

$$\sum_k \Delta B_k \sim 0,$$

as it was the case previously.

An example run was conducted, with the same configuration of the already discussed RMM case but this time RMM was disabled. The resulting behavior of the baseline points B_k is shown in Figure 5.8a, compared to Figure 5.5. One can clearly spot that indeed the baseline is not stable anymore but, as expected, fluctuates since now there is no common allocator layer between each library and the driver.

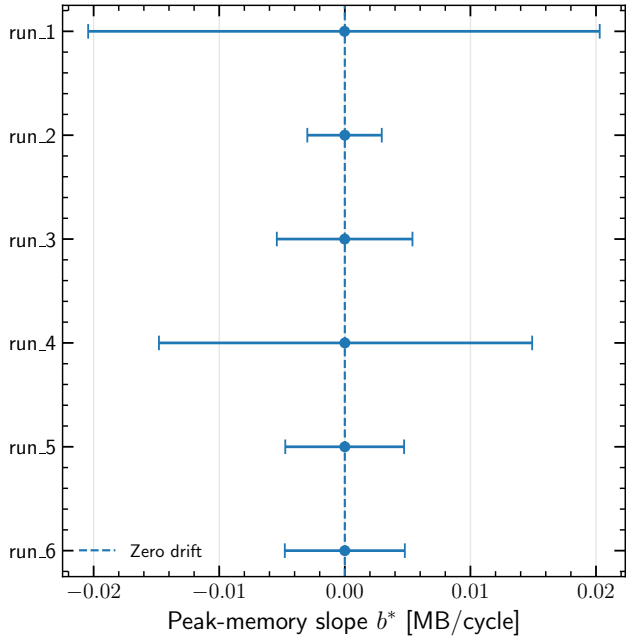
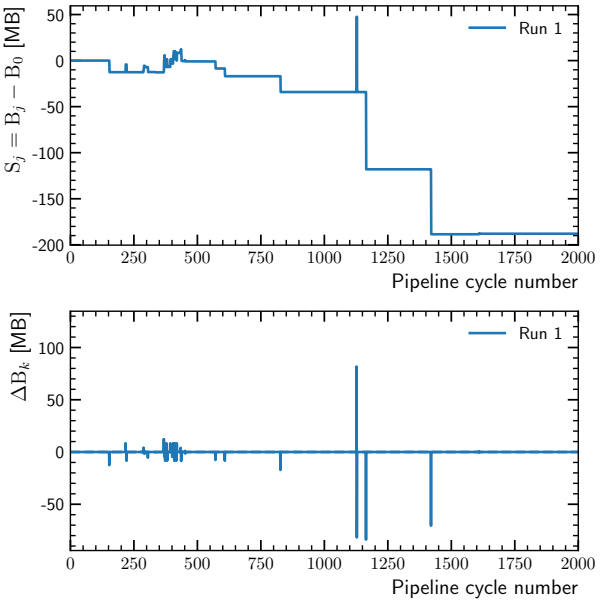
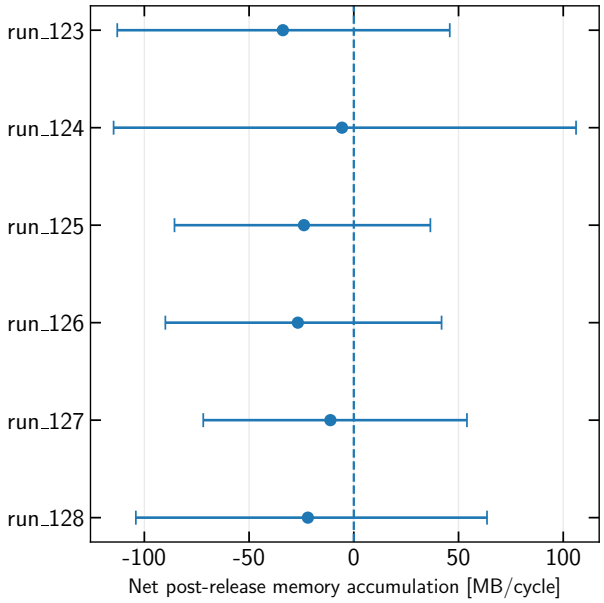


Figure 5.7: Forest plot of the bootstrap peak-memory drift.



(a) Cycle dependence of the low memory points B_k.



(b) Net post-release memory accumulation.

Further, figure Figure 5.8b shows how, on average, the net post-release memory accumulation is now far from exhibiting the same behavior it showed in Figure 5.5. The same analysis that was done earlier for the peaks was repeated again for this case.

The resulting distributions are shown in Figure 5.9. A direct comparison to Figure 5.6 show how this time the distributions are much broader but they suggest the same behavior: the absence of a systematic increase of the peak memory over pipeline cycles.

This together with the data reported in Table 5.3 show no evidence of a net drift. Yet, the broader distributions now that RMM is disabled could be indicative of a larger cycle-to-cycle variability of peak memory, as it was expected since, without a unified allocator and memory resource, the independent allocation policies of each GPU library play a major role. Also, the large allocations and de-allocations have a meaningful impact on performances as it was already pointed out earlier and will again be shown in the following discussion. Following the same arguments as before, the forest plot for the no-RMM

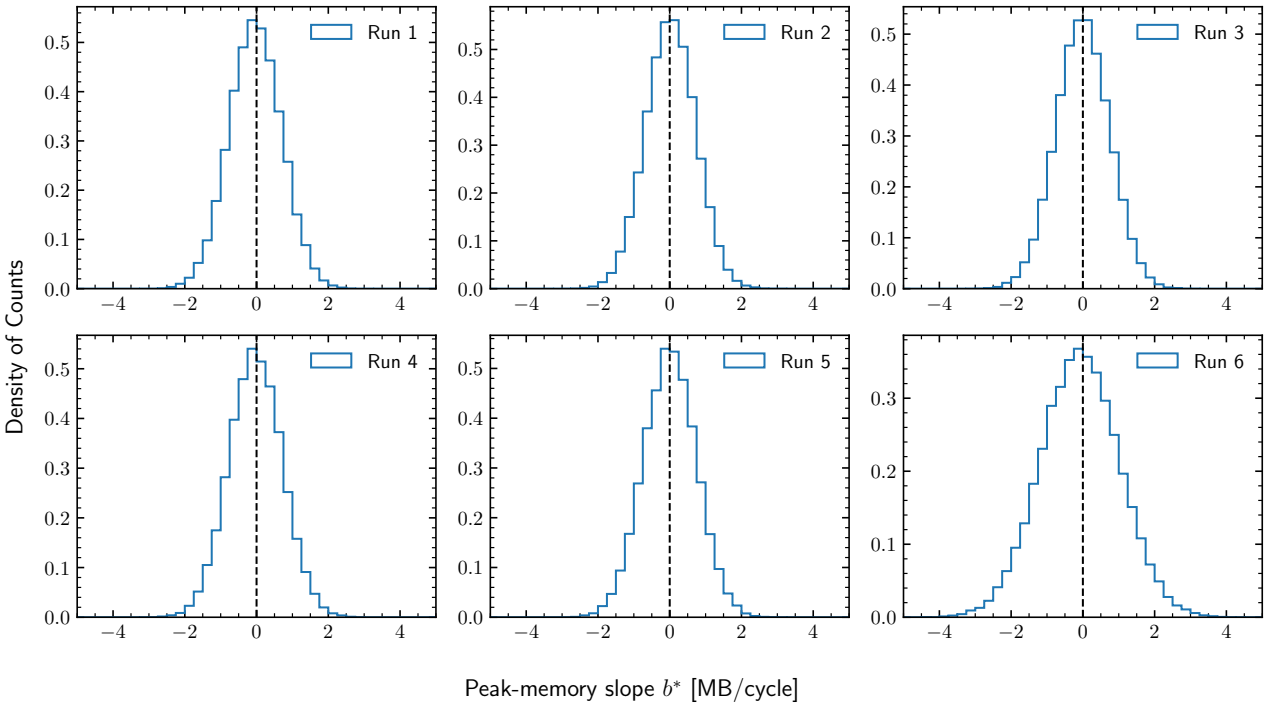


Figure 5.9: Bootstrap distribution of the peak-memory slopes.

Run	Median b^* [MB/cycle]	95% CI [MB/cycle]	$P(b^* > 0)$
Run 1	-3.95×10^{-2}	[−1.46, 1.39]	0.479
Run 2	-2.89×10^{-2}	[−1.33, 1.38]	0.516
Run 3	-0.76×10^{-2}	[−1.47, 1.44]	0.490
Run 4	-2.98×10^{-2}	[−1.46, 1.44]	0.478
Run 5	0.36×10^{-2}	[−1.44, 1.44]	0.503
Run 6	-8.42×10^{-2}	[−2.22, 2.06]	0.470

Table 5.3: Summary of the moving-block bootstrap distributions of the peak-memory drift in the absence of RMM.

case is shown in Figure 5.10.

This again allows to compare the peak-memory stability across repeated runs. Each marker represents the central bootstrap estimate of b^*, while the horizontal bar shows the corresponding 95% bootstrap interval. The vertical line at $b^* = 0$ represents the case of no peak-memory drift. One can see that also in this case (compared to Figure 5.7) all bootstrap intervals are compatible with zero drift, and the corresponding slope distributions are centered close to zero. The fluctuations are much broader then before but they are not associated with a systematic increase of the peak working-set size.

In conclusion, while the RMM-backed configuration exhibits a stable baseline compatible with memory-pool reuse, the non-RMM configuration shows discrete baseline changes and a non-zero endpoint balance. The peak-memory analysis does not indicate a monotonic leak, but it does show that the non-RMM memory behavior is less stationary and perhaps more difficult to predict.

Concluding remarks

In light of this study, the RMM configuration provides a cleaner and more stable memory baseline for long-running multi-library GPU workloads. In the context of this pipeline, where cuDF, CuPy, and PyTorch/Falkon are used in the same processing chain, this is important not only for reducing allocation overhead and increasing performance, but also for making the memory footprint easier to control, monitor, and interpret during online operation, which is a desirable property for long-running

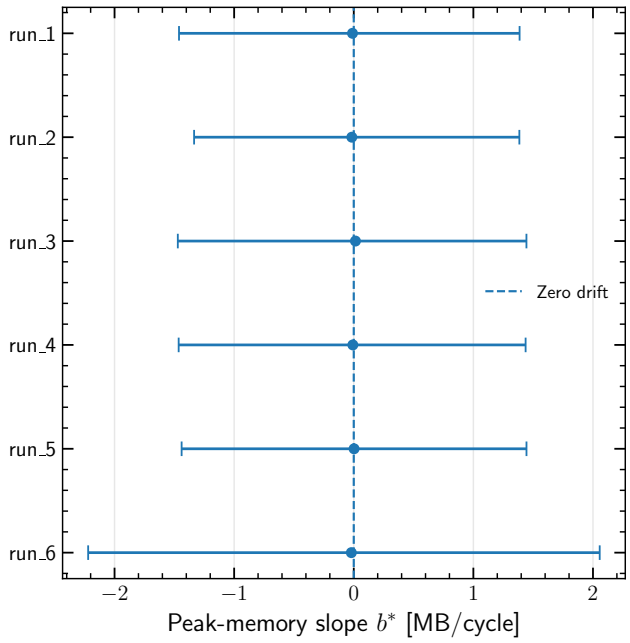


Figure 5.10: Forest plot of the bootstrap peak-memory drift.

DQM applications.

5.1.4 Monitoring dashboard

In order to demonstrate and test the functioning of the pipeline, a lightweight dashboard application was developed. The application consists in a backend Python/Flask application that communicates with the pipeline process via a Redis-based messaging system. The backend then publishes the received data to an HTML/Javascript based web interface via socketIO events. The dashboard allows also to control the pipeline, choosing for instance to inject altered data to show how the pipeline reacts to the anomalous batches. More details are given in Appendix C.

5.2 Latency and online processing

In the following the online processing operating envelope of the pipeline is described within the specific hardware implementation.

5.2.1 Latency and input rate

The total latency t_{lat} is the amount of time that the pipeline requires to output a test statistic/p-value and anomaly label given an input batch. When one looks at it from the perspective of the Falkon/DQM stage, it can mainly be written with two components: an input latency due to the first two stages (pre-processing and track-reconstruction in order to build the input batch for Falkon) and an output latency due to Falkon itself (the time it takes for it to process the input batch and output the corresponding p-value). While the former depends on the size of the input batch, the latter instead depends largely on the model complexity. The smaller t_{lat} is and the faster the pipeline can respond thus yielding greater throughput. Also there is a minimum value $\bar{t}_{\text{lat}}$ of it that acts as a floor value which depends on the algorithm's design.

One can write:

$$t_{\text{lat}} = [t_{\text{pre}} + t_{\text{reco}}] + t_{\text{F}}, \tag{5.1}$$

where t_{pre} is the time required by the pre-processor block to build the candidate dataset to be passed over to the track reconstruction algorithm which in turn requires a time t_{reco} to build the final input

dataset for Falkon. Finally t_{F} is the output latency, that is the time required for the test to be computed by Falkon.

The latency must be compatible with the input rate, meaning that if the source produces hits at a rate R_{in} and the pipeline input batch at the pre-processor input is N, then the time required to accumulate sufficient amount of data is N/R_{in}. So the latency must be such that:

$$t_{\mathrm{lat}} \lesssim \frac{N}{R_{\mathrm{in}}},$$

in order to for the pipeline to remain synchronized with the source and not introducing delays that would result in input batch losses.

Example Let $N = 40M$ hits and $R_{\mathrm{in}} = 100$Mhits/s, then $t_{\mathrm{lat}} \lesssim 0.4$s. Looking at Figure 4.54a one can see that a input batch with $N = 40M$ is pre-processed and reconstructed in < 0.2s and that Falkon requires a $t_{\mathrm{F}} \lesssim 90$ms with the $N_{\mathcal{D}} = 1000$ model to produce the test-statistic/p-value having a total latency then of $t_{\mathrm{lat}} \lesssim 0.3$s which satisfies the input requirement.

Note on the input batch size When the input batch N is chosen much smaller than the input rate, the upper requirement for t_{lat} becomes tighter. Since the pipeline algorithm cannot have an arbitrarily small latency but will eventually hit a floor $\bar{t}_{\mathrm{lat}}$ as previously anticipated, it is useful to quantify it in order to properly define the operating margins. This is the concept of minimum latency.

5.2.2 Minimum latency

Pre-processor minimum t_{pre}

When one measures t_{pre} for very small raw-input batches sizes (N ranging from 500 to 1×10^6 raw hits), an almost flat value is found for t_{pre} that fluctuates around:

$$\bar{t}_{\mathrm{pre}} = 89.37^{+4.47}_{-1.86}\mathrm{ms}. \tag{5.2}$$

Within the present implementation, this value thus represent the minimum latency of the pre-processor algorithm so that it is not possible to produce a dataset of candidate events in less time than the one shown in Equation (5.2).

Track-reconstruction minimum t_{reco}

For t_{reco} instead, the GPU track-reconstruction algorithm requires a fixed time to build each track that depends on the *aperture* (the number of input processing slots for the GPU algorithm where each slot gets populated with a candidate event to reconstruct) and the saturation of the total number of processing slots. As it was previously shown in Section 4.2, the GPU reconstruction algorithm is a chain of kernels. They process the candidates being generated by the pre-processor in **sub-batches** of fixed size (the aperture), and accumulate the results in the reconstructed dataset that is later passed to the DQM stage.

For instance if one has that the pre-processor furnishes 12345 candidates and the aperture is set at 4096, the number of sub-batches is 4. The kernel chain will then execute 4 times in order to process all the candidates. In this case one has 3 sub-batches completely filled with 4096 candidates each, and the last one instead has only 57 filled candidates, leaving the others empty. The ratio between the number of filled candidates in the last sub-batch and the aperture is termed the **saturation**:

$$\mathrm{SAT} = \frac{N_{\mathrm{in}}}{N_{\mathrm{L}} \times AP}, \qquad N_{\mathrm{L}} = \left\lceil \frac{N_{\mathrm{in}}}{AP} \right\rceil, \tag{5.3}$$

where N_{in} is the number of input candidates generated by the pre-processor, N_{L} is the number of sub-batches (i.e. kernel launches) and AP is the aperture. This factor has an impact on the latency of

the track-reconstruction block since the GPU algorithm, in order to reduce execution divergence, still performs some initial processing instructions even on the empty candidates. Since these will yield no tracks, this processing time does not correspond to more output tracks and is effectively wasted. This increases the median time required to process the input candidates with respect to a situation where instead no time is wasted because the GPU is fully saturated.

Aperture	Median $\bar{t}_{\text{reco}}$ [μs]
256	$2.11^{+0.03}_{-0.02}$
512	$2.01^{+0.01}_{-0.01}$
1024	$1.97^{+0.04}_{-0.02}$
2048	$1.93^{+0.02}_{-0.01}$
4096	$1.84^{+0.03}_{-0.01}$

Table 5.4: Minimal median times to reconstruct a single track as a function of the aperture when the last sub-batch is fully saturated.

This is exemplified in Table 5.4. The table shows the ideal situation in which the last sub-batch is fully filled, that is for instance if the pre-processor was to produce exactly an integer multiple of the aperture, like 8192 candidate events, then by setting the aperture at 4096, with 2 kernel chain launches one can fill each time the whole 4096 input slots obtaining a reconstruction time of $\approx 1.84\mu$s per track. This is however the ideal situation and it represents the *minimal reconstruction latencies* per track.

Realistic example based on this work Considering for instance a small batch of $1M$ hits at the input which, with the datasets considered in this work, corresponds to roughly 1200 candidate events, one has the results shown in Table 5.5.

Aperture	Median t_{reco} [μs]	Last sub-batch saturation [%]
256	$2.22^{+0.04}_{-0.02}$	95.00
512	$2.54^{+0.02}_{-0.02}$	79.17
1024	$3.32^{+0.07}_{-0.03}$	59.38
2048	$3.30^{+0.03}_{-0.01}$	59.38
4096	$6.55^{+0.01}_{-0.03}$	29.69

Table 5.5: Median times to reconstruct a single track as a function of the aperture for partially saturated last sub-batches.

The less saturated the last sub-batches are and the larger the departure from the minimal latency. In order to achieve the minimum possible latency at the reconstruction stage, one has to tune the aperture (AP) with respect to the expected number (N_{in}) of candidate events that the pre-processor generates per input batch when the data is taken at nominal conditions. The aperture should be chosen so as to maximize the saturation defined in Equation (5.3).

In conclusion, one can have a reconstruction latency that is floored, at each value of the aperture, by the ideal fully saturated setup and by looking at Table 5.4 one can see that this value is in the order of $\approx 2\mu$s per track.

Falkon DQM minimum t_{F}

In the case of Falkon the latency required to output a decision (process the input from the track-reconstruction stage and produce the p-value) is heavily dependent on the complexity of the model. For instance in the case of this work, t_{F} was shown earlier in Table 4.15 for the selected `fp64` models and in Table 4.16 for the `fp32` models. In a numerically stable model, Falkon always needs roughly the same number of iterations in order to converge to the solution. For this reason in this case it is not entirely correct to speak of a minimum time but rather of an almost constant time given a particular model.

5.2.3 Final remarks on the online capabilities of the pipeline

To better showcase the pipeline capabilities, some example runs with varying input batch sizes at a fixed GPU aperture and Falkon model have been conducted. When one has the total latency of the pipeline, also named the end-to-end latency, that measures the required time to obtain the final p-value starting from the raw-hits input batch at the pipeline input, it is possible to define the output throughput as:

$$R_{\text{out}}(N) = \frac{1}{t_{\text{lat}}(N)}.$$

where N is the size of the raw-hits input batch. Thus R_{out} is the number of end-to-end processing jobs that the pipeline can execute in the unit of time which equals the number of raw-hits input batches that can be processed in the unit of time because of the synchronous design that was enforced in this implementation[1]. For this reason, it is possible to also define the maximum acceptable input rate in terms of how many raw hits can be processed at the input:

$$R_{\text{in}} = N \times R_{\text{out}},$$

where N is the raw-hits input batch size.

Batch Size [MHits]	Sat.	t_{lat} [ms]	Max. R_{in} [Mhits/s]
2	59.59%	$176.99^{+4.54}_{-3.55}$	$11.30^{+0.23}_{-0.28}$
10	76.44%	$225.53^{+3.17}_{-3.92}$	$45.58^{+0.83}_{-0.65}$
50	99.60%	$290.81^{+2.21}_{-4.66}$	$172.37^{+2.82}_{-1.30}$
100	99.51%	$399.95^{+2.13}_{-4.69}$	$250.09^{+2.97}_{-1.33}$
200	99.16%	$628.07^{+3.97}_{-4.30}$	$321.67^{+2.24}_{-2.04}$
300	99.67%	$841.38^{+5.29}_{-4.14}$	$356.56^{+1.76}_{-2.23}$

Table 5.6: Total latency t_{lat} as described in Equation (5.1) and maximum acceptable input rate R_{in} for various input batch sizes with GPU aperture set at 4096 and selected Falkon model with $N_{\mathcal{D}} = 1000$ and active RMM.

An example of this is shown in Table 5.6. One can see how the pipeline is able to cope with a wide spectrum of raw-hits input rates R_{in} from the source. As the raw-hits input batch size increases, the maximum acceptable input rate tends to saturate as it is shown in Figure 5.11.

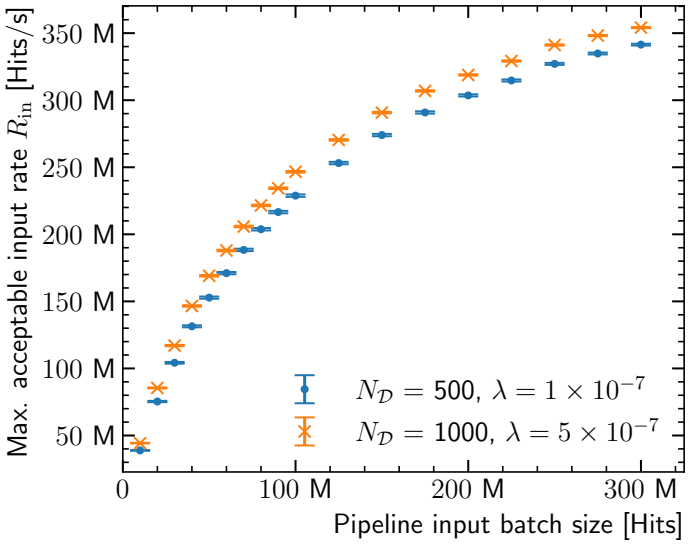


Figure 5.11: Maximum acceptable input rate R_{in} as a function of the input batch size.

[1] As previously mentioned, synchronous refers to the fact that the output p-value is strictly associated to the raw-hits input batch (or equivalently to the set of reconstructed-events corresponding to it). No accumulation of input batches occurs, so that the output of the pipeline is synchronous with respect to the input.

This figure is the "reciprocal" of Figure 4.55a and is re-casted in this form for better clarity. The figure shows, for each raw-hits input batch size to the pipeline the highest input hit-rate that it can manage. For instance for a raw-input batch size of $N = 10$Mhits, the system takes a time $t_{\text{lat}} = 225.53^{+3.17}_{-3.92}$ms to produce the p-value. This means it can run $1/t_{\text{lat}} \approx 4.4$ tests each second or, equivalently, process 4.4 raw-hits input batches in the unit of time, corresponding to a maximum acceptable input rate of $\approx$ 45Mhits/s. If the hit source emits at any rate smaller than this, then the pipeline can keep up otherwise it will be impossible because the time it takes to produce the decision is longer then the time the source needs to produce the raw-hits in sufficient amount. For instance if the source emits at 30Mhits/s, with the raw-hits input batch size of 10M, each second the pipeline needs to process 3 input batches and this is entirely compatible since it can process up to 4.4.

The relevant result is that all of these input rates are much larger than the ones that are observed with cosmic-ray muons as in the case of this work. In fact by the simple and ideal argument presented in Section 3.2.2, one can show that the hit rate with cosmic muons with this setup is $\lesssim$ 137 hits/s. This is nevertheless interesting since it opens a wide variety of possible applications to systems where the input hit rate is expected to be much larger (for instance at a detector at a particle collider).

Chapter 6

Conclusions and Outlook

6.1 Main results

This work has involved building of a multi-library GPU-resident data processing pipeline for a DQM application at a particle detector prototype, proving its feasibility. The efficient coexistence and cooperation between the different libraries has been made possible by the usage of DLpack and the Rapids Memory Manager (RMM). While DLpack enabled the smooth passage of data through the pipeline from a library to the next, RMM provided a common memory allocator and an unified memory pool. The memory stability study confirmed the usefulness of a unified RMM-backed allocation strategy for the GPU-resident DQM pipeline since it increases raw performances and also provides a cleaner and more stable memory baseline that is easier to control, monitor, and interpret during online operation.

This work has built upon the previous existing implementation [10], integrating the different blocks in a coherent pipeline. In this respect, the pre-processing block has been redesigned to ensure greater performance, now fully exploiting cuDF capabilities. The track reconstruction layer has been designed from scratch, involving the designing and writing of a series of CUDA kernels that are launched via CuPy. These kernels embody a customized track reconstruction algorithm that mirrors the physical geometry of the detector itself within the GPU execution model, leveraging on warp-level operations. The anomaly detection / DQM layer implements the NPLM method through FalkonML which in turn is based upon PyTorch. This last layer required for small yet general extension of the source code of FalkonML, allowing it to work with the RMM allocator since Torch does not expose by itself the full allocator interface when one uses a different allocator than its own native one.

The fundamental result is that the pipeline was shown to be able to cope with an ample range of input data rates, showcasing an ability to scale well beyond the typical input characteristics of the experimental setup due to the cosmic muons acting as the source. With respect to the anomaly detection task, it has shown great sensitivity to the injected anomalies. This has made it possible to also experiment with the classification of the anomalies by means of the multi-class classifiers previously trained on the set of known anomaly cases. These showed a $\approx 96.8\%$ accuracy during tests, thus yielding promising results. The final results are recalled in Table 6.1.

6.2 Limitations of the present study

With respect to the implementation that was described in this study, a limitation consists in the fact that it was conducted on a specific GPU device (NVIDIA A100) and all of the measured quantities are referred to this. The relevant results in this respect that could act as a baseline for future benchmarks and comparisons are surely the measures of end-to-end latency and throughput of the pipeline as a whole since, when seen from the outside as a "black box", the most important feature is that the system is able to deliver the DQM decisions (in this case the anomaly detection via p-value computations) in a

Quantity	Result	Interpretation
End-to-end monitoring latency	$\lesssim 0.9$s (300M hits case)	Time from batch accumulation to DQM output
Output throughput	up to 4.4Hz (5.8Hz in `fp32`)	Number of DQM statistics produced per second
Input throughput	$\lesssim 350$ Mhits/s (300M hits case)	Maximum sustained input rate
Anomaly-detection sensitivity	all tested anomalous batches were rejected	Anomalous batches rejected with small p-values
Classification accuracy	96.8%	Accuracy on known injected anomaly classes
Classifier latency	$\lesssim 90\mu$s	Overhead of the diagnostic classification layer
Memory pool size	up to 30GB (300M hits case)	Size of the RMM allocated memory

Table 6.1: Summary of the main performance and anomaly-detection results obtained in the tested hardware and software configuration.

timely fashion. In this respect, it would be useful to explore how a different GPU device / architecture could have an impact on this figure and the others that were reported earlier.

Another limitation regards instead the exploration of the anomaly-detection sensitivity of the system. In fact, within the available data for the anomalous batches, it was found that all of them were hardly rejected by the test. In this respect it would be useful to obtain more data collected under degraded operational conditions that could prove more challenging for the system, effectively allowing a more in-depth study of the sensitivity.

6.3 Outlook

As future directions, it could be interesting to further explore the ability of the pipeline to scale up to more challenging experimental situations either with simulations or real deployments at full-scale particle detectors, perhaps also on multi-GPU environments. This could also allow for the extensions of the RMM memory stability study to longer periods of real data processing. Further, it should prove interesting to deepen the study on the methodologies to classify the anomalies that have been explored in this work. Another theme that surely deserves further investigation is the GPU track reconstruction algorithm, its relation to more conventional methods, its detector hardware mirroring ability and its scalability to larger and multi-chamber detectors, combining the information from the reconstructed segments of each chamber.

Bibliography

[1] S Chatrchyan et al. "The CMS experiment at the CERN LHC". In: *JINST* 3 (2008). Also published by CERN Geneva in 2010, S08004. DOI: 10.1088/1748-0221/3/08/S08004. URL: https://cds.cern.ch/record/1129810.

[2] Gaia Grosso et al. *Fast kernel methods for Data Quality Monitoring as a goodness-of-fit test.* 2023. arXiv: 2303.05413 [hep-ex]. URL: https://arxiv.org/abs/2303.05413.

[3] Giacomo Meanti et al. *Kernel methods through the roof: handling billions of points efficiently.* 2020. arXiv: 2006.10350 [cs.LG]. URL: https://arxiv.org/abs/2006.10350.

[4] Virginia Azzolini et al. "The Data Quality Monitoring software for the CMS experiment at the LHC: past, present and future". In: *EPJ Web Conf.* 214 (2019), p. 02003. DOI: 10.1051/epjconf/201921402003. URL: https://cds.cern.ch/record/2701776.

[5] L. Tuura et al. "CMS data quality monitoring: Systems and experiences". In: *J. Phys. Conf. Ser.* 219 (2010). Ed. by Jan Gruntorad and Milos Lokajicek, p. 072020. DOI: 10.1088/1742-6596/219/7/072020.

[6] Andrew Brinkerhoff et al. "Anomaly Detection for Automated Data Quality Monitoring in the CMS Detector". In: *EPJ Research Infrastructures* 10.1 (Feb. 2026). ISSN: 3091-4566. DOI: 10.1007/s41781-025-00147-2. URL: http://dx.doi.org/10.1007/s41781-025-00147-2.

[7] Laura Borrello. *The Data Quality Monitoring Software for the CMS experiment at the LHC.* Tech. rep. Geneva: CERN, 2016. DOI: 10.1109/NSSMIC.2014.7431053. URL: https://cds.cern.ch/record/2121269.

[8] Raffaele Tito D'Agnolo and Andrea Wulzer. "Learning new physics from a machine". In: *Physical Review D* 99.1 (Jan. 2019). ISSN: 2470-0029. DOI: 10.1103/physrevd.99.015014. URL: http://dx.doi.org/10.1103/PhysRevD.99.015014.

[9] Marco Letizia et al. "Learning new physics efficiently with nonparametric methods". In: *The European Physical Journal C* 82.10 (Oct. 2022). ISSN: 1434-6052. DOI: 10.1140/epjc/s10052-022-10830-y. URL: http://dx.doi.org/10.1140/epjc/s10052-022-10830-y.

[10] Gaia Grosso et al. *Triggerless data acquisition pipeline for Machine Learning based statistical anomaly detection.* 2023. arXiv: 2311.02038 [hep-ex]. URL: https://arxiv.org/abs/2311.02038.

[11] Gaia Grosso. "Searching for Unexpected New Physics at the LHC with Machine Learning". Doctoral Thesis. Padova, Italy: Università degli Studi di Padova, 2023. URL: https://hdl.handle.net/11577/3490020.

[12] Marco Letizia. "The New Physics Learning Machine: Machine Learning for Goodness-of-Fit via Neyman–Pearson Testing". Presentation slides. ML4Jets2023 (DESY, Hamburg, Germany). Nov. 9, 2023. URL: https://indico.cern.ch/event/1253794/contributions/5588590/attachments/2749113/4784501/MarcoLetizia_NPLM_ML4Jets.pdf (visited on 08/13/2026).

[13] Raffaele Tito D'Agnolo and Andrea Wulzer. "Learning new physics from a machine". In: *Physical Review D* 99.1 (Jan. 2019). ISSN: 2470-0029. DOI: 10.1103/physrevd.99.015014. URL: http://dx.doi.org/10.1103/PhysRevD.99.015014.

[14] Alessandro Rudi, Raffaello Camoriano, and Lorenzo Rosasco. *Less is More: Nyström Computational Regularization.* 2016. arXiv: 1507.04717 [stat.ML]. URL: https://arxiv.org/abs/1507.04717.

[15] CMS Collaboration. *Muon Drift Tubes.* CERN. URL: https://cms.cern/detector/detecting-muons/muon-drift-tubes (visited on 08/19/2026).

[16] CMS Collaboration. "Calibration of the CMS drift tube chambers and measurement of the drift velocity with cosmic rays". In: *Journal of Instrumentation* 5.03 (Mar. 2010), T03016–T03016. ISSN: 1748-0221. DOI: 10.1088/1748-0221/5/03/t03016. URL: http://dx.doi.org/10.1088/1748-0221/5/03/T03016.

[17] NVIDIA Corporation. *GPUDirect RDMA*. CUDA Toolkit Documentation. URL: https://docs.nvidia.com/cuda/gpudirect-rdma/ (visited on 08/20/2026).

[18] RAPIDS Development Team. *RAPIDS: Libraries for End to End GPU Data Science*. 2023. URL: https://rapids.ai.

[19] Ryosuke Okuta et al. "CuPy: A NumPy-Compatible Library for NVIDIA GPU Calculations". In: *Proceedings of Workshop on Machine Learning Systems (LearningSys) in The Thirty-first Annual Conference on Neural Information Processing Systems (NIPS)*. 2017. URL: http://learningsys.org/nips17/assets/papers/paper_16.pdf.

[20] DMLC. *DLPack: Open In-Memory Tensor Structure*. https://github.com/dmlc/dlpack. Accessed: 2026-08-17.

[21] NVIDIA Developer Blog. *Fast, Flexible Allocation for NVIDIA CUDA with RAPIDS Memory Manager*. https://developer.nvidia.com/blog/fast-flexible-allocation-for-cuda-with-rapids-memory-manager/. Accessed: 2026-08-17.

[22] NVIDIA Corporation. *CUDA C++ Programming Guide*. CUDA Toolkit Documentation. NVIDIA Corporation. 2026.

[23] Tianqi Chen et al. *xgboost: Extreme Gradient Boosting*. R package version 3.5.0.0. 2026. URL: https://github.com/dmlc/xgboost.

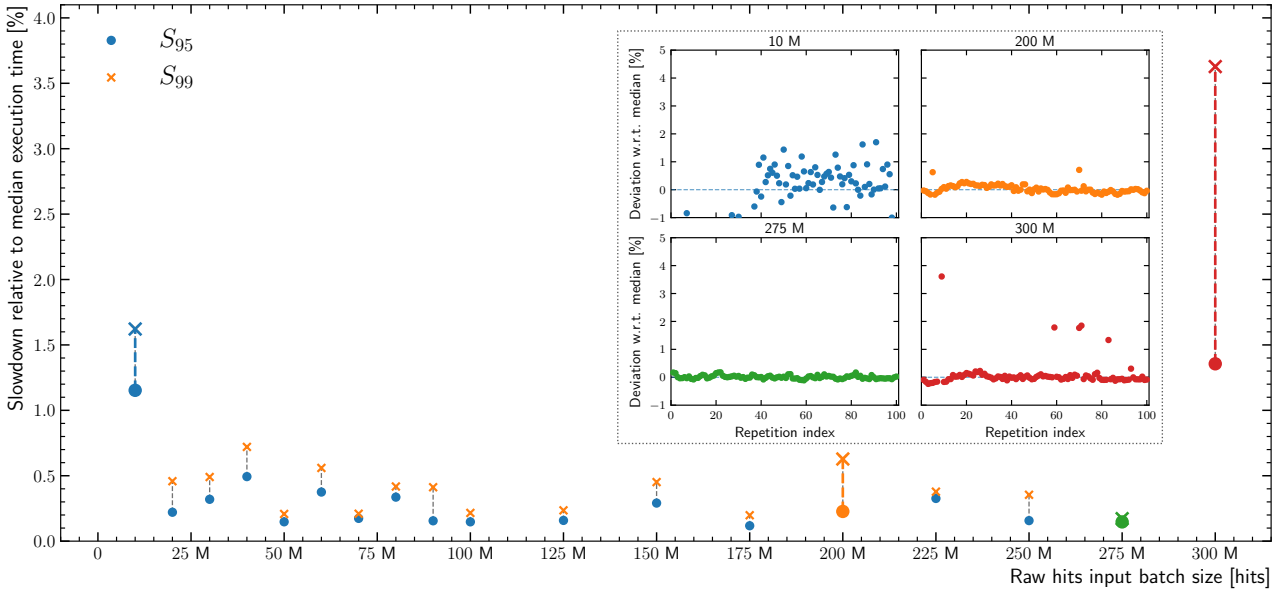
[24] Adam Paszke et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019.

[25] NVIDIA. *CUDA C++ Best Practices Guide*. https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/. Accessed: 2026-08-17.

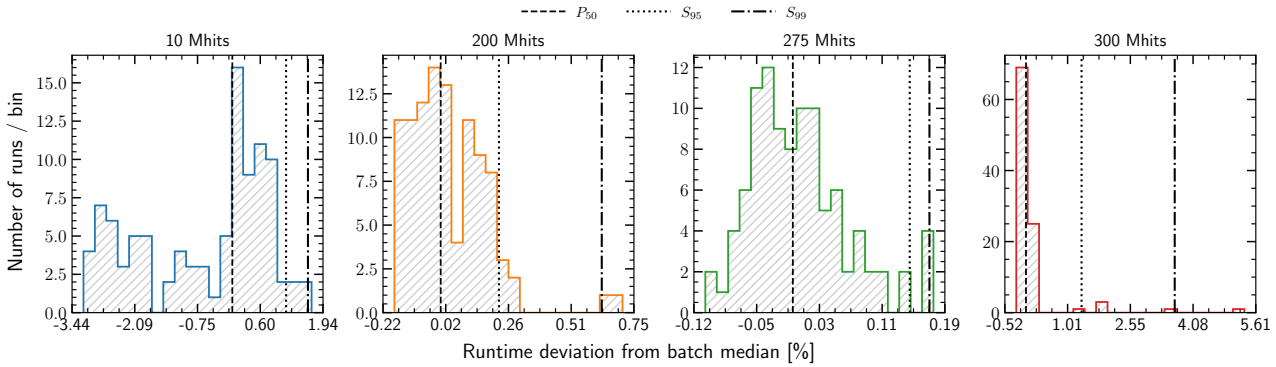
Appendix A

Additional material

A.1 Slowdown for pre-processing and track-reconstruction



(a) Upper-tail timing behavior of the GPU preprocessing and track reconstruction stages. For each input batch size, the markers indicate the relative slowdowns S_{95} and S_{99} of the 95th and 99th execution-time percentiles with respect to the median execution time. Dashed vertical segments connect the two quantities obtained from the same batch, so that their length represents the extension of the extreme timing tail, $S_{99} - S_{95}$.

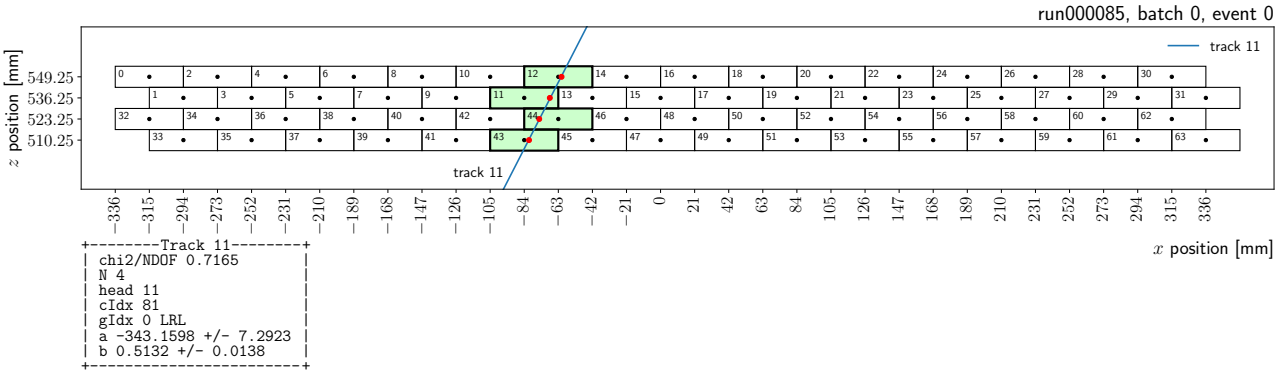


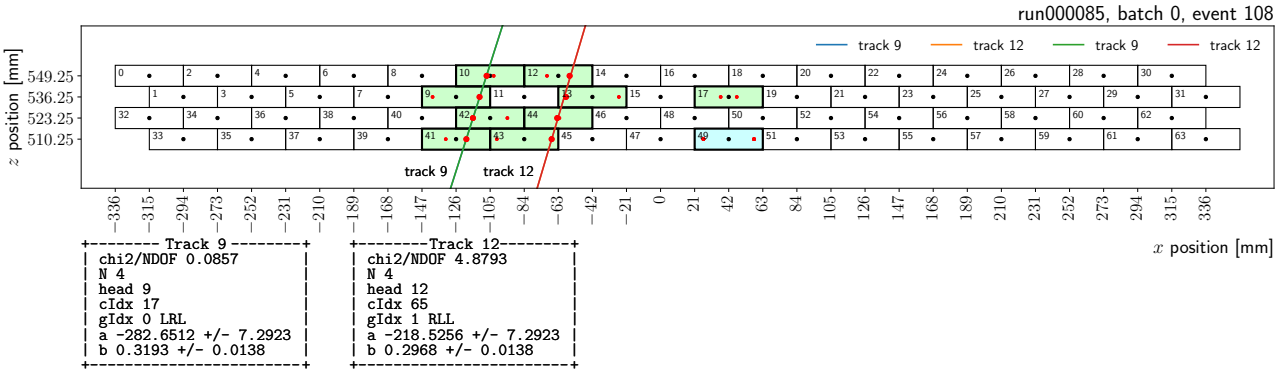
(b) Distributions of the preprocessing and track reconstruction stages runtime fluctuations for four representative input batch sizes: 10, 200, 275 and 300 million raw hits. For each batch size, the runtime measurements are expressed as the percentage deviation from the corresponding median, $\delta t_i = 100(t_i/P_{50}(t) - 1)$. Each distribution contains 100 timing measurements.

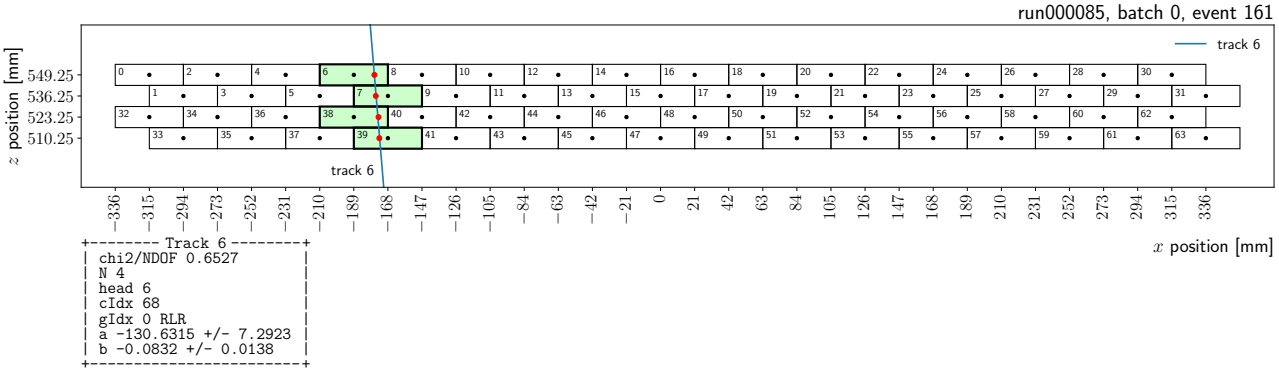
Figure A.1: Slowdowns for preprocessing and track reconstruction.

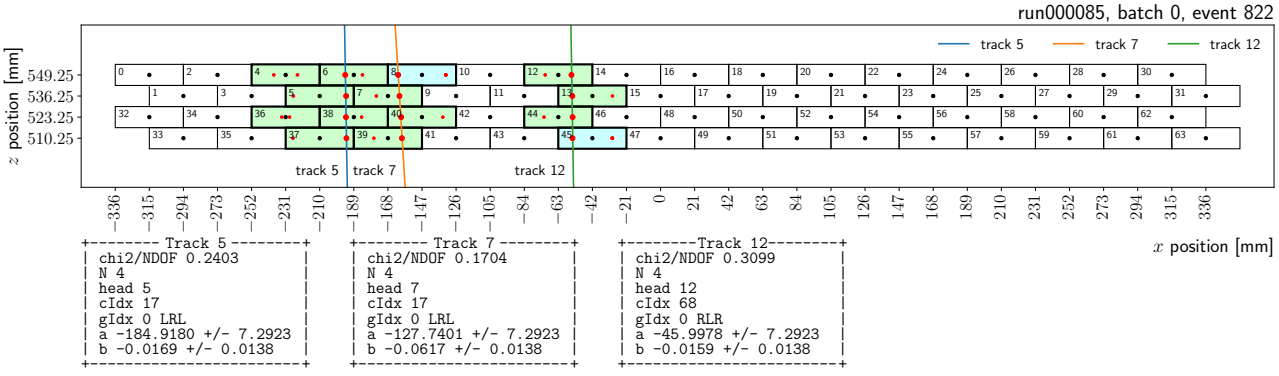
A.2 Extra fit results

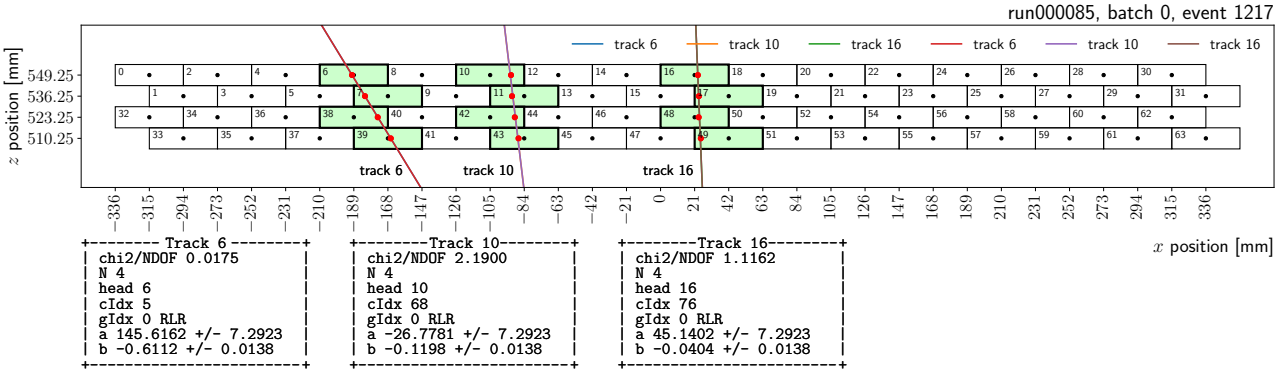
This appendix gathers some reconstructed events from the reference dataset.







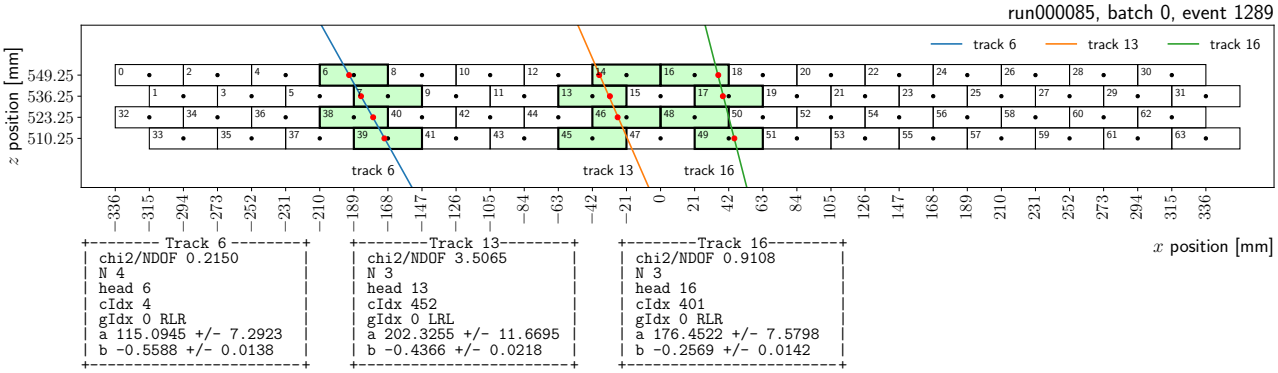





```
+-------- Track 6 --------+
| chi2/NDOF 0.0175       |
| N 4                    |
| head 6                 |
| cIdx 5                 |
| gIdx 0 RLR             |
| a 145.6162 +/- 7.2923  |
| b -0.6112 +/- 0.0138   |
+------------------------+
```

```
+-------Track 10--------+
| chi2/NDOF 2.1900       |
| N 4                    |
| head 10                |
| cIdx 68                |
| gIdx 0 RLR             |
| a -26.7781 +/- 7.2923  |
| b -0.1198 +/- 0.0138   |
+------------------------+
```

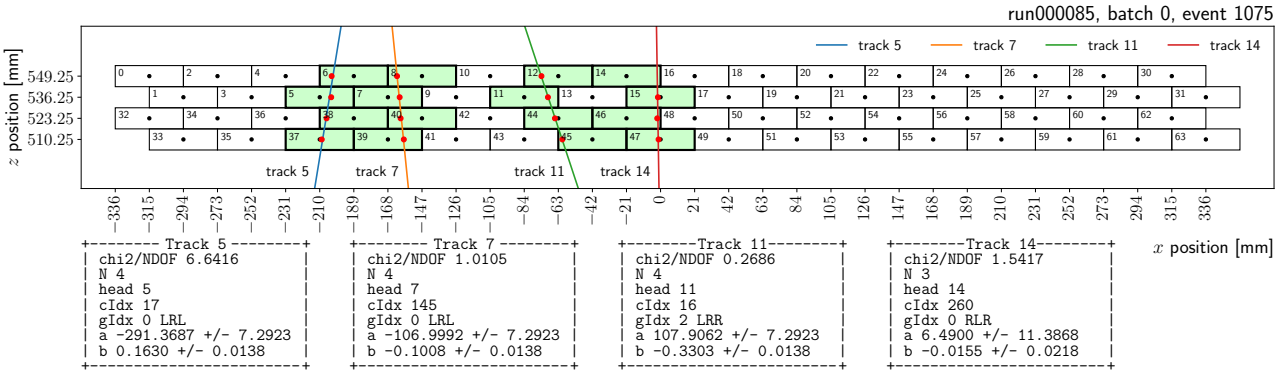
```
+-------Track 16--------+
| chi2/NDOF 1.1162       |
| N 4                    |
| head 16                |
| cIdx 76                |
| gIdx 0 RLR             |
| a 45.1402 +/- 7.2923   |
| b -0.0404 +/- 0.0138   |
+------------------------+
```




```
+-------- Track 6 --------+
| chi2/NDOF 0.2150       |
| N 4                    |
| head 6                 |
| cIdx 4                 |
| gIdx 0 RLR             |
| a 115.0945 +/- 7.2923  |
| b -0.5588 +/- 0.0138   |
+------------------------+
```

```
+-------Track 13--------+
| chi2/NDOF 3.5065       |
| N 3                    |
| head 13                |
| cIdx 452               |
| gIdx 0 LRL             |
| a 202.3255 +/- 11.6695 |
| b -0.4366 +/- 0.0218   |
+------------------------+
```

```
+-------Track 16--------+
| chi2/NDOF 0.9108       |
| N 3                    |
| head 16                |
| cIdx 401               |
| gIdx 0 RLR             |
| a 176.4522 +/- 7.5798  |
| b -0.2569 +/- 0.0142   |
+------------------------+
```

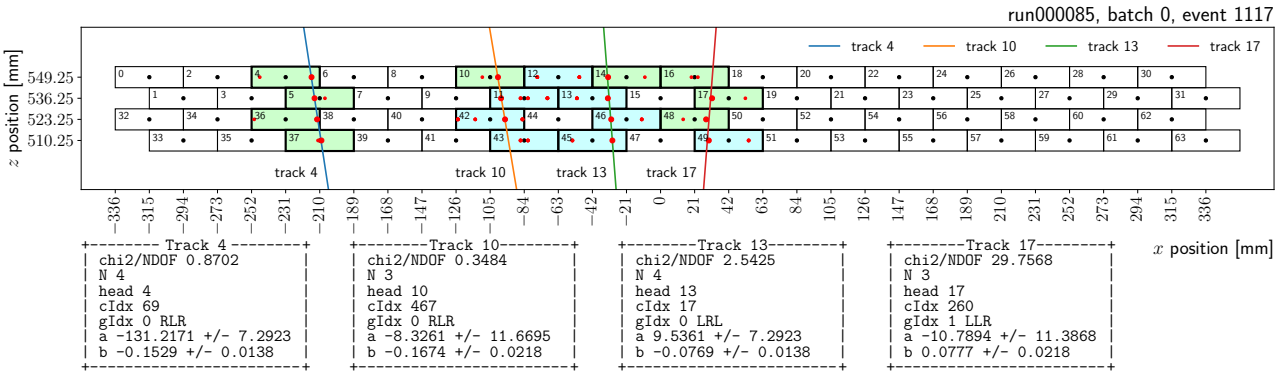



```
+-------- Track 5 --------+
| chi2/NDOF 6.6416       |
| N 4                    |
| head 5                 |
| cIdx 17                |
| gIdx 0 LRL             |
| a -291.3687 +/- 7.2923 |
| b 0.1630 +/- 0.0138    |
+------------------------+
```

```
+-------- Track 7 --------+
| chi2/NDOF 1.0105       |
| N 4                    |
| head 7                 |
| cIdx 145               |
| gIdx 0 LRL             |
| a -106.9992 +/- 7.2923 |
| b -0.1008 +/- 0.0138   |
+------------------------+
```

```
+-------Track 11--------+
| chi2/NDOF 0.2686       |
| N 4                    |
| head 11                |
| cIdx 16                |
| gIdx 2 LRR             |
| a 107.9062 +/- 7.2923  |
| b -0.3303 +/- 0.0138   |
+------------------------+
```

```
+-------Track 14--------+
| chi2/NDOF 1.5417       |
| N 3                    |
| head 14                |
| cIdx 260               |
| gIdx 0 RLR             |
| a 6.4900 +/- 11.3868   |
| b -0.0155 +/- 0.0218   |
+------------------------+
```




```
+-------- Track 4 --------+
| chi2/NDOF 0.8702       |
| N 4                    |
| head 4                 |
| cIdx 69                |
| gIdx 0 RLR             |
| a -131.2171 +/- 7.2923 |
| b -0.1529 +/- 0.0138   |
+------------------------+
```

```
+-------Track 10--------+
| chi2/NDOF 0.3484       |
| N 3                    |
| head 10                |
| cIdx 467               |
| gIdx 0 RLR             |
| a -8.3261 +/- 11.6695  |
| b -0.1674 +/- 0.0218   |
+------------------------+
```

```
+-------Track 13--------+
| chi2/NDOF 2.5425       |
| N 4                    |
| head 13                |
| cIdx 17                |
| gIdx 0 LRL             |
| a 9.5361 +/- 7.2923    |
| b -0.0769 +/- 0.0138   |
+------------------------+
```

```
+-------Track 17--------+
| chi2/NDOF 29.7568      |
| N 3                    |
| head 17                |
| cIdx 260               |
| gIdx 1 LLR             |
| a -10.7894 +/- 11.3868 |
| b 0.0777 +/- 0.0218    |
+------------------------+
```

A.3 GPU vs NumPy track reconstruction

It is interesting to explore whether the track reconstruction algorithm implemented with the CUDA kernels introduces biases or artifacts in the data with respect to a more classical and sequential approach. Also, CUDA kernels operate in `fp32` mode and thus it should be interesting to check if the cumulative effects (algorithmic bias, precision, etc) could produce a reconstructed track distribution that is significantly different from the one that is obtainable with more conventional methods.

In order to address the issue of whether the track reconstruction algorithm implemented with the CUDA kernels introduces biases or artifacts in the data with respect to a more classical and sequential approach, the following protocol was designed:

- Using exactly the same input data, one has two algorithmically independent reconstruction pathways: the CUDA one and another that runs with NumPy.
- In both cases, only 4-points tracks are considered and a cut is imposed on the reduced χ^2 requiring it to be < 5.
- Falkon is used as a microscope, to try and detect eventual incompatibility between the two distributions.

With respect to the latter point, the CUDA reconstructed tracks are taken as the reference, R_{CUDA}, while the NumPy reconstructed tracks are taken as the data, D_{NPY}. The reference-vs-reference test statistic has already been calibrated on the CUDA reconstructed dataset. Defining now the **null hypothesis** H_0 as:

H_0 : `the data points are drawn from the same distribution as the reference points`,

which in essence is like saying that no difference exists between the two reconstruction pathways, it is possible to test the hypothesis thanks to the p-values emitted by Falkon. Further, to better investigate the issue, the test has been conducted with varying degrees of track-angular amplitude.

θ_{MIN}	θ_{MAX}	p-value
-10	10	0.568
-15	15	0.532
-20	20	0.524
-25	25	0.532
-30	30	0.518
-35	35	0.428
-40	40	0.326
-45	45	0.237
-50	50	0.184

Table A.1: Falkon-generated p-values of the REF_{CUDA}-vs-REF_{NPY} test at varying inclusive angle intervals.

The results are summarized in Table A.1. They show an almost flat region for $|\theta| \leq 30°$, signalling statistical equivalence between the tracks reconstructed via the CUDA algorithm and via the classical NumPy approach. Further, when larger intervals are included, the p-value starts to gradually drop to lower values signalling that stronger difference might exist in the way the algorithms reconstruct the tracks at larger angles. Overall there is no statistically significant evidence that supports rejecting the null hypothesis.

Appendix B

Technical notes

B.1 Kernel profilings

This appendix reports the profiler outputs for each of the major kernels that constitute the GPU reconstruction algorithm. The algorithm was run with an aperture of 4096 (meaning processing 4096 candidate events for each launch). The combinatorial kernel is the most compute-intensive one, requiring $\approx$ 6ms to run in 4-points track mode, while the track selector is the most memory-intensive one requiring $\approx 925\mu$s to run also in 4-points track mode. The event builder is again memory intensive and runs in $\approx 17\mu$s. Finally, the track merger kernel is reported for completeness but has not been used since only 4-points operation was considered in this work. A brief summary of the results is shown in Table B.1.

Kernel	**Launch Config.**	**Duration**	**Compute T.put**	**Memory T.put**	**Occupancy Ach. (Theor.)**
combinatorial	(64,4,4096)×(64,4,1)	6.39 ms	80.64%	208.01 GB/s	52.93% (100.0%)
track selector	(1,1,4096)×(32,4,1)	924.7 μs	21.41%	1.31 TB/s	58.00% (62.50%)
event builder	(1,1,4096)×(64,1,1)	16.48 μs	16.77%	445.93 GB/s	30.28% (100.0%)
track merger	(1,1,4096)×(32,2,1)	37.22 μs	40.24%	290.40 GB/s	33.83% (75.0%)

Table B.1: Summary of the output of the kernel profiler for the various kernels. *T.put* is the shortened version of *throughput* and *Ach.* is the shortened for *achieved*.

B.1.1 Combinatorial kernel

4-points and 3-points tracks simultaneous reconstruction

```
combinatorial_more_fast_ext_DEBUG (64, 4, 4096)x(64, 8, 1), Context 1, Stream 7, Device 0, CC 8.0
Section: GPU Speed Of Light Throughput
---------------------- ----------- ------------
Metric Name            Metric Unit Metric Value
---------------------- ----------- ------------
DRAM Frequency                 Ghz         1.21
SM Frequency                   Mhz       765.00
Elapsed Cycles               cycle    9,941,342
Memory Throughput                %        44.44
DRAM Throughput                  %        13.22
Duration                        ms        13.00
L1/TEX Cache Throughput          %        44.46
L2 Cache Throughput              %        16.43
SM Active Cycles             cycle 9,936,814.49
Compute (SM) Throughput          %        79.38
---------------------- ----------- ------------
```

```
Section: Memory Workload Analysis
---------------------------- ----------- ------------
Metric Name                  Metric Unit Metric Value
---------------------------- ----------- ------------
Memory Throughput                Gbyte/s       205.54
Mem Busy                               %        44.44
Max Bandwidth                          %        40.95
L1/TEX Hit Rate                        %        72.13
L2 Compression Success Rate            %            0
L2 Compression Ratio                                0
L2 Compression Input Sectors      sector            0
L2 Hit Rate                            %        99.86
Mem Pipes Busy                         %        40.95
---------------------------- ----------- ------------

Section: Occupancy
------------------------------ ----------- ------------
Metric Name                    Metric Unit Metric Value
------------------------------ ----------- ------------
Block Limit SM                       block           32
Block Limit Registers                block            4
Block Limit Shared Mem               block            8
Block Limit Warps                    block            4
Theoretical Active Warps per SM       warp           64
Theoretical Occupancy                    %          100
Achieved Occupancy                       %        71.79
Achieved Active Warps Per SM          warp        45.94
------------------------------ ----------- ------------

Section: Source Counters
------------------------- ----------- ------------
Metric Name               Metric Unit Metric Value
------------------------- ----------- ------------
Branch Instructions Ratio           %         0.08
Branch Instructions              inst  253,096,148
Branch Efficiency                   %        98.77
Avg. Divergent Branches                  4,190.00
------------------------- ----------- ------------
```

4-points tracks only operation

```
combinatorial_more_fast_ext_DEBUG (64, 4, 4096)x(64, 4, 1), Context 1, Stream 7, Device 0, CC 8.0
Section: GPU Speed Of Light Throughput
---------------------- ----------- ------------
Metric Name            Metric Unit Metric Value
---------------------- ----------- ------------
DRAM Frequency                 Ghz         1.21
SM Frequency                   Mhz       765.00
Elapsed Cycles               cycle    4,886,689
Memory Throughput                %        45.85
DRAM Throughput                  %        13.38
Duration                        ms         6.39
L1/TEX Cache Throughput          %        45.89
L2 Cache Throughput              %        16.80
SM Active Cycles             cycle 4,882,724.71
Compute (SM) Throughput          %        80.64
---------------------- ----------- ------------

Section: Memory Workload Analysis
---------------------------- ----------- ------------
Metric Name                  Metric Unit Metric Value
---------------------------- ----------- ------------
Memory Throughput                Gbyte/s       208.01
Mem Busy                               %        45.85
Max Bandwidth                          %        41.56
L1/TEX Hit Rate                        %        68.30
L2 Compression Success Rate            %            0
L2 Compression Ratio                                0
L2 Compression Input Sectors      sector            0
L2 Hit Rate                            %        99.72
Mem Pipes Busy                         %        41.56
---------------------------- ----------- ------------

Section: Occupancy
------------------------------ ----------- ------------
```

```
Metric Name                     Metric Unit Metric Value
------------------------------- ----------- ------------
Block Limit SM                        block           32
Block Limit Registers                 block            8
Block Limit Shared Mem                block            8
Block Limit Warps                     block            8
Theoretical Active Warps per SM        warp           64
Theoretical Occupancy                     %          100
Achieved Occupancy                        %        52.93
Achieved Active Warps Per SM           warp        33.87
------------------------------- ----------- ------------

Section: Source Counters
------------------------- ----------- ------------
Metric Name               Metric Unit Metric Value
------------------------- ----------- ------------
Branch Instructions Ratio           %         0.08
Branch Instructions              inst  125,189,255
Branch Efficiency                   %        98.80
Avg. Divergent Branches                  2,028.18
------------------------- ----------- ------------
```

B.1.2 Track selection kernel

4-points and 3-points tracks simultaneous reconstruction

```
  get_tracks_imp_ext_DEBUG (1, 1, 4096)x(32, 8, 1), Context 1, Stream 7, Device 0, CC 8.0
Section: GPU Speed Of Light Throughput
---------------------- ----------- ------------
Metric Name            Metric Unit Metric Value
---------------------- ----------- ------------
DRAM Frequency                 Ghz         1.21
SM Frequency                   Mhz       764.99
Elapsed Cycles               cycle    1,397,835
Memory Throughput                %        85.45
DRAM Throughput                  %        85.45
Duration                        ms         1.83
L1/TEX Cache Throughput          %        25.19
L2 Cache Throughput              %        92.65
SM Active Cycles             cycle 1,387,289.92
Compute (SM) Throughput          %        21.85
---------------------- ----------- ------------

Section: Memory Workload Analysis
--------------------------- ----------- ------------
Metric Name                 Metric Unit Metric Value
--------------------------- ----------- ------------
Memory Throughput              Tbyte/s         1.33
Mem Busy                             %        54.01
Max Bandwidth                        %        85.45
L1/TEX Hit Rate                      %            0
L2 Compression Success Rate          %            0
L2 Compression Ratio                              0
L2 Compression Input Sectors      sector            0
L2 Hit Rate                          %        24.95
Mem Pipes Busy                       %        13.60
--------------------------- ----------- ------------

Section: Occupancy
------------------------------- ----------- ------------
Metric Name                     Metric Unit Metric Value
------------------------------- ----------- ------------
Block Limit SM                        block           32
Block Limit Registers                 block            5
Block Limit Shared Mem                block           10
Block Limit Warps                     block            8
Theoretical Active Warps per SM        warp           40
Theoretical Occupancy                     %        62.50
Achieved Occupancy                        %        59.20
Achieved Active Warps Per SM           warp        37.89
------------------------------- ----------- ------------

Section: Source Counters
------------------------- ----------- ------------
Metric Name               Metric Unit Metric Value
```

```
------------------------- ----------- ------------
Branch Instructions Ratio           %         0.08
Branch Instructions              inst   10,273,768
Branch Efficiency                   %        85.89
Avg. Divergent Branches                   2,250.14
------------------------- ----------- ------------
```

4-points tracks only operation

```
get_tracks_imp_ext_DEBUG (1, 1, 4096)x(32, 4, 1), Context 1, Stream 7, Device 0, CC 8.0
Section: GPU Speed Of Light Throughput
---------------------- ----------- ------------
Metric Name             Metric Unit Metric Value
---------------------- ----------- ------------
DRAM Frequency                  Ghz         1.21
SM Frequency                    Mhz       764.98
Elapsed Cycles                cycle      707,388
Memory Throughput                 %        84.46
DRAM Throughput                   %        84.46
Duration                         us       924.70
L1/TEX Cache Throughput           %        25.09
L2 Cache Throughput               %        91.37
SM Active Cycles              cycle   696,465.71
Compute (SM) Throughput           %        21.41
---------------------- ----------- ------------

Section: Memory Workload Analysis
--------------------------- ----------- ------------
Metric Name                 Metric Unit Metric Value
--------------------------- ----------- ------------
Memory Throughput               Tbyte/s         1.31
Mem Busy                              %        53.37
Max Bandwidth                         %        84.46
L1/TEX Hit Rate                       %            0
L2 Compression Success Rate           %            0
L2 Compression Ratio                                0
L2 Compression Input Sectors      sector            0
L2 Hit Rate                           %        24.85
Mem Pipes Busy                        %        13.44
--------------------------- ----------- ------------

Section: Occupancy
------------------------------ ----------- ------------
Metric Name                    Metric Unit Metric Value
------------------------------ ----------- ------------
Block Limit SM                       block           32
Block Limit Registers                block           10
Block Limit Shared Mem               block           10
Block Limit Warps                    block           16
Theoretical Active Warps per SM       warp           40
Theoretical Occupancy                    %        62.50
Achieved Occupancy                       %        58.00
Achieved Active Warps Per SM          warp        37.12
------------------------------ ----------- ------------

Section: Source Counters
------------------------- ----------- ------------
Metric Name               Metric Unit Metric Value
------------------------- ----------- ------------
Branch Instructions Ratio           %         0.07
Branch Instructions              inst    4,864,013
Branch Efficiency                   %        93.77
Avg. Divergent Branches                     496.79
------------------------- ----------- ------------
```

B.1.3 Event builder kernel

```
get_event_ext_FALKON (1, 1, 4096)x(64, 1, 1), Context 1, Stream 7, Device 0, CC 8.0
Section: GPU Speed Of Light Throughput
---------------------- ----------- ------------
Metric Name             Metric Unit Metric Value
---------------------- ----------- ------------
DRAM Frequency                  Ghz         1.21
```

```
SM Frequency                    Mhz       762.89
Elapsed Cycles                cycle       12,577
Memory Throughput                 %        28.77
DRAM Throughput                   %        28.77
Duration                         us        16.48
L1/TEX Cache Throughput           %        25.56
L2 Cache Throughput               %        45.26
SM Active Cycles              cycle     8,731.90
Compute (SM) Throughput           %        16.77
----------------------- ----------- ------------

Section: Memory Workload Analysis
---------------------------- ----------- ------------
Metric Name                  Metric Unit Metric Value
---------------------------- ----------- ------------
Memory Throughput                Gbyte/s       445.93
Mem Busy                               %        23.37
Max Bandwidth                          %        28.77
L1/TEX Hit Rate                        %            0
L2 Compression Success Rate            %            0
L2 Compression Ratio                                0
L2 Compression Input Sectors      sector            0
L2 Hit Rate                            %        39.68
Mem Pipes Busy                         %        15.23
---------------------------- ----------- ------------

Section: Occupancy
------------------------------ ----------- ------------
Metric Name                    Metric Unit Metric Value
------------------------------ ----------- ------------
Block Limit SM                       block           32
Block Limit Registers                block           32
Block Limit Shared Mem               block           40
Block Limit Warps                    block           32
Theoretical Active Warps per SM       warp           64
Theoretical Occupancy                    %          100
Achieved Occupancy                       %        30.28
Achieved Active Warps Per SM          warp        19.38
------------------------------ ----------- ------------

Section: Source Counters
------------------------- ----------- ------------
Metric Name               Metric Unit Metric Value
------------------------- ----------- ------------
Branch Instructions Ratio           %         0.05
Branch Instructions              inst       44,372
Branch Efficiency                   %        34.69
Avg. Divergent Branches                      18.28
------------------------- ----------- ------------
```

B.1.4 Track merger kernel

```
paster (1, 1, 4096)x(32, 2, 1), Context 1, Stream 7, Device 0, CC 8.0
Section: GPU Speed Of Light Throughput
----------------------- ----------- ------------
Metric Name             Metric Unit Metric Value
----------------------- ----------- ------------
DRAM Frequency                  Ghz         1.21
SM Frequency                    Mhz       764.19
Elapsed Cycles                cycle       28,448
Memory Throughput                 %        20.80
DRAM Throughput                   %        18.71
Duration                         us        37.22
L1/TEX Cache Throughput           %        24.37
L2 Cache Throughput               %        23.49
SM Active Cycles              cycle    24,281.29
Compute (SM) Throughput           %        40.24
----------------------- ----------- ------------

Section: Memory Workload Analysis
---------------------------- ----------- ------------
Metric Name                  Metric Unit Metric Value
---------------------------- ----------- ------------
Memory Throughput                Gbyte/s       290.40
Mem Busy                               %        20.80
```

```
Max Bandwidth                            %        18.71
L1/TEX Hit Rate                          %            0
L2 Compression Success Rate              %            0
L2 Compression Ratio                                  0
L2 Compression Input Sectors      sector              0
L2 Hit Rate                              %        21.19
Mem Pipes Busy                           %        21.13
---------------------------- ----------- ------------

Section: Occupancy
------------------------------ ----------- ------------
Metric Name                    Metric Unit Metric Value
------------------------------ ----------- ------------
Block Limit SM                       block           32
Block Limit Registers                block           24
Block Limit Shared Mem               block           40
Block Limit Warps                    block           32
Theoretical Active Warps per SM       warp           48
Theoretical Occupancy                    %           75
Achieved Occupancy                       %        33.83
Achieved Active Warps Per SM          warp        21.65
------------------------------ ----------- ------------

Section: Source Counters
------------------------- ----------- ------------
Metric Name               Metric Unit Metric Value
------------------------- ----------- ------------
Branch Instructions Ratio           %         0.18
Branch Instructions              inst      894,601
Branch Efficiency                   %        86.80
Avg. Divergent Branches                     168.49
------------------------- ----------- ------------
```

B.2 FalkonML modifications

B.2.1 RMM allocator related changes

falkon/utils/devices.py At the beginning added

```
1  import rmm
```

Then, after `_get_cpu_device_info` function, the following functions have been added at Line 93 and Line 110 and Line 116 in order to make Falkon able to retrieve the memory information through the not yet implemented interface on PyTorch's PluggableAllocator:

```
1  def _rmm_find_resource_with_attr(attr_name):
2      """Walk the RMM resource chain and return the first resource exposing attr_name."""
3      res = rmm.mr.get_current_device_resource()
4      seen = set()
5      while res is not None:
6          obj_id = id(res)
7          if obj_id in seen:
8              break
9          seen.add(obj_id)
10         if hasattr(res, attr_name):
11             return res
12         if hasattr(res, "get_upstream"):
13             res = res.get_upstream()
14         else:
15             break
16     return None
17
18 def _rmm_memory_allocated() -> int:
19     res = _rmm_find_resource_with_attr("get_allocated_bytes")
20     if res is None:
21         raise RuntimeError("RMM resource does not expose get_allocated_bytes().")
```

```
    return int(res.get_allocated_bytes())

def _rmm_memory_reserved() -> int:
    res = _rmm_find_resource_with_attr("pool_size")
    if res is None:
        raise RuntimeError("RMM resource does not expose pool_size().")
    return int(res.pool_size())
```

Then, in function `_get_gpu_device_info` the following line has been added after `tcd.init()`:

```
torch_backend = tcd.memory.get_allocator_backend()
```

and at Line 167, the following has been added to avoid Torch's calls to not implemented functions of the PluggableAllocator:

```
if torch_backend != 'native':
    try:
        rmm_alloc = _rmm_memory_allocated()
        rmm_reserved = _rmm_memory_reserved()
        # in this case, we use the custom functions to give reserved
        # and allocated memory (Torch does not support a full interface
        # for CUDAPluggableAllocator
        cached_free_mem = max(0, rmm_reserved - rmm_alloc)
    except Exception as e:
        # Fallback: if RMM cannot provide info, avoid crashing
        warning.warn(f"RMM memory query failed: {e}")
        cached_free_mem = 0
else:
    # this is the original code
    cached_free_mem = tcd.memory_reserved(g) - tcd.memory_allocated(g)
```

B.2.2 GPU-residency related

falkon/options.py Promoted all numpy constants to torch scalars, with `device=cuda`. Also, added `verbose: bool = True` flag to enable/disable printing out status messages.

falkon/model_utils.py Edited `FalkonBase` class by adding protected method to wrap `print`:

```
def _log(self, msg, flush=True, end="\n"):
    if (self.options.verbose):
        print(msg, flush=flush, end=end)
```

then substituted raw calls to `print` with `self._log`.

falkon/la_helpers/wrapper.py Edit on Line 107, function `trsm`. Rewritten as:

```
def trsm(v: torch.Tensor, A: torch.Tensor, alpha: float, lower: int = 0, transpose: int = 0)
    -> torch.Tensor:
if isinstance(A, torch.Tensor):
    if isinstance(v, torch.Tensor):
        if not check_same_device(A, v):
            raise ValueError("A and v must be on the same device.")
        if A.is_cuda and v.is_cuda:
            from falkon.la_helpers.cuda_trsm import cuda_trsm
            return cuda_trsm(A, v, alpha, bool(lower), bool(transpose))
        else:
            A = A.numpy()
            v = v.numpy()
        else:  # v is numpy array (thus CPU)
            if A.is_cuda:
                raise ValueError("A and v must be on the same device.")
```

```
            A = A.numpy()
    vout = cpu_trsm(A, v, alpha, lower, transpose)
    return torch.from_numpy(vout)
```

falkon/models/logistic_falkon.py Edit on Line 185, modified memory pinning request to allow it only for host-bound tensors. Rewritten as:

```
ny_X, ny_Y = self.center_selection.select(X, Y)
beta_it = None
if self.use_cuda_ and ny_X.device.type=='cpu':
    ny_X = ny_X.pin_memory()
    ny_Y = ny_Y.pin_memory()
    beta_it = torch.zeros(ny_X.shape[0], 1, dtype=X.dtype)  # Temporary iterative solution
else:
    beta_it = torch.zeros(ny_X.shape[0], 1, dtype=X.dtype, device='cuda')
```

while the old code was:

```
if self.use_cuda_:
    ny_X = ny_X.pin_memory()
    ny_Y = ny_Y.pin_memory()
    beta_it = torch.zeros(ny_X.shape[0], 1, dtype=X.dtype)  # Temporary iterative solution
```

Also, corrected a minor bug at Line 246, where:

```
print(f"Iteration {out_iter:d} - penalty {penalty:e} - sub-iterations max_iter:d", flush=
    True)
```

was changed to:

```
self._log(f"Iteration {out_iter:d} - penalty {penalty:e} - sub-iterations {max_iter:d}",
    flush=True)
```

falkon/optim/conjgrad.py Edit on Line 107, changed tol variable declaration and initialization to:

```
tol = torch.from_dlpack(self.params.cg_tolerance**2)
```

since now `self.params` is a CuPy ndarray and not a numpy array anymore.

Changed the base class `Optimizer` which was empty, in order to implement the `_log` method here as well:

```
class Optimizer:
    """Base class for optimizers. This is an empty shell at the moment."""

    def __init__(self, params):
        self.params = params

    def _log(self, msg, flush=True, end="\n"):
        if (self.params.verbose):
            print(msg, flush=flush, end=end)
```

and changing the initialization slightly in both `ConjugateGradient` and `FalkonConjugateGradient` so that now:

```
super().__init__(opt)
```

the base class `Optimizer` is initialized passing the option `object`. Then substituted the print calls to `self._log`.

falkon/preconditioner/logistic_preconditioner.py Edit on Line 85, rewriting of the `_trmm` function:

```
def _trmm(self, C: torch.Tensor, alpha: torch.Tensor) -> torch.Tensor:
    if C.is_cuda and alpha.is_cuda:
        import cupy as cp
        C_cp = cp.from_dlpack(torch.utils.dlpack.to_dlpack(C))
        alpha_cp = cp.from_dlpack(torch.utils.dlpack.to_dlpack(alpha))
        out_cp = C_cp.T @ alpha_cp
        return torch.utils.dlpack.from_dlpack(out_cp)
    else:
        alpha_np = alpha.numpy()
        if not alpha_np.flags.f_contiguous:
            alpha_np = np.copy(alpha_np, order="F")
        trmm = choose_fn(C.dtype, sclb.dtrmm, sclb.strmm, "TRMM")
        out = trmm(alpha=1.0, a=C.numpy(), b=alpha_np, side=0, lower=0, trans_a=1, diag=0,
    overwrite_b=1)
        return torch.from_numpy(out)
```

while the old code was:

```
def _trmm(self, C: torch.Tensor, alpha: torch.Tensor) -> torch.Tensor:
    alpha_np = alpha.numpy()
    if not alpha_np.flags.f_contiguous:
        # This never happens since alpha is always 1D
        alpha_np = np.copy(alpha_np, order="F")

    trmm = choose_fn(C.dtype, sclb.dtrmm, sclb.strmm, "TRMM")
    out = trmm(alpha=1.0, a=C.numpy(), b=alpha_np, side=0, lower=0, trans_a=1, diag=0,
    overwrite_b=1)
    return torch.from_numpy(out)
```

falkon/center_selection.py In function `select_indices` of `UniformSelector`, Line 241 writing:

```
th_idx = torch.from_dlpack(idx.astype(cp.int64))
torch.index_select(X, dim=0, index=th_idx, out=Xc)
```

instead of the old code:

```
th_idx = torch.from_numpy(idx.astype(np.int64)).to(X.device)
```

falkon/utils/init.py Definition of new function to make use of CuPy random number generator instead of NumPy's:

```
def check_random_generator(seed):
    """Turn seed into a CuPy random generator (GPU resident).

    Parameters
    ----------
    seed : None | int | cupy.random.Generator
    If seed is None, return the global CuPy RNG.
    If seed is an int, set CuPy's RNG seed and return the global RNG.
    If seed is already a CuPy RNG, return it.
    Otherwise raise ValueError.
    """
    # None or cp.random => return global CuPy RNG
    if seed is None or seed is cp.random:
        return cp.random

    # Integer seed => set the global CuPy RNG seed, return the module
    if isinstance(seed, numbers.Integral):
```

```
        cp.random.seed(seed)
        return cp.random

    # Already a CuPy Generator (less common)
    if hasattr(cp.random, "Generator") and isinstance(seed, cp.random.Generator):
        return seed

    raise ValueError(f"{seed!r} cannot be used to seed a cupy.random.Generator instance")
```

falkon/gsc_losses.py Lines 376 and 377, edited to use the appropriate device instead of forcing back to cpu.

```
class1 = true * torch.log(torch.tensor(1, dtype=pred.dtype, device=pred.device) + torch.exp(-pred)) # EDIT
class0 = self.neg_weight * (1 - true) * torch.log(torch.tensor(1, dtype=pred.dtype, device=pred.device) + torch.exp(pred)) # EDIT
```

falkon/hopt/objectives/transforms.py Line 31 changed to:

```
threshold = torch.log(torch.tensor(torch.finfo(y.dtype).eps, dtype=y.dtype, device='cuda')) + torch.tensor(2.0, dtype=y.dtype, device='cuda')
```

falkon/kernels/distance_kernel.py Line 36 changed to:

```
    return torch.tensor([float(sigma)], dtype=torch.float64, device='cuda')
```

Appendix C

Pipeline monitoring dashboard

The pipeline monitoring dashboard is an application that consists in two essential components: a back-end written in Python (a Flask application) and a front-end which is a web page that, through Javascript and the usage of socketIO, receives events from the back-end. The pipeline program streams the data to the back-end of the dashboard application via a Redis stream.

C.1 Main features

A screenshot of the dashboard in operation is shown in Figure C.1.

Time traces for the test statistics and p-values The dashboard shows the time-traces of the test statistic and the p-values associated to them in the upper area of the screen. Under the null hypothesis (data compatible with reference) the p-values are expected to be uniformly distributed between 0 and 1 and this is shown on the time plot at the right. When an anomaly is detected, the p-values are strongly pushed towards very small values as the figure shows.

Below the right time-plot of the p-values, a dynamical histogram shows the distributions of the p-values. This is expected to fluctuate around the uniform. Since there are 10 bins, each bin is expected to contain 0.1 of the total density. Below the left time-plot, the histogram of the test statistic is shown.

Detector status and drift times monitoring At the bottom row to the left, one can assess the detector occupancy status. Each cell reports a measure of the relative importance of each cell with respect to the particle tracks that have been reconstructed for a given input batch. The more "trafficked" a cell is and the more towards the red its fill color will shift.

At the bottom right instead, one can see the drift times histogram for each layer. These histograms show the difference, bin-per-bin, between the reference drift times distributions and the ones that have been extracted from the input data batch. When an input batch is compatible with the reference, these histogram show an almost-exclusively "green status". When instead an anomalous batch is detected, such as the case shown in the figure, the drift times differences will appear in these plots.

C.2 Remarks

This dashboard application constitutes a prototype. It has been developed for the purpose of testing the online operation of the pipeline.

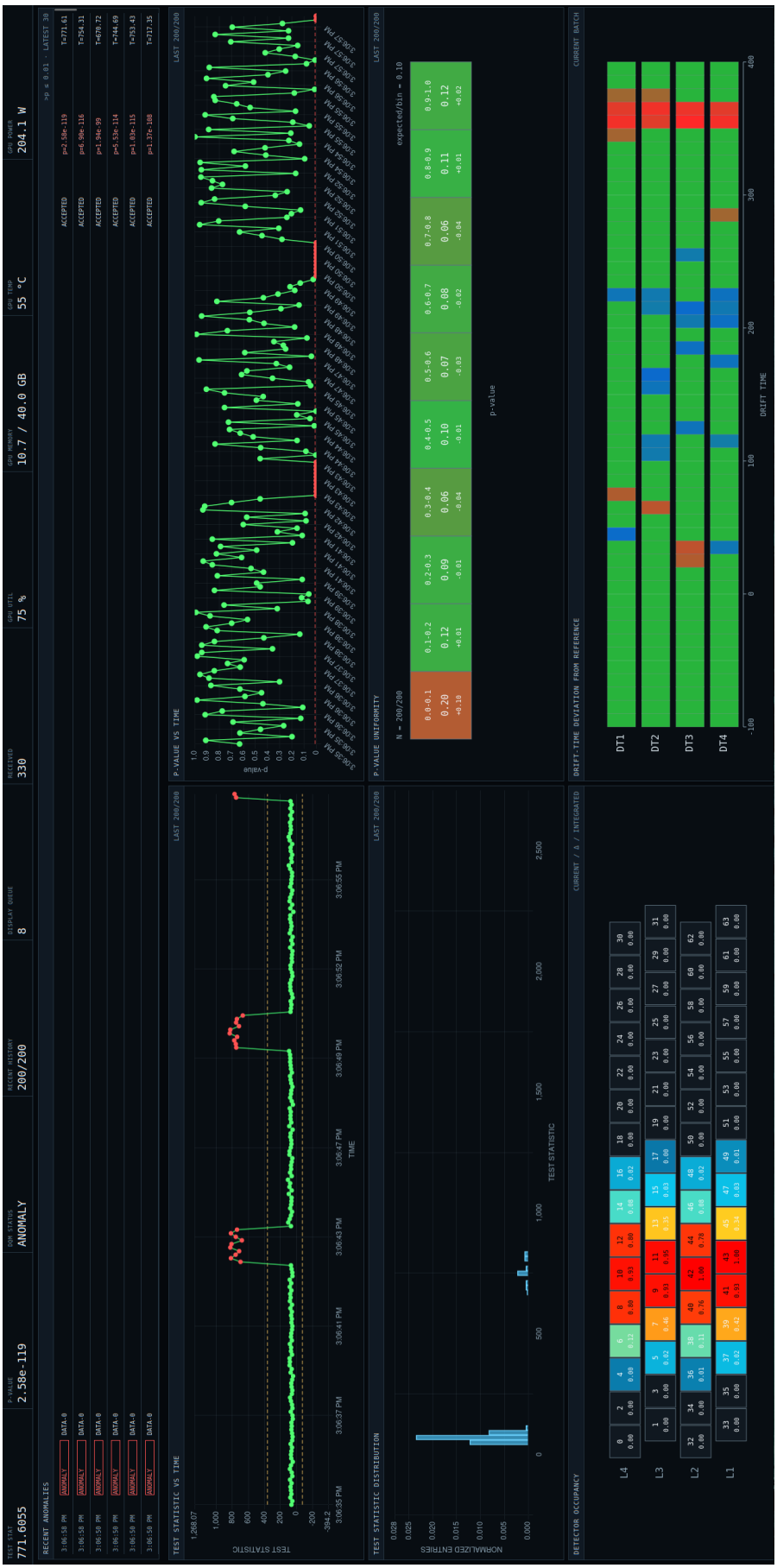


Figure C.1: Dashboard in operation, showing the detection of an anomalous batch.